Beyond Individual Intelligence: A Survey of Multi-agent Collaboration, Attribution and Evolution

IEEE Publication Technology, *Staff, IEEE,*

CONTENTS

I Introduction 2

II Individual Intelligence: Core Capabilities and Boundary Analysis 3
- II-A Overview: From LLM to LLM-based Agent . . . 3

III Reasoning 4
- III-A Input-Stage Enhancement . . . 4
  - III-A1 Knowledge Augmentation . . . 4
  - III-A2 Modality Augmentation . . . 5
- III-B Reasoning-Process Enhancement . . . 5
  - III-B1 Search Space Expansion . . . 5
  - III-B2 Reasoning Path Verification . . . 5
- III-C Output-Stage Regulation . . . 5
  - III-C1 Output reliability assessment . . . 5
  - III-C2 Response Behavior Regulation . . . 5

IV Memory 6
- IV-A Memory Formation . . . 6
  - IV-A1 Direct Recording . . . 6
  - IV-A2 Abstractive Distillation . . . 6
  - IV-A3 Typed Routing . . . 6
- IV-B Memory Maintenance . . . 6
  - IV-B1 Storage Structure . . . 7
  - IV-B2 Dynamic Maintenance . . . 8
- IV-C Memory Retrieval and Utilization . . . 8
  - IV-C1 Retrieval Mechanisms . . . 8
  - IV-C2 Integration Patterns . . . 8

V Planning 8
- V-A Decomposition-Based Planning . . . 8
  - V-A1 Proactive Decomposition . . . 8
  - V-A2 Progressive Decomposition . . . 9
- V-B Search-Based Planning . . . 9
  - V-B1 Step-Level Search . . . 10
  - V-B2 Plan-Level Search . . . 10

VI Tool Use 10
- VI-A Tool Capability Acquisition . . . 10
  - VI-A1 Trajectory-Based Learning . . . 10
  - VI-A2 Protocol-Based Alignment . . . 10
- VI-B Tool Invocation . . . 10
  - VI-B1 One-Shot Calling . . . 11
  - VI-B2 Closed-Loop Calling . . . 11
  - VI-B3 Workflow-Based Orchestration 11
- VI-C Tool Generalization . . . 11
  - VI-C1 Unseen Tool Generalization . . . 11
  - VI-C2 Tool Discovery and Retrieval 12

VII Multi-Agent Collaboration 12
- VII-A Role . . . 13
  - VII-A1 Role Capability . . . 13
  - VII-A2 Role Allocation . . . 14
- VII-B Communication . . . 15
  - VII-B1 Communication Modes . . . 15
  - VII-B2 Communication Protocols . . . 16
- VII-C Orchestration . . . 17
  - VII-C1 Centralized Orchestration Topology . . . 17
  - VII-C2 Distributed Orchestration Topology . . . 17
  - VII-C3 Hybrid Orchestration Topology 18
- VII-D Interaction . . . 18
  - VII-D1 Information Flow . . . 18
  - VII-D2 Interaction Patterns . . . 19
- VII-E Evaluation . . . 19
- VII-F Discussion . . . 20

VIII Multi-agent anomaly attribution 20
- VIII-A Formal definition . . . 21
  - VIII-A1 Task Inputs and Objectives . . . 21
  - VIII-A2 Formal Expression of the Attribution Process . . . 21
- VIII-B Multi-agent anomaly classification . . . 21
- VIII-C Anomaly Attribution Taxonomy . . . 22
  - VIII-C1 Data-driven method . . . 22
  - VIII-C2 Rule-driven method . . . 24
  - VIII-C3 Causal inference method . . . 24
- VIII-D Anomaly Attribution Evaluation . . . 25
- VIII-E Existing challenges and future research directions . . . 26
  - VIII-E1 Core challenges . . . 26
  - VIII-E2 Future research direction . . . 26

IX Multi-Agent System Self-Evolution 26
- IX-A Motivation . . . 26
- IX-B From Attribution to Evolution . . . 27
- IX-C Formalizing Multi-Agent System Self-Evolution . . . 27
- IX-D MAS Self-Evolution Taxonomy . . . 28
  - IX-D1 Micro-Level: Agent Self-Evolution . . . 28

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

IX-D2 Meso-level: Organizational Self-Evolution 29
IX-D3 Macro-level: Systemic Self-Evolution 29

X Conclusion 29

Appendix: Proof of the Zonklar Equations 29

A References Section 29

A Simple References 29

References 29

***Abstract*—This document describes the most common article elements and how to use the IEEEtran class with LaTeX to produce files that are suitable for submission to the IEEE. IEEEtran can produce conference, journal, and technical note (correspondence) papers with a suitable choice of class options.**

***Index Terms*—Article submission, IEEE, IEEEtran, journal, LaTeX, paper, template, typesetting.**

I. Introduction

Large language models (LLMs) have rapidly evolved from fluent text generators into systems capable of complex reasoning, long-horizon planning, and interaction with external environments [1, 2, 3, 4]. This progression has been accompanied by the emergence of chain-of-thought capabilities [5, 6], advances in scalable instruction tuning [7], and reinforcement learning–driven reasoning [8], which together have substantially expanded the ability of LLMs to tackle mathematical problem-solving, competitive programming, and professional examinations [8, 9]. Despite these advances, individual LLMs still struggle to maintain coherent behavior over extended interactions, adapt to rapidly changing task conditions, and coordinate multi-step actions that span diverse tools and environments [10, 11]. These gaps become especially pronounced in real-world deployments that demand sustained autonomy, including autonomous software engineering [12, 13], AI-driven scientific discovery [14, 15], and embodied robotic control [16, 17], where success hinges not on a single inference pass but on persistent observation, iterative decision-making, and closed-loop adaptation. This recognition has catalyzed the emergence of LLM-based autonomous agents: systems that augment foundation models with explicit modules for perception, memory, planning, and tool use, operating through continuous cycles of reasoning, action, and reflection [10, 11, 18, 19].

While these capabilities have significantly raised the ceiling of what a single agent can accomplish, a fundamental tension emerges as tasks grow in scope and duration: the diverse competencies they demand become increasingly difficult to concentrate within a single agent without encountering conflicts in prompt design, context allocation, and behavioral objectives [20]. Multi-agent collaboration offers a principled resolution by decomposing complex tasks into specialized roles that operate through structured communication and coordinated orchestration [21, 22]. Recent systems have moved well beyond early explorations: role-specialized agent teams now collaboratively solve tasks that require sustained multi-step coordination across planning, execution, and verification [23, 24], while standardized communication protocols such as MCP and A2A are beginning to establish the infrastructure for open, interoperable agent ecosystems [25, 26]. These developments signal a shift from building individual capable agents to engineering the organizational structures that govern how agents coordinate at scale. Yet this very capacity for tight coordination introduces a less examined consequence: as agents become more deeply interdependent, the system's collective behavior grows increasingly difficult to predict, diagnose, and improve when failures occur.

In tightly coupled multi-agent systems, a localized error such as a hallucinated fact, a misrouted message, or an incorrect tool invocation can propagate through successive interaction rounds, triggering cascading failures that obscure the original root cause [20, 27]. As execution trajectories grow longer and inter-agent dependencies deepen, the causal chain linking a fault to its downstream consequences becomes increasingly opaque, rendering manual identification of the responsible agent and the decisive step both inefficient and unscalable [28, 29, 30]. Meanwhile, even when root causes are successfully identified, existing multi-agent systems largely lack the ability to translate diagnostic insights into structural adaptation, whether by reorganizing coordination topologies, revising role assignments, or refining collaboration policies in light of observed failure patterns [31, 32]. These two deficits in attribution and in self-correction are deeply intertwined: without reliable diagnosis, improvement efforts lack direction; without the ability to act on diagnostic results, attribution yields limited practical value [33]. This coupling exposes a fundamental asymmetry: the capacity to build increasingly sophisticated multi-agent systems has far outpaced the machinery available for understanding why they fail and enabling them to improve autonomously.

The growing significance of LLM-based agent systems has prompted a substantial body of survey work, yet existing reviews remain compartmentalized along the boundaries identified above. One line of surveys systematically examines the core capabilities of individual agents, covering reasoning, memory, planning, and tool use as modular components of a single decision-making system [10, 11, 34, 35, 36]. A second line shifts focus to multi-agent collaboration, analyzing how role specialization, communication protocols, orchestration topologies, and interaction patterns enable collective problem-solving [22, 37, 38, 39, 40, 41]. A third, more recent line addresses agent self-evolution, investigating how agents can autonomously refine their prompts, memory, tools, and workflows through environmental feedback [31, 42]. These lines collectively cover considerable ground, but they leave the post-deployment failure loop largely unexamined. Notably, no existing survey provides a systematic treatment of multi-agent anomaly attribution, despite its emergence as a distinct and rapidly growing research area [20, 27, 33]. More fundamentally, no prior work has examined the complete operational lifecycle of multi-agent systems as an integrated whole: from foundational single-agent capabilities, through collaborative organization, to failure diagnosis and autonomous structural improvement. This survey is motivated by precisely this gap.

To the best of our knowledge, this is the first survey that covers the complete operational lifecycle of LLM-based multi-agent systems. Rather than treating individual capabilities, collaboration, attribution, and evolution as independent topics, we organize them as a causally linked progression: individual capabilities define what each agent can contribute; collaboration design determines how agents coordinate; attribution diagnoses where and why coordination breaks down; and evolution translates diagnostic outcomes into structural refinement.

Contributions

- **Lifecycle-oriented organizational framework.** We propose a unified analytical perspective that connects individual agent capabilities, multi-agent collaboration mechanisms, runtime anomaly attribution, and system self-evolution into a coherent progression, revealing cross-stage dependencies that prior surveys have treated in isolation.
- **Comprehensive research resources.** We compile and maintain a publicly available collection of structured literature, taxonomy visualizations, and comparative tables to support continued research and broader community engagement.

II. Individual Intelligence: Core Capabilities and Boundary Analysis

A. Overview: From LLM to LLM-based Agent

While large language models have demonstrated remarkable capabilities in language understanding and generation [43, 44], their limitations become evident in complex real-world scenarios such as automated software development [12], scientific experiment design [15], and multi-step web navigation [45], where LLMs operate as stateless response generators that lack persistent memory, cannot actively perceive or act upon external environments, and remain confined to the static knowledge encoded during training. These constraints motivate the transition from reactive text generators to LLM-based autonomous agents [10], which extend the foundation model with the ability to perceive environmental states, formulate goals, execute actions through external tools, and adapt behavior based on feedback—operating through an iterative loop of observation, reasoning, action, and reflection [18, 19]. To systematically characterize the capabilities that enable such autonomy, we organize the core capabilities into four dimensions: *reasoning*, *memory*, *planning*, and *tool use* [10, 46]. Formally, we model a single LLM-based agent as a decision-making system that, at each step t, selects an action based on the current input and accumulated experience, executes it through external tools, and updates its memory with the resulting interaction record $h_t = (x_t, a_t, o_t)$:

$$\begin{aligned} a_t &= \pi(x_t,\, M_t), \\ o_t &= T(a_t), \\ M_{t+1} &= M.\mathrm{update}(M_t,\, h_t), \end{aligned} \tag{1}$$

where π denotes the decision function conditioned on memory state M_t, and T the tool execution interface. Within this cycle, each capability assumes a distinct role:

- **Reasoning** R: maps the current input and context to an intermediate judgment, $r = R(x_t,\, c_t)$;
- **Memory** M: a persistent state module that provides context to reasoning and planning via $c_t = M.\text{read}(q_t)$ and is updated after each step via $M_{t+1} = M.\text{update}(M_t,\, h_t)$;
- **Planning** P: converts a high-level goal g and memory-derived context into an action sequence, $\mathbf{a} = P(g,\, c_t)$;
- **Tool Use** T: executes an action and returns an observation, $o = T(a)$, whose result enters h_t and is subsequently written to memory.

These four capabilities do not operate in isolation; rather, they interact dynamically throughout task execution: reasoning guides tool selection, planning coordinates multi-step tool invocations, and memory preserves intermediate results and learned strategies to inform future decisions.

Formally, we model an LLM-based agent as a structured sequential decision-making system with explicit internal functional decomposition. The agent operates over discrete time steps $t = 1, 2, \ldots$, and is defined by the tuple

$$\mathcal{G} = (\mathcal{X}, \mathcal{U}, \mathcal{O}, \mathcal{M}, R, P, T), \tag{2}$$

where $\mathcal{X}$ denotes the input (observation) space, $\mathcal{U}$ the action space, $\mathcal{O}$ the observation space returned by tools or the environment, and $\mathcal{M}$ the memory state space. The functions R, P, and T correspond to reasoning, planning, and tool execution, respectively.

At each step t, the agent receives an input $x_t \in \mathcal{X}$ and maintains an internal memory state $M_t \in \mathcal{M}$. The decision-making process is factorized into the following stages.

First, the agent retrieves contextual information from memory:

$$c_t = M.\text{read}(x_t, M_t). \tag{3}$$

Second, a reasoning module produces an intermediate representation:

$$r_t = R(x_t, c_t). \tag{4}$$

Third, a planning module maps the goal g and inferred context into a (possibly multi-step) action plan:

$$\mathbf{u}_{t:t+k} = P(g, r_t, c_t). \tag{5}$$

An executable action is then selected (e.g., the first action in the plan):

$$u_t \sim \pi(\cdot \mid \mathbf{u}_{t:t+k}, r_t, c_t), \tag{6}$$

where π is an action selection policy that may incorporate stochasticity.

The selected action is executed via external tools, yielding an observation:

$$o_t \sim T(\cdot \mid u_t). \tag{7}$$

Let $h_t = (x_t, u_t, o_t)$ denote the interaction tuple. The memory is updated as

$$M_{t+1} = M.\text{update}(M_t, h_t). \tag{8}$$

Overall, the agent defines a closed-loop system in which inputs, internal reasoning and planning processes, and external tool interactions are tightly coupled. The policy is no longer treated as a monolithic mapping, but is instead instantiated through the composition of reasoning, planning, and memory modules, enabling interpretable and structured decision-making.

III. Reasoning

Reasoning constitutes the cognitive foundation of LLM-based agents, underpinning their capacity for decision-making, problem-solving, and coherent inference across complex task scenarios [10, 46]. Chain-of-Thought (CoT) prompting [**?**] demonstrated that eliciting intermediate reasoning steps through carefully designed prompts can substantially improve performance on complex tasks without modifying model parameters. Building on this insight, a rich body of subsequent research has explored diverse strategies to further strengthen the reasoning ability of LLM-based agents [47, 48]. In the context of agent systems, the intrinsic reasoning capability embedded within the foundation model is generally treated as a given prior; the central challenge thus shifts to how external mechanisms can bridge the gap between a model's static parametric knowledge and the dynamic demands of real-world tasks. Following this perspective, we organize existing methods along the information flow of the reasoning pipeline, categorizing them into three stages: (1) *input-stage enhancement*, which expands the informational boundary accessible to the model through external knowledge retrieval and multimodal perception; (2) *reasoning-process enhancement*, which optimizes the reasoning trajectory via search space expansion and path verification; and (3) *output-stage regulation*, which improves the reliability and factual accuracy of generated results through post-hoc evaluation and adaptive correction.

A. Input-Stage Enhancement

Input-stage enhancement expands the information available to the model before reasoning begins, shifting from sole reliance on parametric knowledge toward dynamic integration of external information. This category encompasses two main directions: knowledge augmentation and modality augmentation.

1) Knowledge Augmentation: Retrieval-Augmented Generation (RAG) [49] introduced the idea of coupling a parametric generator with a non-parametric retrieval module, enabling the model to fetch relevant documents from external corpora at inference time. Self-RAG [50] improves upon this by training the model to adaptively decide when to retrieve and to critique both retrieved passages and its own generations through reflection tokens. Beyond the retrieve-then-read paradigm, several alternative strategies have emerged. On the structured knowledge side, SubgraphRAG [51] extracts query-relevant subgraphs from large-scale knowledge graphs, and Think-on-Graph [52] enables LLMs to interactively explore knowledge graphs through iterative beam search over entity–relation paths, tightly coupling retrieval with multi-hop reasoning. On the knowledge generation side, Knowledge Card [53] trains domain-specialized small models as modular knowledge repositories that generate background documents on demand, augmenting closed-source LLMs without relying

on retrieval infrastructure. RuAG [54] takes a compression-oriented approach, distilling external data into compact first-order logic rules via LLM-aided Monte Carlo Tree Search and injecting them directly into prompts.

2) Modality Augmentation: Many real-world reasoning tasks involve non-textual evidence that cannot be adequately captured in words alone. Multimodal-CoT [55] first demonstrated that fusing visual features into chain-of-thought reasoning yields substantial gains on science QA, even with sub-billion-parameter models. However, it treats visual input as a static feature. Visual Sketchpad [56] lifts this restriction by equipping models with a drawing canvas for creating intermediate visual artifacts during reasoning, and CogCoM [57] internalizes step-by-step visual manipulations, such as region grounding and zooming, while producing interpretable reasoning traces.

B. Reasoning-Process Enhancement

Input-stage methods enrich the information available before inference begins; the techniques discussed below instead intervene on the reasoning trajectory itself. We organize them along two dimensions: *search-space expansion*, which increases the coverage of candidate solutions, and *path verification*, which provides the selection pressure needed to identify the most reliable outcome.

1) Search Space Expansion: CoT prompting and its zero-shot variant [**?**, 58] demonstrated that step-by-step decomposition is a latent capability activated by appropriate prompts. Tree of Thoughts [59] and Graph of Thoughts [60] generalized the search topology from a single chain to tree- and graph-structured spaces supporting branching, backtracking, and thought merging. These methods, however, rely on hand-designed prompting protocols, leaving open how to scale reasoning more systematically.

Recent work addresses this from two angles. On the learning side, DeepSeek-R1 [61] trains a model via large-scale reinforcement learning to produce extended chains of thought autonomously, exhibiting emergent self-verification and spontaneous backtracking that arise purely from reward optimization. CPO [62] takes a distillation perspective, collecting step-level preference pairs from tree search and fine-tuning via DPO so that ordinary decoding reproduces the quality of deliberate search at lower cost. On the resource-allocation side, Snell et al. [63] show that routing easy prompts to lightweight revision and hard prompts to parallel sampling or PRM-guided search enables a smaller model to match a substantially larger one under comparable compute budgets.

2) Reasoning Path Verification: Expanding the search space produces richer candidate solutions, yet without effective selection the additional diversity may introduce as many erroneous paths as correct ones. Self-Consistency [64] addresses this at the output level by sampling multiple chains and returning the majority answer, while Self-Refine [65] and Reflexion [19] implement iterative critique loops, with Reflexion further maintaining an episodic memory of past failures. These approaches improve reliability without external supervision but evaluate entire outputs rather than pinpointing which step first goes wrong. Process reward models (PRMs) address this limitation by scoring each reasoning step individually. Lightman et al. [66] train a PRM on human-annotated step-level labels and show that step-wise supervision substantially outperforms outcome-level reward models. Math-Shepherd [67] eliminates the annotation bottleneck by estimating step correctness through Monte Carlo rollouts, matching human-supervised PRMs while scaling to larger training sets. WizardMath [68] further integrates PRMs into both PPO training and inference-time verification, confirming that process supervision is most effective when it shapes both the learned policy and the selection mechanism.

C. Output-Stage Regulation

The strategies above improve reasoning quality from the perspectives of information supply and search mechanisms. However, model outputs may still contain factual errors—commonly termed hallucination [69]. Output-stage regulation addresses this risk along two complementary dimensions: *output reliability assessment*, which evaluates the factual consistency and trustworthiness of generated content, and *response behavior regulation*, which dynamically adjusts the model's generation strategy based on reliability signals to keep error risk within acceptable bounds.

1) Output reliability assessment: Reliability assessment aims to evaluate the factual trustworthiness of generated content. Evidence-based methods verify claims against external knowledge: FActScore [70] decomposes long-form text into atomic claims and checks each against a knowledge base, while FacTool [71] and Factcheck-GPT [72] extend this pipeline to code generation, mathematical reasoning, and scientific literature with finer-grained error localization. An alternative line of work detects hallucination without external knowledge. SelfCheckGPT [73] detects hallucination by measuring agreement across multiple independent samples without any external reference, and Semantic Entropy [74] refines this by clustering semantically equivalent outputs before estimating uncertainty, capturing the model's true confidence at the meaning level rather than the surface form.

Deeper approaches extract reliability signals directly from model internals. HaloScope [75] uses membership estimation scores from hidden states to distinguish factual from hallucinated generations in an unsupervised manner, while HalluLens [76] provides a systematic taxonomy distinguishing extrinsic from intrinsic hallucinations. Chain-of-Verification (CoVe) [77] takes an active approach: after generating an initial answer, the model automatically produces verification questions, independently answers them, and revises inconsistencies; RARR [78] follows a similar verify-then-revise pattern using retrieved external evidence.

2) Response Behavior Regulation: Reliability assessment provides error-identification signals; response behavior regulation acts on these signals to constrain the model's generation. Existing mechanisms divide into inference-time decoding adjustments and training-time alignment.

At the decoding level, contrastive decoding suppresses hallucination by amplifying differences between distinct signal

sources. DoLa [79] contrasts logit distributions from higher versus lower Transformer layers to surface factual knowledge encoded in deeper representations. CAD [80] contrasts output distributions with and without context to strengthen reliance on retrieved evidence, and ITI [81] directly shifts attention-head activations along pre-identified truthfulness directions during inference. More recent extensions include SADI [82], which introduces semantically aligned dynamic intervention, and MCD [83], which extends contrastive decoding to a three-model architecture for tighter hallucination suppression.

At the training level, FactTune [84] constructs factuality preference pairs from automated evaluation scores and fine-tunes via DPO, requiring no human annotation. For queries exceeding the model's knowledge boundaries, R-Tuning [85] explicitly teaches the model to output refusal signals on out-of-scope samples, and calibrated RL approaches [86] incentivize stochastic abstention when confidence is insufficient. CRITIC [87] takes a tool-interactive approach, having the model verify its own outputs against external tools such as calculators and search engines before committing to a final answer. SEAL [88] trains models to embed calibrated confidence directly into their natural-language responses, enabling downstream systems to assess reliability without probing internal states.

IV. Memory

Reasoning, as formalized above, is inherently episodic. However, complex real-world tasks often span multiple episodes, requiring the agent to leverage lessons from prior interactions to improve future performance. To address this need, recent research has introduced explicit memory mechanisms that enable agents to record, consolidate, and recall their own task-execution experiences [108, 109]. In the agent's cognitive flow, reasoning generates the raw material of experience, memory consolidates and organizes that material, and the consolidated memory in turn informs the planning and tool-use strategies discussed in subsequent sections. We specifically focus on *agent-level experiential memory*, rather than model-level parametric knowledge or static external knowledge bases already discussed under retrieval-augmented generation (Section III-A). Under this scoping, memory serves as the substrate for a closed loop: interaction produces experience, experience is consolidated into memory, and memory is retrieved to inform better actions in subsequent interactions. This loop not only strengthens the individual agent's reasoning and planning but also underpins the multi-agent coordination, attribution, and evolution discussed in later sections.

This closed-loop perspective naturally suggests three research questions that correspond to the three stages of the memory lifecycle: how raw interaction traces are transformed into storable entries (*memory formation*), how accumulated memories are structured, updated, and selectively forgotten over time (*memory maintenance*), and how relevant memories are recalled and converted into actionable context at decision time (*memory retrieval and utilization*).

A. Memory Formation

Memory formation concerns how an agent transforms its stream of interactions into discrete, storable memory entries. Existing approaches differ primarily in the degree of abstraction applied during this transformation, and can be grouped into three categories: direct recording, abstractive distillation, and typed routing.

1) Direct Recording: Direct recording retains raw interaction traces with minimal processing. Generative Agents [110] introduced the *memory stream*, a timestamped log of all observations the agent perceives, serving as the foundational data layer for higher-level memory construction. MemGPT [111] frames this process through an operating-system metaphor, writing interaction segments to persistent archival storage via self-directed function calls.

2) Abstractive Distillation: Abstractive distillation compresses raw experience into more compact representations, spanning a spectrum of granularity. At the coarsest level, the *reflection* mechanism of Generative Agents [110] periodically synthesizes recent observations into higher-level insights that are stored alongside raw entries. MemoChat [112] fine-tunes LLMs to compose structured memos through iterative memorization–retrieval–response cycles. Recursive summarization [113] maintains a continuously updated global summary through hierarchical compression of session-level memories. At a finer granularity, Mem0 [114] extracts individual facts from each exchange via a two-phase extraction-and-update pipeline. SeCom [115] demonstrates that the segmentation granularity itself critically affects quality, showing that topically coherent segments with compression-based denoising outperform both turn-level and session-level alternatives.

3) Typed Routing: Typed routing goes beyond deciding how much to abstract and additionally determines where each entry should be stored. This is a natural extension of abstractive distillation: once experience is processed, routing it into semantically distinct stores enables more targeted downstream retrieval. ENGRAM [116] classifies each turn via an LLM-based router into episodic, semantic, or procedural stores with normalized schemas; ablation experiments confirm that collapsing types into a single store substantially degrades retrieval quality. CoALA [117] provides the theoretical grounding for such designs, formally defining working, episodic, semantic, and procedural memory as distinct components of a language agent's cognitive architecture.

Notably, the type of memory formed at this stage directly determines what downstream capabilities can draw upon: episodic traces support reasoning-time self-correction (Section III), semantic knowledge informs experience-driven planning (Section V), and procedural entries enable cross-task skill reuse (Section V).

B. Memory Maintenance

As interactions accumulate, the agent's memory grows without bound, yet the capacity to attend to stored content at any given moment remains limited. Memory maintenance addresses this tension through two complementary aspects: the static structure that organizes stored entries, and the dynamic

TABLE I
PROFILE OF METHODS FOR SINGLE-AGENT REASONING ENHANCEMENT (SECTION III), ORGANIZED BY INTERVENTION STAGE AND SUB-TYPE. *Task Scope*: THE EVALUATION DOMAINS. *Prim. Signal*: THE PRIMARY INFORMATION SOURCE FOR VERIFICATION OR CORRECTION. *Correction*: THE ERROR-CORRECTION MECHANISM EMPLOYED. *TF*: WHETHER THE METHOD IS TRAINING-FREE. *TTC*: WHETHER THE METHOD SUPPORTS TEST-TIME COMPUTE SCALING.

Method	Sub-type	Task Scope	Prim. Signal	Correction	Ext. Ret.	Tool	TF	TTC	Year
A. Input-Stage Enhancement									
ViperGPT [89]	PE	Mu	Inter			•	•		2023
RAPTOR [90]	KE	QA, LF	Evid		•		•		2024
Self-RAG [50]	KE	QA, LF	Evid	SF	•				2024
LLaVA [91]	PE	Mu	Inter			•			2024
TopViewRS [92]	PE	Mu	Inter				•		2024
VisCoT [93]	PE	Mu	Inter				•		2024
HippoRAG2 [94]	KE	QA, LF	Evid		•		•		2025
Theanine [95]	KE	QA, LF	Evid		•		•		2025
SubgraphRAG [51]	KE	QA	Evid		•		•		2025
B. Reasoning-Process Enhancement									
Self-Consistency [64]	VRP	Ma, QA	Consis	Consis			•	•	2023
LATS [96]	ESS	Mu	Inter	Refl		•	•	•	2024
CoC [97]	ESS	Ma, QA, Co	Tool			•	•		2024
AnyTool [98]	ESS	Ag	Tool	SF		•	•		2024
RAIN [99]	ESS	QA	Inter	SF			•	•	2024
Math-Shepherd [67]	VRP	Ma	Log					•	2024
CPO [62]	VRP	Ma	Log						2024
rStar [100]	ESS	Ma	Consis	Consis			•	•	2025
rStar-Math [101]	ESS	Ma	Consis	Consis				•	2025
TTS [102]	ESS	Ma	Log				•	•	2025
TOPS [103]	ESS	Ma	Log				•	•	2025
Rewarding Progress [104]	VRP	Ma	Log					•	2025
WizardMath [68]	VRP	Ma	Log					•	2025
C. Output-Stage Regulation									
FActScore [70]	RAOC	LF	Evid		•		•		2023
FacTool [71]	RAOC	Mu	Evid		•	•	•		2023
SelfCheckGPT [73]	RAOC	LF	Consis				•		2023
Semantic Entropy [105]	RAOC	QA, LF	Inter				•		2023
RARR [78]	RAOC	QA, LF	Evid	TV	•		•		2023
ITI [81]	ACRB	QA	Inter				•		2023
Reflexion [19]	ACRB	Ag, Co	Inter	Refl			•	•	2023
L2R [106]	ACRB	QA	Inter						2023
Factcheck-GPT [72]	RAOC	QA, LF	Evid		•		•		2024
HaloScope [75]	RAOC	QA, LF	Inter				•		2024
DoLa [79]	ACRB	QA, LF	Inter				•		2024
CAD [80]	ACRB	QA, LF	Evid				•		2024
FactTune [84]	ACRB	LF, QA	Evid						2024
CoVe [77]	ACRB	QA, LF	Log	SF			•	•	2024
CRITIC [87]	ACRB	QA, Ma, Co	Tool	TV		•	•		2024
R-Tuning [85]	ACRB	QA	Inter						2024
HalluLens [76]	RAOC	Mu	Inter				•		2025
SADI [82]	ACRB	QA	Inter				•		2025
MCD [83]	ACRB	QA, LF	Inter				•		2025
SEAL [88]	ACRB	QA, LF	Inter						2025
SMC steering [107]	ACRB	QA, Co	Log				•		2025

Sub-type: KE = Knowledge Enhancement; PE = Perceptual Enhancement; ESS = Expanding Search Space; VRP = Verification of Reasoning Paths; RAOC = Reliability Assessment of Output Content; ACRB = Action/Constraint on Response Behavior. *Task Scope*: Ma = Math; QA = Question Answering; Co = Code; LF = Long-form Generation; Ag = Agent; Mu = Multi-task. *Prim. Signal*: Evid = Evidence-based; Inter = Internal states; Consis = Consistency; Log = Logic/Verification; Tool = Tool-based. *Correction*: SF = Self-Feedback; Refl = Reflection; TV = Tool-Verify; Consis = Consistency-based; blank = None. • = applicable, blank = not applicable.

mechanisms that keep the store current and manageable over time.

1) Storage Structure: The choice of storage structure determines the representational capacity and retrieval affordances of the memory system. The simplest approach is a flat, append-only list, as exemplified by the memory stream of Generative Agents [110]. Flat storage, however, provides no mechanism for relating or aggregating entries, limiting its scalability.

Graph-based structures introduce relational organization. Zep [118] builds a temporally-aware knowledge graph with

three hierarchical sub-graphs (episode, semantic entity, and community layers) and a bitemporal data model that tracks both event time and ingestion time. HippoRAG [119] draws on hippocampal indexing theory, constructing a schemaless knowledge graph via open information extraction and retrieving through Personalized PageRank. Editable Memory Graph [120] supports dynamic memory modification through explicit graph editing operations for personalized agent behavior.

Hierarchical structures offer multi-granularity abstraction. MemTree [121] organizes memories as a dynamic tree with varying abstraction levels across depths, routing new information to the most semantically similar branch. A-MEM [122] adopts a Zettelkasten-inspired note-based architecture with LLM-generated metadata and dynamically constructed inter-note links that evolve as new knowledge accretes.

2) Dynamic Maintenance: Beyond static organization, long-lived agents require mechanisms to update, compress, or discard stored memories as facts change and relevance decays. Existing approaches span a spectrum from passive decay to active policy learning.

Decay-based methods model forgetting as a continuous process. MemoryBank [123] applies the Ebbinghaus forgetting curve, exponentially decaying each entry's strength since last access and removing entries below a threshold. Timeline-based memory management [124] organizes dialogue history along a temporal axis for selective retention based on narrative relevance.

LLM-driven policies make maintenance decisions through explicit reasoning. Mem0 [114] uses an LLM to choose among four atomic operations (add, update, delete, or no-op) for each incoming fact. A-MEM [122] implements memory evolution, retroactively refining existing notes when new experience arrives. COMEDY [125] progressively compresses older interaction segments while preserving key factual content. MemOS [126] elevates memory to a first-class operating-system resource with unified lifecycle management primitives across multiple memory types.

Learned policies optimize maintenance through training signals. MemRL [127] trains a Q-function that estimates the future utility of each stored experience, retaining high-utility entries and discarding low-utility ones. MemGPT [111] takes a resource-management approach, evicting the least relevant segments from main context to archival storage when the context window approaches its limit, analogous to page swapping in virtual memory systems.

C. Memory Retrieval and Utilization

Stored memories become useful only when recalled at the right moment and in the right form. Memory retrieval and utilization closes the experiential loop by converting accumulated state back into actionable context.

1) Retrieval Mechanisms: The most widely adopted baseline is dense vector retrieval, where memory entries and the current query are embedded and ranked by cosine similarity [111, 114, 123]. This approach treats all entries as interchangeable and ignores temporal or importance signals.

Multi-factor scoring addresses this limitation by combining orthogonal dimensions. Generative Agents [110] weight recency, LLM-assigned importance, and embedding relevance into a composite score, a design that remains influential across subsequent systems. Type-aware retrieval further exploits the structure introduced during formation: ENGRAM [116] retrieves top-k neighbors independently from each typed store before merging results, ensuring balanced representation across evidence modalities. Zep [118] combines semantic search, BM25 keyword matching, and graph traversal, routing queries through the most effective path for each type. Utility-driven retrieval moves beyond relevance to task-level value: MemRL [127] first filters candidates by similarity, then reranks by a learned Q-value estimating each memory's contribution to task success.

2) Integration Patterns: How retrieved memories enter the agent's decision process varies substantially across systems, and can be characterized along a spectrum of integration depth.

Existing integration patterns can be characterized along a spectrum of increasing agent autonomy. *Context injection* serializes retrieved entries into the prompt for the LLM to consume via in-context learning [114, 123], requiring no architectural modification but offering the agent no control over what is injected or when. *Experience-conditioned reasoning* allows retrieved memories to actively shape the agent's inference trajectory: Generative Agents [110] feed retrieved reflections into a planning module to generate proactive daily schedules, and Agent S [128] retrieves relevant past task completions to constrain its plan search space. *Skill execution* further elevates memory from informational input to direct action: Voyager [129] (Section V) retrieves verified code-based skills and composes them to solve new tasks, bypassing deliberative planning entirely.

V. Planning

Planning refers to the ability of an agent to convert a high-level goal into a structured sequence of executable actions. Unlike single-step reasoning, planning must account for sub-task dependencies, environmental feedback, and potential execution failures over extended horizons [35, 130]. We organize existing methods along two paradigms: *decomposition-based planning*, which structures goals into sub-tasks prior to or during execution, and *search-based planning*, which explores alternative plans through systematic evaluation and backtracking.

A. Decomposition-Based Planning

Decomposition-based methods reduce a complex goal into manageable sub-tasks, differing primarily in *when* the decomposition occurs relative to execution.

1) Proactive Decomposition: Proactive decomposition generates a complete plan before any action is taken. Least-to-Most prompting [131] introduced the idea of progressively breaking a problem into simpler sub-problems and solving them in sequence, while Plan-and-Solve prompting [132] replaces the generic "think step by step" instruction with an explicit planning phase followed by sequential execution. HuggingGPT [133] extends this idea to multi-modal task

TABLE II
PROFILE OF METHODS FOR MEMORY CAPABILITY ENHANCEMENT (SECTION IV). EACH ROW CAPTURES A METHOD'S DESIGN CHOICES ACROSS THE MEMORY LIFECYCLE. *Formation*: HOW RAW EXPERIENCE BECOMES MEMORY ENTRIES. *Storage*: THE DATA STRUCTURE HOLDING MEMORIES. *Maintenance*: HOW MEMORIES ARE UPDATED OR DISCARDED. *Retrieval*: HOW RELEVANT ENTRIES ARE RECALLED. *Mem. Type*: E (EPISODIC), S (SEMANTIC), P (PROCEDURAL).

Method	Formation	Storage	Maintenance	Retrieval	Mem. Type	Task Domain	Year
Generative Agents [110]	Record+Distill	Flat	Append	Multi-factor	E, S	Simulation	2023
MemoChat [112]	Distill	Flat	Append	Similarity	E, S	Conversation	2023
Reflexion [19]	Distill	Flat	Append	Injection	E	General	2023
MemoryBank [123]	Record	Flat	Decay	Similarity	E, S	Conversation	2024
MemGPT [111]	Record	Tiered	Eviction	Autonomous	E	Conv, QA	2024
HippoRAG [119]	Distill	Graph	Incremental	Graph traversal	S	QA	2024
EMG [120]	Distill	Graph	Rewrite	Similarity	E, S	Conversation	2024
SeCom [115]	Distill	Flat	Append	Similarity	E, S	Conversation	2025
ENGRAM [116]	Routing	Typed	Append	Type-aware	E, S, P	Conversation	2025
Mem0 [114]	Distill	Graph	CRUD	Similarity	S	Conversation	2025
MemTree [121]	Distill	Tree	Compression	Tree traversal	S	QA	2025
A-MEM [122]	Distill	Graph	Evolution	Similarity	S	Conversation	2025
Zep [118]	Distill	Graph	Versioning	Hybrid	E, S	Conv, Ent	2025
RecurSum [113]	Distill	Flat	Compression	Similarity	S	Conversation	2025
COMEDY [125]	Distill	Flat	Compression	Similarity	E, S	Conversation	2025
MemOS [126]	Routing	Unified	Lifecycle	Multi-path	E, S, P	General	2025
MemRL [127]	Record	Flat	Learned	Utility-driven	E	General	2026

Formation: Record = raw logging; Distill = abstractive distillation; Routing = typed classification. *Storage*: Tiered = main context + archival; Typed = type-separated stores; Unified = multi-type unified store. *Maintenance*: CRUD = LLM-driven add/update/delete; Evolution = retroactive refinement; Versioning = bitemporal tracking; Lifecycle = unified management; Learned = RL-based utility estimation. *Retrieval*: Multi-factor = recency $\times$ importance $\times$ relevance; Type-aware = per-type top-k then merge; Hybrid = semantic + keyword + graph; Utility-driven = similarity + Q-value re-ranking; Autonomous = self-directed via function calls; Injection = full buffer injection; Multi-path = multi-modal retrieval.

orchestration: given a user request, an LLM generates a task dependency graph whose nodes are dispatched to specialist models hosted on Hugging Face. LLMCompiler [134] draws an analogy to classical compilers, producing a directed acyclic graph (DAG) of function calls that are executed in parallel wherever dependencies permit.

When the action space is grounded in physical or simulated environments, proactive decomposition often takes the form of hierarchical task networks. LLM-Planner [135] generates few-shot grounded plans for embodied agents, while LLM+P [136] translates natural-language goals into PDDL specifications and delegates solving to a classical planner, combining the linguistic flexibility of LLMs with the optimality guarantees of symbolic planners. More recent work explores richer plan representations: Planning Anything [137] formulates planning problems as optimization programs that are solved by formal solvers, and Skeleton-of-Thought [138] first generates a plan skeleton and then fills in details in parallel to reduce latency. Plan-and-Act [139] specifically targets long-horizon web tasks by separating a high-level plan from low-level action execution, demonstrating that explicit proactive planning significantly outperforms reactive approaches on complex web benchmarks.

2) Progressive Decomposition: Progressive decomposition interleaves planning with execution, refining or extending the plan based on intermediate observations. ReAct [18] established the foundational pattern by alternating reasoning traces with environment actions in a single prompt, enabling the agent to adjust its next step based on the most recent observation. The verbal self-reflection mechanism introduced by Reflexion [19] (discussed in Section III) further augments this loop, allowing agents to generate natural-language critiques of failed episodes and consult them in subsequent attempts.

ADaPT [140] introduces recursive decomposition—when the executor fails on a sub-task, a planner module further decomposes it into finer-grained steps, continuing until each step is within the model's execution capability. SelfGoal [141] dynamically constructs and refines a hierarchical goal tree (GoalTree) during interaction, selecting the most relevant sub-goals for the current state and decomposing them further when guidance is insufficiently specific. In open-world settings, Voyager [129] combines incremental planning with a growing skill library: the agent proposes sub-goals, writes executable code to achieve them, and stores verified skills for future reuse, enabling continual capability expansion in Minecraft. CodeAct [142] reframes the action space itself as executable code, allowing the agent to express complex, multi-step plans as Python programs that interact with the environment through function calls, naturally supporting iteration and branching. DEPS [143] follows a Describe-Explain-Plan-Select loop in which the agent generates a plan, explains its rationale, and selects among alternatives based on environmental feedback. KnowAgent [144] injects domain-specific action knowledge into the planning loop, constraining the agent's plan space to reduce hallucinated actions and improve plan feasibility.

B. Search-Based Planning

While decomposition-based methods commit to a single plan trajectory (possibly with replanning), search-based methods explicitly explore multiple candidate paths and select among them through evaluation, pruning, or backtracking. The tree search mechanisms introduced by Tree of Thoughts [145]

and Language Agent Tree Search [96] (discussed in Section III) provide the algorithmic foundation for this paradigm; here we focus on methods that apply and extend these ideas specifically to agent planning scenarios.

1) Step-Level Search: Step-level search expands and evaluates candidates at each decision point. MCTS has proven particularly effective for LLM-based planning. LLM-MCTS [146] uses the LLM as a commonsense prior policy to guide MCTS rollouts in large-scale household task planning. PromptAgent [147] applies MCTS to search over the prompt space itself, treating prompt optimization as a planning problem with strategic exploration. RAP [148] formulates reasoning as planning with a world model: the LLM simultaneously serves as both the world model (predicting next states) and the reasoning agent, with MCTS balancing exploration and exploitation. More recently, REST-MCTS* [149] integrates process reward models to guide tree search, and AFlow [150] uses MCTS at a higher level of abstraction, searching over entire agentic workflows composed of reusable operators rather than individual actions. Beyond tree search, ToolChain* [151] formulates tool-use planning as an A* search problem over chains of API calls, and DiffuSearch [152] proposes implicit search via discrete diffusion modeling, bypassing explicit tree expansion by predicting future states to inform current action selection.

2) Plan-Level Search: Rather than searching step by step, plan-level search generates and compares multiple complete or partial plans. This approach is natural when global coherence matters more than local optimality. The multi-plan selection paradigm identified by Huang et al. [153] represents an early systematization, where the LLM generates several candidate plans and a selection mechanism (voting, scoring, or verification) chooses the best one. PlanRAG [154] combines plan generation with retrieval-augmented decision making: the agent generates candidate plans, retrieves relevant evidence from a knowledge graph, and re-ranks plans based on retrieved information. Agent S [128] employs a hierarchical planning architecture with experience-augmented search, using a manager agent to generate high-level plans and a worker agent to execute individual steps, with an experience library that enables the agent to learn from past task completions.

VI. Tool Use

Tool use extends an agent's action space beyond text generation by enabling it to invoke external functions, APIs, code interpreters, and web services, and other external systems [142]. Unlike reasoning or planning, which operate within the model's internal representation, tool use requires the agent to cross the boundary between language and execution—generating structured calls, processing external returns, and integrating results back into the reasoning flow. We organize existing work along three dimensions that capture the lifecycle of this capability: *tool capability acquisition*, which concerns how models learn to use tools; *tool invocation*, which concerns the control-flow patterns through which tools are called during task execution; and *tool generalization*, which concerns how learned tool-use abilities transfer to unseen tools and environments.

A. Tool Capability Acquisition

Tool capability acquisition addresses how a model learns to recognize when a tool is needed, select the appropriate one, generate valid invocation parameters, and incorporate the returned results. Existing approaches can be broadly divided into trajectory-based learning, which internalizes tool-use behavior from demonstration sequences, and protocol-based alignment, which grounds the model's understanding in tool interface descriptions.

1) Trajectory-Based Learning: Toolformer [155] pioneered self-supervised tool learning by having the model annotate its own training corpus with API calls and retaining only those that reduce perplexity, enabling autonomous acquisition of tool-use behavior without task-specific supervision. ToolLLM [156] scales this idea to 16,000+ real-world APIs by using ChatGPT to generate diverse instruction–trajectory pairs and training open-source models via supervised fine-tuning, while APIGen [157] further improves data quality through a three-stage verification pipeline covering format, execution, and semantic correctness. GPT4Tools [158] and ToolACE [159] explore complementary directions in data synthesis: the former uses self-instruction from GPT-4, while the latter introduces tool self-evolution to systematically expand API diversity and dialog complexity.

Beyond static supervision, reinforcement learning enables models to refine tool-use strategies through interaction. ToolPlanner [160] employs a two-stage RL framework with path planning and dual feedback mechanisms for task completion and instruction following. WebRL [161] applies self-evolving curriculum RL to web agent training, progressively generating harder tasks from failed attempts and using a learned outcome reward model.

2) Protocol-Based Alignment: Rather than learning from execution traces, protocol-based methods align the model's representation with tool interface specifications. Gorilla [162] introduces Retriever-Aware Training (RAT), which conditions fine-tuning on retrieved API documentation so that the model learns to ground its calls in up-to-date specifications and adapt to documentation changes at test time. ToolkenGPT [163] takes a representation-level approach, encoding each tool as a special token in the model's vocabulary so that tool invocation becomes a natural extension of next-token prediction without modifying the frozen language model. EasyTool [164] addresses the upstream bottleneck of documentation quality by transforming diverse and verbose tool documentation into unified, concise instructions, significantly reducing token consumption while improving tool-use accuracy. At the ecosystem level, the Model Context Protocol (MCP) [25] standardizes agent-tool integration through a universal JSON-RPC interface, replacing per-tool custom connectors with a single interoperable protocol.

B. Tool Invocation

Once tool-use capability has been acquired, the central question shifts to how tools are invoked during task execution. We distinguish three control-flow patterns of increasing complexity: one-shot calling, closed-loop calling, and workflow-based orchestration.

TABLE III
PROFILE OF METHODS FOR PLANNING CAPABILITY ENHANCEMENT (SECTION V), ORGANIZED BY PLANNING PARADIGM. *Strategy*: FOR DECOMPOSITION METHODS, WHETHER PLANNING IS PROACTIVE OR PROGRESSIVE; FOR SEARCH METHODS, THE SEARCH ALGORITHM EMPLOYED. *Structure*: FOR DECOMPOSITION METHODS, THE PLAN FORM; FOR SEARCH METHODS, THE SEARCH SPACE. *Exec Mode*: HOW SUB-TASKS ARE EXECUTED. *Replan*: WHETHER THE METHOD SUPPORTS DYNAMIC REPLANNING (● = YES, BLANK = NO).

Method	Strategy	Structure	Exec Mode	Replan	Task Domain	Year
A. Decomposition-Based Planning						
Least-to-Most [131]	Proactive	Seq	Seq		General	2023
Plan-and-Solve [132]	Proactive	Seq	Seq		General	2023
HuggingGPT [133]	Proactive	DAG	Hier		General	2023
LLM-Planner [135]	Proactive	Seq	Seq	●	Robotics	2023
LLM+P [136]	Proactive	Seq	Seq		Robotics	2023
ReAct [18]	Progressive	Seq	Intlv		General	2023
DEPS [143]	Progressive	Seq	Intlv	●	General	2023
LLMCompiler [134]	Proactive	DAG	Hier	●	Tool	2024
Skeleton-of-Thought [138]	Proactive	Tree	Hier		General	2024
Voyager [129]	Progressive	Seq	Intlv	●	General	2024
ADaPT [140]	Progressive	Tree	Intlv	●	General	2024
CodeAct [142]	Progressive	Program	Intlv	●	General	2024
KnowAgent [144]	Progressive	Seq	Intlv	●	General	2024
Planning Anything [137]	Proactive	Program	Seq		General	2025
Plan-and-Act [139]	Proactive	Seq	Hier		Web	2025
SelfGoal [141]	Progressive	Tree	Intlv	●	General	2025
B. Search-Based Planning						
LLM-MCTS [146]	MCTS	State	Seq		Robotics	2023
RAP [148]	MCTS	State	Seq		General	2023
PromptAgent [147]	MCTS	State	Seq		General	2024
ToolChain* [151]	A*	Action	Seq		Tool	2024
REST-MCTS* [149]	MCTS	State	Seq		General	2024
PlanRAG [154]	Retrieve	Plan	Seq	●	QA	2024
AFlow [150]	MCTS	Plan	Seq		General	2025
DiffuSearch [152]	Diffusion	State	Seq		General	2025
Agent S [128]	Greedy	Plan	Hier	●	Web	2025

Strategy: Proactive = plan-then-execute; Progressive = interleaved planning and execution; MCTS, A*, Diffusion, Greedy, Retrieve = search algorithms. *Structure*: Seq = Sequence; DAG = Directed Acyclic Graph; State, Action, Plan = search space granularity. *Exec Mode*: Intlv = Interleaved; Hier = Hierarchical.

1) One-Shot Calling: In the simplest setting, a single user instruction triggers a single tool call whose result directly produces the final response. This pattern underlies the basic function-calling interfaces of commercial APIs [165] and is the primary evaluation target of benchmarks such as BFCL [166], which systematically assesses serial, parallel, and multi-language function calls using Abstract Syntax Tree matching. Toolformer's single-step API insertion [155] and Gorilla's syntax-aware call generation [162] both operate in this regime, where the model must select the right tool and produce well-formed parameters in a single generation pass.

2) Closed-Loop Calling: Many tasks require multiple rounds of tool interaction, where each tool's output informs the next decision. CRITIC [87] exemplifies this pattern: the model generates an initial response, verifies it against external tools such as calculators or search engines, and revises based on the feedback. AnyTool [98] implements a hierarchical, self-reflective closed loop over a large API pool, dynamically selecting and invoking tools across multiple rounds. ReAct [18] provides the foundational interleaving of reasoning traces and tool actions that underpins most closed-loop tool-use systems.

3) Workflow-Based Orchestration: For complex goals requiring coordinated use of multiple tools, explicit workflow structures govern the invocation order and data flow. LLM-Compiler [134] generates a DAG of function calls with dependency tracking, enabling parallel execution wherever possible. WorkflowLLM [167] constructs a large-scale benchmark of real-world workflows from Apple Shortcuts and fine-tunes models to orchestrate complex multi-tool sequences with nested branches and loops. GPTSwarm [168] models the orchestration as an optimizable graph of LLM invocations, while AFlow [150] uses MCTS to automatically search over workflow structures composed of reusable operators.

C. Tool Generalization

A practical agent must handle tools it has never seen during training and retrieve appropriate tools from large, evolving libraries. Tool generalization addresses these challenges along two complementary directions: adapting to unseen tools and discovering relevant tools from large collections.

1) Unseen Tool Generalization: Generalizing to previously unseen tools requires the model to rely on documentation comprehension rather than memorized call patterns. GenTool [169] systematically trains for this through two complementary strategies—zero-to-one generalization (handling entirely new tools) and weak-to-strong generalization (transferring from simpler to more complex tools). TOOLVERIFIER [170] improves generalization by generating contrastive questions that

TABLE IV
PROFILE OF METHODS FOR TOOL USE CAPABILITY ENHANCEMENT (SECTION VI). *Training*: THE LEARNING PARADIGM FOR ACQUIRING TOOL-USE CAPABILITY. *Tool Interface*: THE INTERACTION MODALITY BETWEEN AGENT AND TOOLS. *Tool Set*: THE TOOL AVAILABILITY ASSUMPTION AT TEST TIME. *Control Flow*: THE INVOCATION PATTERN DURING TASK EXECUTION. *Gen Target*: THE GENERALIZATION TARGET EVALUATED.

Method	Training	Tool Interface	Tool Set	Control Flow	Gen Target	Task Domain	Year
Toolformer [155]	Self-sup	Func-Call	Closed	One-shot	None	General	2023
ToolkenGPT [163]	SFT	Func-Call	Closed	One-shot	New-Task	General	2023
GPT4Tools [158]	SFT	Func-Call	Closed	One-shot	New-Task	General	2023
CRITIC [87]	None	Func-Call	Open	Closed-loop	None	General	2024
Gorilla [162]	SFT	Func-Call	Dynamic	One-shot	New-Tool	Code	2024
ToolLLM [156]	SFT	Func-Call	Open	Closed-loop	New-Tool	General	2024
AnyTool [98]	None	Func-Call	Open	Closed-loop	New-Tool	General	2024
APIGen [157]	SFT	Func-Call	Open	One-shot	New-Tool	General	2024
ToolPlanner [160]	RL	Func-Call	Open	Closed-loop	New-Task	General	2024
Re-Invoke [171]	None	Retrieval	Open	One-shot	New-Tool	General	2024
TOOLVERIFIER [170]	SFT	Func-Call	Dynamic	One-shot	New-Tool	General	2024
MetaTool [173]	None	Func-Call	Open	One-shot	New-Tool	General	2024
ToolACE [159]	SFT	Func-Call	Dynamic	One-shot	New-Tool	General	2025
EasyTool [164]	None	Func-Call	Dynamic	One-shot	New-Tool	General	2025
WebRL [161]	RL	Web	Open	Closed-loop	New-Env	Agent	2025
WorkflowLLM [167]	SFT	Func-Call	Open	Workflow	New-Tool	General	2025
GenTool [169]	SFT	Func-Call	Dynamic	One-shot	New-Tool	General	2025
BFCL [166]	—	Func-Call	Dynamic	One-shot	—	General	2025
ToolRet [172]	—	Retrieval	Open	—	New-Tool	General	2025

Training: Self-sup = self-supervised; SFT = supervised fine-tuning; RL = reinforcement learning; None = prompting-based; "—" = benchmark works. *Tool Interface*: Func-Call = structured API/function calling; Retrieval = search/retrieval interface; Web = browser-based interaction. *Tool Set*: Closed = fixed, pre-defined; Dynamic = may change across tasks; Open = large-scale, open-ended pool. *Control Flow*: One-shot = single call per query; Closed-loop = iterative call-observe-revise; Workflow = explicit DAG/program orchestration. *Gen Target*: New-Tool = unseen tools; New-Task = unseen task types; New-Env = unseen environments.

help the model self-verify its tool selection and parameter generation for unfamiliar APIs. Gorilla's retriever-aware architecture [162] also contributes here, as its conditioning on retrieved documentation enables adaptation to test-time API changes without retraining. ToolLLM [156] demonstrates strong zero-shot generalization on the out-of-distribution APIBench benchmark despite being trained on a disjoint API set.

2) Tool Discovery and Retrieval: As tool libraries scale to thousands of APIs, identifying the right tool for a given query becomes a retrieval problem in its own right. ToolLLM [156] trains a neural API retriever that recommends relevant APIs from a large pool, while Re-Invoke [171] proposes an unsupervised approach that enriches tool documents with synthetic queries and extracts user intents through multi-view similarity ranking, achieving strong zero-shot retrieval without any labeled query–tool pairs. ToolRet [172] provides the first comprehensive tool retrieval benchmark and reveals that existing information retrieval models, despite strong performance on general retrieval tasks, perform poorly on tool retrieval, highlighting a significant gap between conventional document retrieval and tool-oriented retrieval. MetaTool [173] addresses the upstream decision of whether to use a tool at all and which tool to select, framing these as meta-cognitive capabilities that precede the invocation itself.

A related line of work abstracts tool discovery to the level of reusable skills. SkillWeaver [174] enables web agents to autonomously synthesize reusable skills as APIs through iterative environment exploration, building transferable skill libraries. SAGE [175] enhances this paradigm with reinforcement learning, using sequential rollouts across similar tasks to improve skill acquisition.

VII. MULTI-AGENT COLLABORATION

Multi-Agent Systems (MAS) are analogous to division of labour and cooperation in human societies, where agents collaborate to solve complex problems. By integrating agents with distinct capabilities, MASs can tackle tasks beyond the ability of any individual agent [176, 177]. In this paper, we formally define a MAS S as

$$S = (A, E, C, G, \Pi), \tag{9}$$

where

- $A = \{a_1, \ldots, a_N\}$ is the set of agents;
- E denotes the shared environment that may influence the operation of a MAS S;
- C is the communication protocol that governs message exchange;
- G denotes the orchestration topology describing the architecture of a MAS S;
- Π denotes the set of collaboration policies.

Under this formulation, we argue that multi-agent collaboration should be understood as a structured and goal-oriented computational paradigm, which can be decomposed into four key components: **Role**, **Communication**, **Orchestration**, **Interaction**. Each component addresses a fundamental question in collaborative intelligence.

a) Role: Who acts? Let z_i denote the role assigned to agent $a_i \in A$. We define role specialization through the mapping $z_i = \rho(a_i), \quad z_i \in Z$, where $\rho(\cdot)$ is the role allocation function and Z denotes the set of possible roles. This component determines which agents are needed for

collaboration and clearly specifies their responsibilities, such as planning, reasoning, or execution.

b) Communication: How do agents communicate? During collaboration, agents exchange messages with one another subject to the protocol C. Let $m_{i \to j}$ denote a message sent from agent a_i to agent a_j. The communication process can be modeled by a generative function P_C, such that $m_{i \to j} \sim P_C(a_i, a_j, h_i)$, where h_i denotes the information available to the agent a_i, including its local history and any accessible shared context. This component specifies how the message is generated, structured, and transmitted among agents.

c) Orchestration: How are agents connected? Let G denote the organizational structure of the MAS S. The orchestration topology is defined as $G = (A, M), \quad M \subseteq A \times A$, where M represents the set of communication links between agents. This component focuses on the organizational structure of agents and the design of message-passing pathways.

d) Interaction: How do agents collaborate during execution? Let I_π denote the interaction mechanism induced by a policy $\pi \in \Pi$, defined as $I_\pi = (\phi_\pi, \psi_\pi)$, where ϕ_π specifies the policy-dependent information flow among agents, and ψ_π denotes the interaction pattern throughout the collaborative process. Interaction can be viewed as the implementation of the policy π at the execution-level.

Given the above components, a MAS S under policy π induces trajectories $\tau \sim \pi$, consisting of actions, messages, and intermediate reasoning steps. Evaluating a MAS S and its trajectories τ provides a unified lens for comparing different design choices across the four core components mentioned above.

A. Role

Understanding a role requires answering two fundamental questions: what can agents do and how responsibilities are distributed among them. Therefore, we decompose roles into two dimensions: **Role Capability** answers the first question, and **Role Allocation** answers the second question by assigning these capabilities through a mapping function $\rho(\cdot)$.

1) Role Capability: In multi-agent collaboration, role capability refers to the set of competencies, action spaces, and observation spaces associated with agents engaged in the collaborative process [176, 178]. It captures not only what an agent a_i can do (such as reasoning, planning, or tool use, as seen in **??**), but also what information it can access and how it can influence the environment E or other agents $a_j \in A$. In some frameworks [179, 180, 181], multiple agents are instantiated from the same LLM and consequently share identical capabilities, resulting in homogeneous role configurations. In contrast, other frameworks [182, 183, 184] employ agents built on different models or deliberately assign specialized responsibilities, producing heterogeneous role configurations characterized by asymmetric capabilities and differential access to information. As shown in Fig.1, role capabilities in multi-agent collaboration can be broadly classified into **homogeneous** and **heterogeneous** types.

a) Homogeneous Role Capabilities: In MASs, multiple agents are driven by the same underlying LLM and consequently share highly similar reasoning capability, action space, and observation scope. In this paradigm, the variation among agents arises primarily from role prompting or task framing rather than differences in intrinsic capabilities or information access. This homogeneous setup provides a controlled environment to systematically study how role definitions and interaction protocols shape collective behavior, without confounding factors from model heterogeneity.

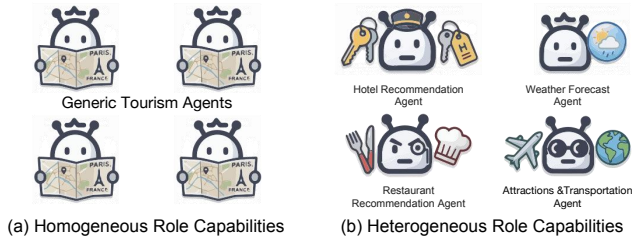


Fig. 1. Different role capabilities of agents.

A canonical example of homogeneous role capabilities is frequently observed in multi-agent frameworks where agents are instantiated from the same underlying language model. In CAMEL [185], both AI User and AI Assistant are created from a single LLM and interact symmetrically, with differences mainly arising from prompt instructions rather than model heterogeneity. Similarly, Generative Agents [110] simulate human-like behaviours through multiple agent instances derived from the same base model, enabling complex social interactions and emergent narratives in a controlled, homogeneous environment. Frameworks like SelfCorrect-Agent [181] and Chateval [180] further explore homogeneous multi-agent setups for robust reasoning and evaluation, leveraging symmetric LLM-based agents to iteratively correct or debate responses. VillagerAgent [179] demonstrates that even in graph-structured task coordination, MAS can be instantiated from identical models, emphasizing interaction over agent heterogeneity.

Building on these observations, homogeneous roles offer several practical advantages. Reusing the same model interfaces and toolsets across agents reduces engineering complexity, and coordination mechanisms are simplified because capability mismatches are minimized. At the same time, entirely homogeneous teams may encounter limitations when addressing highly complex or specialized tasks, where diversity in role capabilities—such as verification, domain-specific expertise, or advanced reasoning—becomes critical for robust and effective collaboration.

b) Heterogeneous Role Capabilities: In contrast, heterogeneous role capabilities describe systems in which agents are driven by different models or are designed with distinct functional specializations. Agents may differ in computational reasoning ability, observation access, or permitted actions. This form of heterogeneity expands system expressiveness, enabling division of labor, modular interaction, and specialization, which is crucial for complex tasks that require expertise beyond general reasoning.

A notable example is the X-MAS framework [186], which explicitly constructs multi-agent systems with diverse LLMs to harness collective intelligence across models of varying

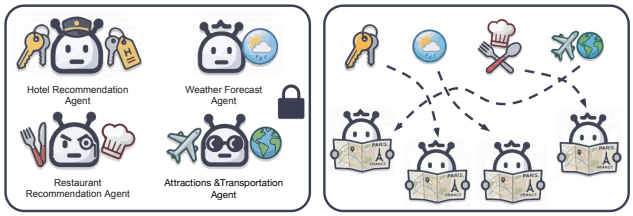
strengths and specializations, demonstrating significant improvements in both general reasoning and specialized domains compared to homogeneous setups. Similarly, the Adaptive Heterogeneous Multi-Agent Debate (A-HMAD) framework [187] assigns distinct roles to agents—such as logical reasoning, factual verification, and strategic planning—enabling more effective error-checking and domain coverage, and empirically reducing factual errors across benchmarks.More recently, MAPORL [24] demonstrates the emergence of heterogeneous roles through post-training and RL, enabling specialized coordination and task optimization in multi-agent systems.

Heterogeneous capabilities also arise naturally in systems that assign task-specific responsibilities. For example, multi-agent educational personalization systems assign Profiler, Rewriter, and Evaluator agents with distinct objectives and information access to tailor STEM content to individual learners [188]. In medical decision-making, frameworks like Mdagents employ LLM-based agents with complementary roles—such as symptom analysis, treatment recommendation, and risk assessment—improving diagnostic performance and safety [189]. In software engineering, automated document summarization benefits from multi-agent LLM collaboration, where Extractor, Summarizer, and Validator agents coordinate to generate concise and accurate software documentation [190], while in industrial optimization, OptDisPro [191] dynamically assigns agents to search, evaluate, or refine solutions, flexibly balancing exploration and exploitation and outperforming homogeneous optimization agents. Collaborative frameworks like Autogen [192] further illustrate heterogeneity through differentiated observation spaces, where commanders interact directly with users or environments while subordinate agents receive only intermediary context.

Overall, these studies demonstrate that heterogeneous role capabilities consistently enhance multi-agent systems by promoting specialization, improving task performance, and increasing robustness and adaptability across diverse domains.

2) Role Allocation: Role allocation refers to the division of labour in collaborative agent systems, involving the assignment of tasks and the clarification of responsibilities among agents. Although the concepts of *task* and *role* are often used in the multi-agent literature [193, 194, 195, 196], they describe different aspects of collaboration. Tasks usually refer to the units of work to be executed, whereas roles z_i represent the functional positions that enable agents a_i to perform those tasks. Across a range of existing frameworks [197, 198, 199], roles emerge from the mapping $\rho : A \rightarrow Z$ between agent capabilities and task requirements, which explains why the two notions are often closely coupled in practice. Here, *role* refers to the entities endowed with the capability to address particular tasks, rather than the tasks themselves. According to the mechanism by which roles z_i are distributed via $\rho(\cdot)$, **static role allocation** and **dynamic role allocation** approaches are illustrated in Fig.2.

a) Static Role Allocation: Under static role allocation in multi-agent systems, agent roles and responsibilities are specified prior to task execution. By defining the team structure in advance, this approach establishes clear boundaries in terms of agent capabilities, role identities, and task objectives. In practice, static role allocation is typically implemented through manually designed templates, often realized using prompt engineering techniques to specify role descriptions.



(a) Static Role Allocation (b) Dynamic Role Allocation

Fig. 2. Different approaches to role allocation.

This paradigm has been widely used in multi-agent systems designed for structured software engineering workflows. For example, ChetDev [13] organizes agents into predefined roles within a software development pipeline, enabling coordinated execution across stages such as planning, coding, and testing. Similarly, CodeDelegator [200] assigns agents specialized roles such as context manager and code reviewer to mitigate context pollution in collaborative coding tasks. Atomizer [201] adopts a comparable design for version control scenarios, where agents handle distinct aspects of commit untangling and conflict resolution. Static role allocation is also effective in data-centric and document-oriented tasks where responsibilities can be clearly partitioned. For instance, SheetAgent [202] divides spreadsheet-related operations into predefined roles such as data entry and data analysis, allowing agents to focus on specific subtasks while reducing coordination overhead. In requirements engineering, the MARE system [203] similarly distributes responsibilities among agents with specialized expertise to support structured requirement analysis and task delegation.

Beyond general data processing tasks, static role allocation has also been applied in domain-specific multi-agent systems where reliability and consistency are essential [204]. In healthcare applications, ColaCare [205] employs predefined roles for collaborative health record modeling to ensure stable and accurate task execution. MEDCO [206] adopts a similar design in medical education, where agents assume roles such as tutor and assessor to guide and evaluate learners within a structured training process. Several systems employ static role allocation to facilitate structured conversational or collaborative interactions. For example, CGMI [207] assigns agents roles based on task requirements to support coordinated multi-agent reasoning, while MultiAgentESC [208] defines specialized conversational roles to improve the quality and stability of emotional-support dialogues.

Overall, static role allocation provides a simple yet effective mechanism for organizing multi-agent collaboration in structured environments. By clearly separating responsibilities and limiting role ambiguity, it significantly reduces coordination complexity and improves execution efficiency. However, its reliance on predefined structures also introduces rigidity. When task requirements evolve or unexpected situations arise, static role allocation may lack the flexibility required for adaptive collaboration, motivating the development of dynamic role

allocation strategies.

b) Dynamic Role Allocation: In contrast to static role allocation, dynamic role allocation enables agents to adaptively select or adjust their roles based on task requirements and environmental context. Rather than relying on predefined templates, agents can determine the expertise required for a task and organize collaboration at runtime. As a result, roles may emerge and evolve during execution, allowing multi-agent systems to operate more flexibly in dynamic and open-ended environments.

Early research on dynamic role allocation primarily focuses on dynamic role selection, where agents recruit collaborators from a candidate pool according to task requirements. In this paradigm, agents identify the skills needed for a task and select suitable participants to form a task-oriented team. For instance, AutoAgents [209] allows agents to autonomously choose roles from a pool of available agents, enabling flexible collaboration in environments where tasks and agent capabilities continuously change. Similarly, the MedAide framework [210] explores this mechanism in medical decision-making scenarios, where agents dynamically select collaborators and adjust their roles based on evolving medical intents and information sources. In Agent Hospital [211], agents are recruited to address specific clinical situations, with roles dynamically determined according to the needs of the medical scenario.

More recent work extends this paradigm toward active initialization, where agents not only select collaborators but also dynamically instantiate or expand agent teams during task execution. In this setting, agents can initiate collaboration by generating new agents or integrating additional participants as task complexity increases. For example, AgentVerse [212] allows agents to dynamically initialize and organize multi-agent teams according to task demands, enabling roles to emerge through real-time coordination rather than predefined assignment. Auto-scaling LLM-based multi-agent systems [213] further explore this idea by dynamically integrating agents to scale system capacity based on workload and task complexity. Beyond systems, dynamic role allocation has also been investigated in reinforcement learning settings. Roma [214] demonstrates that agents can develop role specialization through Multi-Agent Reinforcement Learning (MARL) [215, 216, 217], allowing roles to emerge and evolve through interaction. Building on this idea, ResMAS [218] dynamically assigns roles to maintain system resilience when unexpected events or failures occur. Similarly, ARL-SMCS [219] enables agents to learn role assignments through evolutionary reinforcement learning in UAV–vehicle collaboration tasks.

In summary, dynamic role allocation significantly enhances the flexibility and scalability of multi-agent collaboration by allowing agents to reorganize themselves according to task requirements. However, this adaptability introduces additional coordination complexity, as agents must continuously evaluate task states, available resources, and the roles of other participants. Consequently, while dynamic role allocation is well suited for open-ended and evolving environments, it may also incur additional overhead in maintaining stable and efficient coordination.

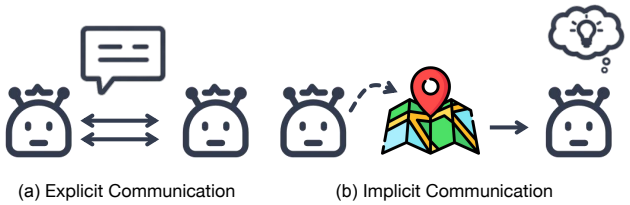


Fig. 3. Different communication modes among agents.

B. Communication

Communication can be described at different levels. **Communication Modes** capture the high-level patterns of information sharing, whereas **Communication Protocols** define the concrete rules and procedures through which these patterns are implemented in practice.

1) Communication Modes: Communication is a fundamental mechanism that enables coordination and information sharing in MASs [220, 221]. In collaborative settings, agents exchange information to align their decisions, share intermediate results, and collectively progress toward task objectives. In LLM-based MASs, communication typically occurs through structured or unstructured message exchanges, where the communication protocol C governs how messages $m_{i \to j}$ are generated and transmitted between agents a_i and a_j. In addition to such direct message passing, agents may also communicate by leveraging their perception capabilities, inferring information from observed behaviours, environmental states, or the actions reflected in others' histories. As shown in Fig.3, communication modes in agentic systems can be broadly categorized into **explicit communication** and **implicit communication**.

a) Explicit Communication: Explicit communication refers to agents exchanging information through well-defined message formats. In this mode, agents explicitly transmit task instructions, intermediate results, reasoning traces, or feedback through structured or unstructured messages.

Communication messages can take various forms, ranging from structured data such as code or documents to more flexible natural-language exchanges. In MetaGPT [222], agents including Product Managers, Architects, and Engineers coordinate software development workflows by exchanging structured documents, such as product requirement documents, system design specifications, and implementation plans. Similarly, TalkHier [223] introduces a structured communication protocol to support context-rich exchanges among agents. Beyond structured artifacts, some communication rely on natural-language messages. For example, MAGIS [224] coordinates multiple specialized agents to collaboratively analyze and resolve GitHub issues through iterative dialogue, where agents exchange natural-language analyses, proposed solutions, and verification feedback. Autodata [225] similarly organizes agents to collect and process open web data, enabling them to share extracted information, data summaries, and task progress through shared communication channels. EcoLANG [226] leverages context and social simulation, enabling agents to establish interpretable coordination protocols for efficient

task execution. MegaAgent [227] further extends this paradigm by supporting large-scale autonomous collaboration without rigid predefined workflows, allowing agents to dynamically generate messages describing plans, intermediate findings, or coordination signals.

Subsequent studies further investigate how the design and organization of messages influence the efficiency and scalability of multi-agent communication. The economical communication pipeline proposed in AgentPrune [228] reduces redundant interactions by filtering, summarizing, and compressing agent messages, demonstrating that carefully designed message representations can significantly lower token consumption while preserving essential task information. To further improve communication efficiency, TodyComm [229] introduces a task-oriented dynamic communication strategy in which agents selectively exchange task-relevant messages across interaction rounds, adaptively determining both communication targets and message contents. Other approaches [230, 231, 232] focus on adaptive or consensus-based communication mechanisms, where agents dynamically adjust messaging policies and exchange viewpoints to reach collective decisions through negotiation or voting. Beyond improving communication efficiency, recent research also examines how message exchange structures affect collaboration dynamics. For instance, AgentCoord [233] provides visualization tools to analyze the structure and flow of agent messages, revealing how different communication patterns influence task efficiency and coordination behaviours.

In summary, explicit communication remains the dominant paradigm in LLM-based MASs due to its transparency, controllability, and compatibility with role-based collaboration. By combining structured communication protocols with flexible dialogue mechanisms, modern frameworks continue to improve coordination efficiency and scalability in increasingly complex agent ecosystems.

b) Implicit Communication: Under implicit communication, agents infer the intentions and states of others by observing environmental changes and behavioral patterns. In this mode, communication emerges from perception, memory retrieval, and reasoning processes: agents monitor their surroundings, interpret the actions of peers, and use internal representations to infer context and coordinate decisions. A representative example of this paradigm is the Generative Agents framework [110]. In this system, agents simulate human-like social behaviour by continuously perceiving environmental events, retrieving relevant memories, and generating contextually appropriate actions or utterances.

In MARL [234], observation-driven communication can emerge naturally when agents optimize shared objectives. Shaw et al. [235] study cooperative foraging agents that learned coordination behaviours through environmental feedback. Subsequently, Buscemi et al. [236] demonstrate that LLM–based agents could implicitly align their numerical strategies through repeated interactions, effectively conveying information through observable choices rather than explicit messages.

Subsequent work has extended this principle to scenarios requiring more sophisticated reasoning about other agents. Wang et al. [237] introduce a theory-of-mind–based framework in which agents anticipate teammates' goals by interpreting their observable actions, thereby improving coordination in the absence of explicit communication channels. In addition, communication-free approaches have demonstrated that direct message passing may not always be necessary. For example, Ge et al. [238] show that agents could achieve effective cooperation in tasks such as box-pushing solely through observation and environmental interaction. LatentMAS [239] facilitates implicit communication by enabling agents to infer intentions and synchronize actions through shared environmental cues, providing a robust coordination mechanism when explicit messaging is costly or unavailable.

2) Communication Protocols: Multi-agent communication protocols C, also referred to as communication rules, define how agents exchange information, coordinate actions, and achieve shared goals in distributed environments. In MASs, communication protocols support a variety of collaborative processes, including task allocation, information sharing, negotiation, decision-making, and conflict resolution.

Similar to how the TCP/IP [240, 241] and HTTP [193, 242] protocols ushered in an unprecedented era of global connectivity on the Internet, standardized agent communication protocols are becoming a key infrastructure for building interconnected intelligent systems [243, 244, 245] and collaborative intelligence networks [246, 247, 248]. By providing common communication standards, these protocols [249] enable heterogeneous agents developed by different platforms or organizations to interact seamlessly, thereby facilitating large-scale agent ecosystems.

Table V compares several representative communication protocols. In this paper, we categorize existing communication protocols into two main groups: internal protocols, which govern communication among agents within a multi-agent system, and external protocols, which regulate interactions between agents and external systems, such as tools, services, or user interfaces.

a) Internal Protocols: Internal protocols define the communication rules among agents within MAS. These protocols primarily focus on enabling coordination, cooperation, and collective decision-making among multiple agents. Representative internal protocols include agent-to-agent communication frameworks such as Agent-to-Agent Protocol (A2A) [26], which enables direct peer-to-peer communication and task handoff between agents, and Agent Network Protocol (ANP) [252], which supports large-scale agent networks through mechanisms such as service discovery and message routing.

b) External Protocols: External protocols govern the interaction between agents and external environments, including tools, databases, APIs, and user interfaces, through standardizing how agents retrieve information, invoke tools, and exchange contextual data with external systems. One representative example is MCP [251], which provides a standardized interface for connecting agents with external tools and data sources and supports context sharing, tool registration, and structured request-response interactions.

Beyond the internal–external distinction, these protocols also differ in interoperability mechanisms and deployment

TABLE V
A TAXONOMY OF AGENT COMMUNICATION PROTOCOLS FROM TECHNICAL PERSPECTIVES. *Comm. Type*: THE PRIMARY INTERACTION ENTITIES. *Msg. Format*: THE DATA SERIALIZATION OR LINGUISTIC STRUCTURE. *Core Functions*: THE PRIMARY CAPABILITY OR SERVICE PROVIDED BY THE PROTOCOL.

Protocol	Proposer	Comm. Type	Msg. Format	Core Functions	Year
Agora [250]	University of Oxford	A2A	PD	Consensus formation	2024
MCP [251]	Anthropic	A2T	JSON-RPC	Standardized context access	2024
ANP [252]	ANP Community	A2A	JSON-LD+NL	Meta-protocol negotiation	2024
LMOS [253]	Eclipse Foundation	A2S	JSON-LD+TD	Group management	2024
ACP [254]	IBM	A2A	JSON-LD	Async & Sync interaction	2025
A2A [26]	Google	A2A	JSON-RPC/SSE	Agent discovery	2025
Agent Protocol [255]	AI Engineer Foundation	A2S	REST/HTTP	Standardized task API	2025

Comm. Type: A2A = Agent-to-Agent; A2T = Agent-to-Tool; A2S = Agent-to-System. *Msg. Format*: PD = Protobuf/Data-driven; JSON-LD = JSON for Linked Data; NL = Natural Language; TD = Thing Description; SSE = Server-Sent Events.

models. A2A and ANP both target interoperability within MASs, but with different emphases: A2A focuses on standardized cross-platform collaboration, whereas ANP is oriented toward AI-native networking and scalable open agent ecosystems. In contrast, MCP is client–server architecture that enables external resources to be integrated through a unified interface, thereby reducing integration overhead and improving modularity, maintainability, and reusability.

C. Orchestration

Orchestration describes the global coordination structure that governs how multiple agents are organized and interact within a MAS S. From a system-level perspective, orchestration can be viewed as the topology $G = (A, M)$ of collaboration: it defines how agents $a_i \in A$ are connected, how information flows through the links M among them, and how coordination decisions are structured during task execution. Orchestration focuses on the higher-level structural organization that determines how these roles and communications are integrated into a coherent collaboration process.

Closely related to orchestration is the concept of agent routing, which determines how tasks or messages are dynamically directed to appropriate agents during execution. Routing mechanisms [197, 256, 257] are typically responsible for selecting which agent should process a particular subtask or piece of information, often based on capability matching, contextual reasoning, or learned policies. In contrast, orchestration defines the broader coordination topology within which routing decisions occur. In other words, routing can be regarded as an operational mechanism embedded inside an orchestration structure, while routing determines which agent should act next. For example, routing decisions may be locally determined by agents in decentralized architectures [258].

Existing research generally organizes orchestration mechanisms into three representative topological paradigms: **centralized**, **distributed**, and **hybrid** orchestration. These paradigms differ primarily in how decision-making authority, coordination responsibilities, and information flow are structured across agents, and they provide distinct trade-offs in terms of scalability, adaptability, and control.

1) Centralized Orchestration Topology: Centralized orchestration topology follows a controller-centric coordination structure in which a single orchestrator maintains global decision authority over the collaboration process. In terms of coordination topology, agents are organized around a central controller that determines interaction flow. Information from different agents is typically aggregated by the orchestrator, which then determines subsequent actions and redistributes instructions. Consequently, routing decisions are largely governed by the central coordinator rather than emerging from peer-to-peer interactions.

Recent studies have explored various enhancements to the centralized orchestration topology. In terms of memory and planning, StackPlanner [259] utilizes a centralized controller with task-level memory to maintain global context during execution. Similarly, AOP [260] relies on the orchestrator to centrally generate and sequence overarching action plans for all participating agents. To improve topological flexibility, the Puppeteer framework [261] enable the central node to dynamically reconstruct the interaction pathways among agents based on real-time task demands. Meanwhile, the efficient approach [262] optimizes the central coordinator's routing decisions to minimize communication overhead within the centralized structure. Furthermore, CORL [263] uses an RL-based orchestrator to dynamically govern agent interactions, acting as a gateway to balance overall system performance and budget. Another domain-specific application, CCMA [264] employs centrally coordinated reinforcement learning, demonstrating how a single orchestrator can manage complex constraints, such as power grid topology, through global state visibility.

While centralized orchestration enables consistent global planning and simplified routing control, it may also introduce scalability limitations and potential single points of failure when the number of participating agents grows.

2) Distributed Orchestration Topology: Distributed orchestration topology removes the global coordinator and distributes decision authority across agents. From the perspective of coordination topology, agents interact through peer-to-peer connections rather than through a centralized controller. Information flows directly among agents, and routing decisions are determined locally based on each agent's capabilities, policies, and observed context.

A representative example is AgentNet [258], which models agent collaboration as a dynamically evolving network, equipping fully connected agents with distributed executor and router pools to autonomously determine task delegation through local reasoning. Symphony [265] establishes a highly

scalable decentralized framework that facilitates collective intelligence by allowing direct peer-to-peer interactions without centralized coordination bottlenecks. To optimize such distributed Orchestration, MAAC approaches [266] empower agents to dynamically learn and refine their decentralized coordination policies via reinforcement learning rather than relying on static routing rules. Furthermore, distributed orchestration requires robust mechanisms for local information management. AHKG [267] supports distributed planning by allowing agents to synchronize localized information clusters while inherently maintaining alignment with global topological objectives.

Similar decentralized coordination strategies have been widely adopted across various domains to preserve agent autonomy. For instance, HIPPO-MAT [268] employs decentralized multi-agent reinforcement learning and graph neural networks for task allocation without a central controller, while AgentFlow [269] proposes a distributed coordination framework based on publish–subscribe communication and dynamic service elections.

Collectively, these approaches demonstrate how distributed orchestration enables scalable collaboration while preserving agent autonomy and system robustness.

3) Hybrid Orchestration Topology: Hybrid orchestration topology combines centralized strategic coordination with decentralized execution mechanisms. In this paradigm, decision authority is partially centralized at higher levels of the system while lower-level agents retain autonomy during task execution.

From the perspective of coordination topology, hybrid orchestration typically adopts hierarchical structures. Early frameworks such as ChatDev [13] and MetaGPT [222] laid the groundwork for this paradigm by relying on centralized Standard Operating Procedures (SOPs) to constrain and guide the decentralized execution of role-playing agents. Building upon more complex hierarchies, AgentOrchestra [270] organizes collaboration into three layers: a planning layer for task decomposition and global coordination, a functional orchestration layer for managing tools and specialized agents, and an execution layer for carrying out concrete operations. Similarly, planner–executor architectures [271] separate global reasoning from action execution. TOA [272] structures agents in a hierarchy where parent nodes provide centralized aggregation and reasoning, while leaf nodes maintain autonomy for parallel context processing. GoA [273] expands decentralized execution by enabling non-linear, localized communication among agents, while maintaining a globally predefined topological constraint to ensure centralized goal alignment. AdaptOrch [274] further advances hybrid orchestration by introducing a centralized, task-adaptive orchestrator that dynamically selects the execution topology.

These approaches illustrate how hybrid orchestration can balance centralized strategic control with decentralized operational flexibility.

D. Interaction

Interaction characterizes how multiple agents coordinate during the execution phase of a task. Unlike static system design, which focuses on agent roles or architectures, interaction emphasizes the dynamic processes through which agents exchange information, update intermediate states, and jointly drive task completion under the current policy π.

1) Information Flow: Information flow ϕ_π governs the execution dynamics of multi-agent systems, dictating the orchestration of collaborative actions and the synchronization of shared states. Based on the temporal and logical dependencies of task execution, information flow can be primarily categorized into **sequential interaction** and **parallel interaction**.

a) Sequential Interaction: In sequential interaction, agents exchange information in an ordered, step-by-step manner, where the output of one agent serves as the input for the next. This pattern is particularly suitable for tasks that can be decomposed into a sequence of dependent subtasks.

Recent studies have extensively leveraged sequential interaction to structure agentic workflows. Frameworks such as HuggingGPT [133] and EAG [275] establish modular pipelines in which tasks are decomposed into discrete reasoning stages, whereas Chain of Agents [276] employs a sequential relay to mitigate information loss in long-context tasks. To enhance adaptability, AnyMAC [277] introduces dynamic cascading via next-agent prediction within sequential flows, a pattern also applied to specialized domains like recommendation in MACRec [278]. Furthermore, frameworks such as MAgIC [279] and recent role-specialized pipelines [280] utilize these linear dependencies to provide systematic benchmarks and ensure traceability and accountability across multi-agent interactions.

The main advantage of sequential interaction lies in its interpretability and modularity. Each agent operates on well-defined inputs and produces traceable outputs, making the overall process easier to debug and optimize. However, this structure may suffer from error propagation, as mistakes in early stages can cascade through subsequent steps. Additionally, strict sequential dependencies may limit efficiency when tasks could otherwise be executed concurrently.

b) Parallel Interaction: In parallel interaction, multiple agents operate simultaneously and independently on shared or decomposed tasks, followed by a mechanism for aggregating their outputs. Unlike sequential interaction, parallel interaction allows agents to explore diverse solution spaces concurrently, which can improve robustness and efficiency.

A foundational example of parallel interaction is the self-consistency decoding strategy [281], which generates multiple reasoning paths concurrently and aggregates the final answer. This principle was soon extended to multi-agent debate frameworks [282], enabling several agents to independently propose and critique solutions in parallel. Building upon these concepts, recent architectures further exploit parallelism to overcome sequential bottlenecks. MoA methodology [283] applies this at the model level by utilizing diverse LLMs as concurrent proposers and iteratively aggregating their outputs. At the execution level, the M1-Parallel framework [284] deploys multiple agent teams simultaneously to explore distinct solution paths, minimizing latency via early termination. To address real-time environment constraints, parallelized planning-acting architectures [285] decouple cognitive planning and physical

execution into independent, concurrent threads. Furthermore, for complex problem-solving, AgentGroupChat-V2 [286] decomposes user queries into a task tree to assign sub-tasks to specialized agents operating in parallel. Similarly, DynTaskMAS [287] employs dynamic DAGs to adaptively manage and execute these parallel sub-tasks, ensuring high system robustness.

Parallel interaction offers significant advantages in terms of scalability and diversity. By enabling multiple agents to explore different hypotheses simultaneously, the system can reduce the risk of converging to suboptimal solutions. However, it also introduces challenges in result aggregation and consistency maintenance. Designing effective mechanisms to reconcile conflicting outputs remains a key research problem in this setting.

2) Interaction Patterns: Interaction patterns ψ_π describe how collective outcomes emerge from actions of individual agent. Existing studies generally categorize multi-agent interaction patterns into two major types: **cooperative interactions** and **competitive interactions**, depending on whether agents pursue shared or conflicting objectives.

a) Cooperative Interaction: Cooperative interaction refers to scenarios in which agents share a common objective and coordinate their actions to maximize collective performance. In such settings, agents typically exchange information, divide responsibilities, and synchronize decisions to improve overall task efficiency and solution quality.

Recent studies have increasingly emphasized the role of structured coordination in enabling effective cooperation among agents. For instance, generative agent frameworks [110] simulate socially interactive agents, demonstrating how cooperation can emerge in complex environments. Similarly, role-based multi-agent systems such as MetaGPT [222] organize agents into specialized roles (e.g., product manager, engineer), where collaboration is achieved through well-defined workflows. Similarly, domain-specific architectures like the DrugAgent framework [288] utilize distinct planner and instructor roles to maintain precise workflow boundaries and avoid conflicts during complex scientific problem-solving.

Addressing the challenge of dynamic environments, the ProAgent framework [289] facilitates zero-shot coordination by enabling agents to proactively infer teammate intentions and adjust their strategies based on observed behaviors. Exploring the structural design of these interactions, socially-inspired collaboration mechanisms [290] reveal that the sequential orchestration of interaction topologies can significantly enhance cooperative outcomes without merely relying on scaling agent numbers. In decentralized settings with partial observability, the COMPASS framework [291] advances cooperative planning by introducing a multi-hop communication protocol, allowing agents to effectively exchange localized state information and synthesize adaptive skills.

To further ensure the reliability of cooperative interactions, recent methods have integrated structured evaluation and collective decision protocols. COPPER [292] introduce system-level feedback loops, utilizing shared reflection mechanisms to resolve credit assignment issues and iteratively refine collective behaviors. Concurrently, electoral approaches [293] to collective decision-making employ ordinal preferential voting mechanisms to systematically aggregate diverse agent preferences, thereby mitigating the limitations of simple majority rules and enhancing the robustness of joint decisions.

Overall, cooperative interaction in modern multi-agent systems is characterized by structured communication, role-based organization, and iterative alignment, which together facilitate efficient and scalable collaboration.

b) Competitive Interaction: Competitive interaction arises when agents pursue individual objectives that may partially or fully conflict with those of other agents. In this setting, agents attempt to advance their own objectives through strategic behaviours such as negotiation, confrontation, or persuasion.

Recent work has explored competitive interaction through mechanisms such as debate, negotiation, and strategic role-playing. For example, the debate framework proposed by Du et al. [282] enables multiple agents to argue over candidate solutions, where each agent attempts to expose weaknesses in others' reasoning. Building on this, Agent4Debate [294] introduces a dynamic framework to test if LLM agents can surpass human strategic logic in structured linguistic confrontations. Negotiation-based settings provide another important perspective on competition. The GPT-Bargaining framework [295] studies how agents engage in bargaining processes, where each agent aims to optimize its own outcome under conflicting interests. To further quantify these behaviours, ALYMPICS [296] provides a game-theoretic playground to observe how agents navigate classic dilemmas like the Tragedy of the Commons, while RTBAgent [297] pushes competitive tactics into high-stakes real-time bidding auctions. In addition, role-playing approaches [298] introduce diverse agent roles to encourage strategic diversity.

Recent architectures leverage competition as an optimization tool. Multi-agent KTO [299] reinforces strategic interactions by aligning agent preferences through game-theoretic objectives. To ensure these competitive processes remain robust, ALIGN [300] introduces performance guarantees to regulate delegated competitive reasoning.

Overall, competitive interaction emphasizes strategic adaptation, conflict resolution, and dynamic response to opponents. Unlike cooperative settings that rely on alignment and shared objectives, competitive scenarios drive agents to refine their strategies through adversarial exchanges continuously.

E. Evaluation

A critical aspect of optimizing MAS lies in the construction of high-quality evaluation datasets [301, 302] that capture realistic interaction trajectories τ and reflect diverse collaboration patterns.

Recent literature has increasingly shifted toward evaluating dynamic MAS. As foundational efforts, LLM-Coordination [303] establishes an early benchmark for analyzing how multiple large language models share information and collaborate to achieve shared objectives. Subsequent benchmarks have introduced heightened environmental complexity and complex interactive dynamics. LLMArena [304] assesses agents'

TABLE VI
COMPARISON OF MULTI-AGENT FRAMEWORKS BASED ON COLLABORATION DESIGN. *Role Cap.*: THE FUNCTIONAL DIVERSITY OF AGENTS. *Role Alloc.*: HOW ROLES ARE ASSIGNED TO AGENTS. *Com.*: COMMUNICATION MODALITY. *Orchestration*: THE MANAGEMENT STRUCTURE OF AGENT INTERACTIONS. *Exec. Seq.*: THE TEMPORAL EXECUTION PATTERN. *Interact.*: THE SOCIAL NATURE OF AGENT INTERACTION. *Opt.*: WHETHER THE FRAMEWORK INCLUDES AN OPTIMIZATION LOOP.

Framework	#Agent	Role Cap.	Role Alloc.	Com.	Orchestration	Exec. Seq.	Interact.	Opt.	Year
Generative Agent [110]	25	Hom	Static	Imp/Exp	Distributed	Par/Seq	Coop		2023
CAMEL [185]	2	Hom	Static	Exp	Distributed	Seq	Coop		2023
GPT-Bargaining [295]	3	Het	Static	Exp	Centralized	Seq	Comp	✓	2023
MetaGPT [222]	unlimited	Het	Static	Exp	Hybrid	Seq	Coop		2023
AutoGen [23]	unlimited	Het	Dynamic	Exp	Hybrid	Par/Seq	Coop		2024
AgentVerse [212]	unlimited	Het	Dynamic	Exp	Hybrid	Par/Seq	Coop		2024
AutoAgents [209]	unlimited	Het	Dynamic	Exp	Hybrid	Seq	Coop	✓	2024
MAPoRL [24]	unlimited	Het	Static	Exp	Hybrid	Seq	Coop	✓	2025
OSC [311]	unlimited	Het	Dynamic	Exp	Distributed	Seq	Coop	✓	2025
Puppeteer [261]	unlimited	Het	Dynamic	Exp	Centralized	Seq	Coop	✓	2025
AgentForge [312]	unlimited	Het	Dynamic	Exp	Centralized	Seq	Coop	✓	2026
MASFactory [313]	unlimited	Het	Dynamic	Exp	Hybrid	Seq	Coop	✓	2026

Role Cap.: Hom = homogeneous; Het = heterogeneous. *Role Alloc.*: Static = fixed roles; Dynamic = runtime assignment. *Com.*: Imp = implicit; Exp = explicit. *Orchestration*: Distributed = peer-to-peer; Centralized = master-slave; Hybrid = mixed. *Exec. Seq.*: Par = parallel; Seq = sequential. *Interact.*: Coop = cooperative; Comp = competitive.

adaptability and sequential decision-making in dynamic environments with incomplete information. Pushing beyond pure collaboration, BattleAgentBench [305] provides a comprehensive framework to evaluate both cooperative and competitive capabilities, offering quantifiable metrics for agents operating in zero-sum and non-zero-sum game scenarios. Furthermore, MultiAgentBench [306] evaluates orchestration quality through milestone-based KPIs and explicitly investigates the efficacy of different communication topologies (e.g., graph-based structures) on collaborative efficiency.

Pushing further toward realism, recent work explores high-fidelity, domain-specific evaluation settings that demand rigorous multi-round coordination. Medical testbeds like AgentClinic [307] and AI Hospital [308] serve as advanced environments for testing sequential decision-making, information gathering, and dispute resolution mechanisms under high uncertainty. Enterprise-oriented scenarios have been studied to highlight end-to-end task completion and coordination efficiency in real-world applications [309]. Pushing further toward realism and complexity, REALM-Bench [310] targets real-world planning tasks requiring long-horizon reasoning and multi-agent coordination.

Overall, trajectory-level benchmarks better reflect the interactive and collaborative nature of MAS. Such datasets enable both accurate evaluation and informed refinement of agent behaviours.

F. Discussion

Table VI summarises and compares various multi-agent frameworks from the perspective of collaboration design, offering insights into broader trends in the evolution of MASs.

- Heterogeneous role configurations have become increasingly prevalent. This trend allows MASs to better match diverse task requirements and improve coordination flexibility.
- Explicit communication remains the dominant mode in current LLM-based MASs, largely due to its transparency, controllability, and compatibility with structured collaboration. In contrast, implicit communication, although widely studied in embodied multi-agent settings [314, 315, 316], is still relatively underexplored in LLM-based MASs.
- No single orchestration topology appears universally optimal. Orchestration topologies are inherently task-dependent, as different tasks impose distinct requirements on coordination, control, and efficiency.
- Interaction patterns are largely cooperative, reflecting the focus on task completion and collective problem solving. Competitive settings are relatively underexplored and mainly appear in specialized scenarios such as bargaining.
- optimization is becoming an increasingly important component of multi-agent collaboration. Compared with earlier frameworks that relied mainly on fixed coordination policies, recent systems more frequently incorporate feedback, refinement, or learning-based mechanisms to iteratively improve collaborative performance.

Taken together, these observations indicate that MASs are gradually evolving from static, rule-based coordination toward dynamic, policy-driven collaboration with increasing adaptability.

VIII. MULTI-AGENT ANOMALY ATTRIBUTION

In recent years, Although MAS has shown potential in complex tasks through role allocation, tool invocation, and interactive collaboration, they still have significantly amplified system vulnerabilities. Inevitable failures and anomalies in the execution process have become increasingly prominent. A fundamental challenge arises from the tight coupling of multi-agent interactions, where initial localized errors such as hallucinations, frequently propagate through the system, culminating in intricate cascading failures that obscure the primary root cause. As execution trajectories grow longer and contain increasing amounts of interactive information, manually locating agent-level faults and their associated time

steps becomes highly inefficient and unscalable [317]. Therefore, an effective automatic attribution mechanism plays a key role in clarifying complex interaction processes in MAS and enabling fine-grained, targeted feedback to support directed improvement.. It is also an important foundation for enhancing system reliability and trustworthiness.

A. Formal definition

Failure attribution in MAS refers to the automatic identification of the responsible agents and critical steps in a MAS with execution failure or abnormal behavior, based on its complete execution trajectory. It also examines the type, causes and propagation mechanism of the anomaly throughout the collaboration process, to explain who introduced the anomaly, where it occurred, and why it ultimately led to system failure.

1) Task Inputs and Objectives:

- Input Conditions: Given a user query q, and the complete execution trajectory τ.
- Attribution Objective: The failure attribution task requires constructing an attribution model f, which can accurately calculate and locate the agent I that caused the anomaly based on the above inputs.

2) Formal Expression of the Attribution Process: To define the occurrence of an anomaly, a trajectory evaluation function $Z(\tau)$ is introduced. When $Z(\tau) = 1$, it indicates that the final result state of the trajectory τ is evaluated as anomalous. The entire multi-agent failure attribution process can be formally expressed as the following piecewise function:

$$(I, t) = \begin{cases} f(M, \tau, q), & \text{if } Z(\tau) = 1, \\ \Phi, & \text{otherwise.} \end{cases} \tag{10}$$

The above equation indicates that if and only if the system trajectory is evaluated as anomalous (i.e., $Z(\tau) = 1$), the attribution model f will utilize the system configuration M, trajectory τ, and user query q to output the identifier of the agent I causing the anomaly and the specific time step t (denoted as the tuple (I, t)); if the system operates normally (otherwise), it outputs the empty set $\emptyset$.

A multi-agent execution trajectory τ refers to the time-ordered dynamic sequence formed during task execution in a MAS, under a given user query, initial environment state, and system configuration. It consists of the contextual information, state transitions, communication interactions, tool usage, decision-making actions, and environmental feedback of the agents. Such a trajectory captures the complete execution process from the initial configuration to the terminal state, and serves as the fundamental basis for analyzing collaborative behaviors, anomaly propagation paths, and root causes of failure in MAS. It can be formally defined as

$$\tau = \big((i_0, u_0), (i_1, u_1), \ldots, (i_T, u_T)\big), \tag{11}$$

where (i_t, u_t) denotes the active agent and its corresponding action at time step t.

B. Multi-agent anomaly classification

With the growing adoption of MAS in complex tasks, researchers have increasingly recognized that anomaly analysis should go beyond identifying which agent or step is responsible. It must also address a more fundamental question: what type of failure has occurred and what underlying mechanism triggered it? In this sense, anomaly classification provides a unified descriptive framework for subsequent attribution, diagnosis, and repair. Existing studies have approached this problem from four broad perspectives: system structure, functional modules, execution stages, and causal lifecycle.

The first dimension concerns system structure and collaboration relationships, with an emphasis on identifying where the collaborative chain breaks down. Based on real execution trajectories, Pan et al. [317] identifies 14 failure modes and organizes them into three broader categories: specification and system design failures, inter-agent misalignment, and task verification and termination failures. This framework indicates that anomalies in MASs are often rooted in system-level deficiencies in collaboration, including suboptimal role assignment, distorted information exchange, and inadequate verification mechanisms, rather than in the isolated capability limitations of individual agents. In contrast, AgentAsk [318] further refines the classification unit to message handoff edges and categorizes anomalies into four types: Data Gap, Referential Drift, Signal Corruption, and Capability Gap. By explicitly coupling anomaly categorization with clarification mechanisms, this approach enables classification results to directly support online follow-up questioning and error interception. AEGIS [319] goes a step further by transforming anomaly types into controllable targets for error injection, thereby allowing classification to serve not only as a descriptive framework for failure phenomena but also as a basis for large-scale attribution-oriented training data generation.

The second category comprises classification methods centered on the functional layer and internal modules, emphasizing that anomalies may originate in different functional subsystems. TRAIL [28] classifies anomalies in agentic workflows into three broad categories—reasoning errors, system execution errors, and planning/coordination errors—and further specifies concrete failure types such as hallucinations, API resource failures, and context-handling failures. This framework indicates that anomalies in multi-agent systems (MASs) often emerge from the tight coupling among language reasoning, tool invocation, and the execution environment. Similarly, Zhu et al. [320] proposes the AgentErrorTaxonomy, which, from the perspective of a modular agent architecture, attributes anomalies to modules such as memory, reflection, planning, action, and system-level operations. Compared with classification schemes that focus primarily on external collaboration processes, these methods are better suited to revealing which internal module first becomes unstable, thereby laying the foundation for subsequent analysis of error propagation paths.

The third category comprises execution-stage-oriented classification methods, which organize anomaly types according to the task completion pipeline. Lu et al. [321] conceptualizes

autonomous agent systems as a planner–generator–executor pipeline and accordingly partitions failures into three stages: task planning, task execution, and response generation. More fine-grained causes are further identified, including improper task decomposition, flawed code generation, and context-window limitations. The primary advantage of this line of work lies in its ability to map failures to the stage at which they surface, thereby facilitating end-to-end evaluation and engineering-oriented debugging. However, its limitation is equally clear: although it effectively captures when an anomaly becomes manifest, it is less capable of determining whether the anomaly is produced locally within the current stage or propagated from defects introduced in preceding stages.

The fourth category comprises lifecycle-oriented classification methods, which extend anomaly categorization from surface-level symptoms to root-cause hierarchies and propagation processes. AgentFail [322] organizes failure root causes across three levels—agent, workflow, and platform—and further refines them into specific types, including tool- or action-planning errors, missing input validation, and resource or service failures. By grounding classification in concrete repair targets, this framework brings anomaly categorization closer to the practical needs of system diagnosis and remediation. FailCycle [323] further emphasizes that the point at which a failure originates does not necessarily coincide with the point at which it becomes visible, thereby revealing the pronounced propagation and evolutionary characteristics of anomalies in multi-agent systems (MASs). Accordingly, classification frameworks should cover propagation processes that span both nodes and stages. Meanwhile, AGENTSCOPE [324] adopts a behavioral-abstraction perspective, organizing anomalies into types such as invalid context, faulty verification, incoherent planning, and reward hacking, and explicitly linking them to structured behavioral representations and detectable invariants. This observation suggests that anomaly categories can function not only as semantic labels, but also as computable and verifiable behavioral properties.

Overall, existing studies have provided preliminary taxonomies of multi-agent anomalies from perspectives such as system structure, execution stage, anomaly granularity, and functional modules. However, a widely accepted unified classification framework is still lacking. On the one hand, most existing classification methods are designed for specific system settings, datasets, or diagnostic objectives and therefore remain limited in terms of generalizability and transferability. This not only makes the results of different studies difficult to align, but also restricts the reuse and comparison of data resources, evaluation frameworks, and diagnostic knowledge. On the other hand, many existing attribution methods still implicitly assume that each failure can be mapped to a single dominant type.In real MAS, however, anomalies are often characterized by multi-cause and multi-effect patterns: multiple local defects may jointly lead to a failure, while an early error may further propagate into diverse secondary anomalies during subsequent collaboration. Therefore, future research should extend existing taxonomies by incorporating mechanisms such as composite labels, cross-layer associations, and propagation chains, so that anomaly classification can better capture the coupled emergence and dynamic evolution of anomalies in MAS.

C. Anomaly Attribution Taxonomy

MAS shows potential in complex tasks, but also significantly amplify system vulnerability. In this context, multi-agent anomaly attribution has gradually moved from engineering experience to systematic research topics. Its core is no longer just 'whether the task is successful', but to answer more operable diagnostic questions : who (which agent) introduced the decisive error at what step / when, and how the error spread into the final failure. Around this problem, the existing research has formed three representative technical routes, which are the Data-driven method, the Rule-driven method, and the causal inference method.

1) Data-driven method: The data-driven method regards multi-agent abnormal attribution as a problem that relies on attribution data construction and pattern learning. On the one hand, it is necessary to construct a failure trajectory with abnormal labels as the basis for training and evaluation ; on the other hand, it is necessary to learn the abnormal propagation mode based on these data, so as to realize automatic attribution.

In terms of attribution data, Who&When [325] first systematically proposes the automatic anomaly attribution task, and publishes the Who&When dataset. It collects a large number of failure logs from 127 LLM MASs, and fine-grained labeling of responsible agents and decisive error steps, which provides a unified basis for supervision and evaluation for subsequent research. Similarly, TRAIL [28] complements another key data resource from the perspective of trace debugging. Based on the formal error classification system, this work constructs 148 manually labeled trajectories covering single-agent and multi-agent agentic workflows, providing high-quality samples for issue localization and long trajectory error analysis. Together, they have laid the starting point for data-driven attribution research, that is, first transforming who makes mistakes and which step makes mistakes into a learnable and comparable labeling problem.

Due to the high cost and limited coverage of manual labeling, subsequent work began to expand training resources by automatically generating attribution data. AEGIS [319] injects context-related errors into the original successful execution trajectory through LLM-based manipulator, constructs 9,533 trajectories with instrumentation agent and error mode annotation, and further supports various training paradigms such as Supervise fine-tuning(SFT), Reinforcement Learning(RL),and Comparative Learning(CL). AgenTracer [326] uses counterfactual replay + programmed fault injection to automatically label the failed multi-agent trajectory, generates TracerTraj data set, and trains the lightweight attribution model AgenTracer-8B on this basis. CORRECT [328] also advances along the idea of data expansion. Through the error injection process close to the real distribution, CORRECT-Error data set of more than 2000 labeled trajectories is constructed, and manual verification is used to ensure the consistency between the generation failure and the natural failure mode. It can

TABLE VII
CLASSIFICATION AND MULTI-DIMENSIONAL COMPARISON TABLE OF MULTI-AGENT ANOMALY ATTRIBUTION METHOD (SECTION VIII).

Method	Focus	Granularity	Data Type	Data Processing	Paradigm	Timing	Remediability
A. Data-driven method							
Who&When [325]	First Error	Agent/Step	FL	Manual	LLM-As-Judge	Offline	
TRAIL [28]	Category	Step/Category	FL	Manual	LLM-As-Judge	Offline	
AGENTRACER [326]	First Error	Agent/Step	SL+FL	Automatic	RL	Offline	•
Spectrum [327]	First Error	Agent/Step	FL	Automatic	SA	Offline	
CORRECT [328]	First Error	Agent/Step	SL	Semi-automatic	KD	Both	
AGENTASK [318]	Interaction	Step/Category	FL	Semi-automatic	SFT+RL	Online	•
AEGIS [319]	AE-Pair	Agent/Category	SL	Automatic	SFT/RL/CL	Offline	
GraphTracer [329]	Root Cause	Agent/Path	SL+FL	Automatic	RL	Offline	•
MASC [330]	Step-Error	Step	SL	–	MASC	Online	•
TRAJECTORY [331]	Pre-Error	Step	FL	Semi-automatic	PreAct	Online	•
Traj-Guard [332]	Traj-Deviation	Trajectory	SL+FL	Semi-automatic	Fine-tuning	Online	
DiLLS [333]	Root Cause	Plan/Step	FL	Manual	HITL	Offline	
TraceElephant [334]	First Error	Agent/Step	FL	Automatic	LLM-As-Judge	Offline	
B. Rule-driven method							
ABDUCT [335]	First Error	Agent/Step	Public Data	–	LLM-As-Judge	Offline	
AgentErrorBench [320]	Category	Step/Category	FL	Manual	LLM-As-Judge	Offline	•
Role-Trace [280]	First error	Stage	SL+FL	Automatic	Stage-comparison	Offline	•
ECHO [336]	Root Cause	Agent/Step	Public Data	–	LLM-As-Judge	Offline	
DOVER [337]	Root Cause	Agent/Step	FL	Automatic	LLM-As-Judge	Offline	•
AgentRx [338]	First error	Step/Category	FL	Manual	LLM-As-Judge	Offline	
XAgen [339]	First error	Agent/Step/WF	FL	Semi-automatic	LLM-As-Judge	Offline	•
ScopeLoc [340]	First error	Step	Public Data	–	LLM-As-Judge	Offline	
C. Causal inference method							
MARL [341]	Step Error	Agent/Step	SL+FL	Automatic	Counterfactual	Offline	•
InterDebug [342]	Fix-to-Succeed	Agent/Step	FL	Semi-automatic	HITL	Offline	•
CDC-MAS [343]	Root Cause	Agent/Step	Public Data	–	CDC-MAS	Offline	•
Specific Effects [344]	Step Error	Agent/Step	FL	Automatic	Cf-ASE	Offline	
AgentFail [322]	Category	Agent/Category	FL	Manual	LLM-As-Judge	Offline	
FailCycle [323]	First error	Agent/Step	FL	Manual	Counterfactual	Offline	•
MMDP-SCM [345]	Step Error	Agent/Step/State	SL+FL	Automatic	TCFE	Offline	
AGENTSCOPE [324]	Step Error	Step/Category	SL	Automatic	AGENTSCOPE	Both	
RAFFLES [346]	First error	Agent/Step	Public Data	–	LLM-As-Judge	Offline	
CHIEF [347]	First error	Agent/Step	Public Data	–	Graph-based	Offline	

Focus: AE-Pair = agent–error pair. *Granularity*: WF = Work Flow. *Data Type*: FL = Failure Log; SL = Success log. *Paradigm*: SA = Spectral Analysis; CL = Contrast Learning; MASC = Metacognitive Self-Correction; HITL = Human-in-the-loop; CDC-MAS = Causal Discovery Algorithm; Cf-ASE = Counterfactual Agent-Specific Effect; TCFE = Total Counterfactual Effect. *Remediability*: • = Attribution effectively supports repair; BLANK = No repair stage is involved.

be seen that the acquisition of attributable data is gradually evolving from small-scale manual labeling to automated and large-scale construction.

In terms of attribution methods, the data-driven route also shows the evolution from statistical attribution to knowledge transfer to structured representation learning. The FAMAS [327] method estimates the source of responsibility by performing multiple replays of the same task, abstracting the execution trajectory, and then using spectrum analysis to calculate the suspiciousness score of agent-action-state, and using the difference between repeated executions to estimate the source of responsibility. In contrast, CORRECT [328] does not rely on retraining the model, but uses the error schemata of online cache to reuse the historical failure structure, and completes targeted error localization in the reasoning stage, thus realizing data-driven knowledge transfer without training. AgenTracer [326] represents the direction of training attribution expert model based on automatic annotation data. With the help of TracerTraj data set collected by automatic annotation and multi-granularity reinforcement learning, AgenTracer enables the model to learn responsibility patterns directly from long trajectories. Furthermore, GraphTracer [329] points out that it is difficult to distinguish between surface symptoms and root causes by analyzing action sequences only in chronological order. Therefore, Information Dependency Graphs (IDGs) are constructed to explicitly describe the information dependencies between agents, and graph-aware synthetic data generation is combined to improve the ability to track cross-agent error propagation. This indicates that the data-driven method is no longer limited to the surface pattern learning of trajectory sequences, but further focuses on the structured dependencies in anomaly propagation.

In general, the data-driven method has formed a clear technical chain : starting from the benchmark data, the attribution samples are continuously expanded through automatic error injection and counterfactual playback, and then combined with statistical analysis, knowledge transfer and graph structure modeling to improve attribution ability. Its advantage is that it

can continue to improve as the data scale expands, but it also relies heavily on the quality of annotation, the consistency between the generated data and the real failure distribution, and the generalization ability of the model to cross-system and cross-task abnormal patterns.

2) Rule-driven method: The rule-driven method emphasizes that abnormal attribution cannot completely rely on the model 's free interpretation of long trajectories, but should reduce the diagnostic space and stabilize the reasoning process through explicit rules, structured processes and external constraints. Different from data-driven methods that focus on learning patterns from samples, such methods focus more on establishing a clear framework for error definition and analysis methods, and then gradually positioning the root cause on this basis. Therefore, the evolution logic from classification rule construction to hierarchical diagnosis to intervention verification is presented as a whole.

Such methods first need to build a clear and executable anomaly attribution rule system. Ma et al. [322] constructed AgentFail based on 307 failure logs, and proposed root cause taxonomy for platform orchestration system, which organizes failures into agent, workflow and platform levels, so that root cause identification can be carried out with clear structural priors. In contrast, Zhu et al. [320] proposed AgentErrorTaxonomy covering memory, reflection, planning, action and system-level operations from the internal module of the agent, and further transformed the classification results into error correction feedback for the responsibility step through AgentDebug. Although they adopt different classification perspectives, their core ideas are the same, both of which constrain and support the subsequent attribution process by establishing an explicit error type framework.

On this basis, the rule-driven method further starts to constrain the diagnosis process itself. Through hierarchical context representation, objective analysis-based evaluation and consensus voting, ECHO [336] decomposes the attribution process of complex trajectories into controlled hierarchical analysis, thereby reducing the instability caused by one-time global judgment. The Abduct-Act-Predict [335] adopts a fixed three-step attribution paradigm. Firstly, it proposes the potential hidden root cause, then gives the minimum correction action, and finally predicts whether the corrected execution trajectory can avoid failure. Its key contribution is not only to introduce a causal perspective, but also to rewrite open attribution into a procedural and executable analysis process. Further, DoVer [337] takes this regularization approach to dynamic debugging, which is not satisfied with generating attribution assumptions from the log, but rather proactively validates these assumptions through targeted interventions such as message editing, plan modification, etc., and uses whether or not to actually fix a failure or drive task progress as an evaluation criterion.

In general, the core advantage of the rule-driven method is its strong interpretability and controllability, and it is closer to the real debugging process. Through the mechanisms of taxonomy, hierarchical analysis, fixed reasoning steps and intervention verification, this kind of method makes the abnormal attribution gradually shift from the empirical judgment relying on log to the diagnosis and repair process constrained by rules. However, the effectiveness of such methods depends on the coverage and adaptation ability of the rule system itself. When the system form, task type or error propagation mechanism changes, the existing rules often need to continue to expand and update.

3) Causal inference method: The causal inference method establishes the multi-agent abnormal attribution based on the counterfactual responsibility analysis framework. The reason why an agent or a step should be attributed is not only because it is related to the failure result, but also because the task result will change after the intervention of the wrong agent or step. Compared with the diagnosis method based on correlation or pattern matching, this method pays more attention to two issues : first, how the influence of an action propagates in multi-agent interaction ; second, how this propagation effect is further decomposed into specific agents, steps, or state variables.

In this context, Triantafyllou et al. 's series of work provides a solid methodological basis [344, 345]. Agent-Specific Effects [344] pointed out that in multi-agent sequential decision-making, the causal effect of an agent's action will continue to propagate through the subsequent response of other agents. Therefore, cf-ASE is proposed to characterize the effect of 'an action propagates to the final result through other agents', and the corresponding identification and estimation methods are given. Then, based on this, MMDP-SCM [345] further decomposes the total counterfactual effect into two parts : one is transmitted through subsequent agent behavior, and the other is transmitted through environmental state transition ; the former can be allocated to individual agents by means of agent-specific effects and Shapley value, while the latter is attributed to specific state variables. Compared with these two works that focus on mechanism analysis, MARL [341] pays more attention to the importance measurement of agent-level. It measures the influence of an agent on the overall result by comparing the return changes before and after randomizing an agent's action, and identifies key agents accordingly.

On the basis of the above theory, causal inference began to be directly used for multi-agent anomaly attribution. Ma et al. [343] unified abnormal attribution and key step prediction into a multi-granularity causal framework. On agent-level, performance causal inversion combined with Shapley value is used to allocate responsibility. At step-level, a causal discovery algorithm CDC-MAS is designed to identify the real critical failure steps, and the validity of the attribution results is verified by counterfactual simulation. In contrast, CHIEF [347] emphasizes that flat logs are difficult to reveal true causal links. It first reconstructs the execution trajectory into a hierarchical causal map, and then realizes counterfactual attribution through hierarchical backtracking and progressive causal screening to distinguish root causes from subsequent symptoms. On the other hand, although the AGENTSCOPE [324] method does not directly calculate the counterfactual effect, its inspiration is to first abstract the long trajectory into a structured behavior representation, and then perform failure analysis based on behavior attributes and constraints, which provides a more stable intermediate representation for subsequent causal attribution.

TABLE VIII
MULTI-DIMENSIONAL COMPARISON TABLE OF MULTI-AGENT ABNORMAL ATTRIBUTION DATA SET (SECTION VIII).

Dataset	Focus	Size	Marking	Granularity	Data source	Metrics	Open Access
Who&When [325]	Localization	184	Manual	Agent+Step+Interp.	FL	Standard accuracy	Y
TRAIL [28]	Category	148	Manual	Step+type	FL	Composite metrics	Y
AgentFail [322]	Category	307	Manual	Agent+Category	FL	Standard accuracy	N
AgentErrorBench [320]	Category	200	Manual	Category+Feedback	FL	Standard accuracy	Y
AGENTASK [318]	Interaction	824	Manual	Interact+Category	FL	Standard accuracy	N
TracerTraj-2.5K [326]	Localization	2000+	Replay+Inject	Agent+Step	FL+SL	Standard accuracy	N
CORRECT-Error [328]	Localization	2000+	Inject	Step	SL	Standard accuracy	N
Aegis-Bench [319]	Category	9533	Inject	Agent+Category	SL	Composite metrics	Y
GraphTraj-2.5K [329]	Failure Trace	2147	Replay+Inject	Agent+Trace	FL+SL	Composite metrics	N
AgentErrata [324]	Category	210	Inject	Agent+Category	SL	Standard accuracy	N
MP-Bench [348]	Localization	289	Manual	Multi-Step	FL	Composite metrics	N
TraceElephant [334]	Localization	220	Manual	Agent+Step	FL	Standard accuracy	Y

Granularity: Interp. = Interpretation. *Data source*: FL = Failure Log; SL = Success log. *Metrics*: Standard accuracy = Ground-truth-based multi-granularity localization accuracy metrics; Composite metrics = Metrics for evaluating attribution accuracy and procedural fidelity. *Open Access*: Y = open access; N = Non-Open Access.

In general, the causal inference method has gradually developed from the early effect propagation analysis to the responsibility allocation, key step discovery and causal map-driven root cause positioning, which makes the multi-agent abnormal attribution further move from empirical correlation judgment to responsibility interpretation with intervention semantics.

D. Anomaly Attribution Evaluation

The development of multi-agent anomaly attribution research largely depends on the construction of data sets and benchmarks. Combined with Table VIII, it can be seen that the existing data set evolves roughly along two paths. One is a small-scale high-quality benchmark constructed manually, which is mainly used to clarify task definition, provide fine-grained annotation and support method evaluation. The other is the automatically generated large-scale attribution data set, which mainly alleviates the problem of high manual labeling cost and limited coverage through error injection, playback or structured synthesis. At the same time, there are also obvious differences in the focus of attention among different data sets. Some focus on agent-step attribution, some focus on error modes or root-cause taxonomy, and some further refine to module feedback, interactive edge or information dependence structure, which makes the current multi-agent anomaly attribution data show the characteristics of diverse task objectives and differentiated labeling granularity.

Judging from the development of manual annotation benchmarks, Who&When [325] is one of the starting points in this direction. This work systematically proposes an automatic anomaly attribution task for the first time, and publishes failure log data from 127 LLM MASs, labeling responsible agents and decisive error steps, providing a unified benchmark for subsequent research on who makes mistakes and which step makes mistakes. TRAIL [28] turns the focus of the data set to trajectory-level debugging, constructs 148 manually labeled execution traces, and supports a formal error classification system, so that the data can not only support attribution, but also support issue localization and trace reasoning.After that, the data set began to expand to more specific system scenarios. The AgentFail [322] dataset contains 307 failure logs from 10 agentic systems, highlighting the root cause labeling in the platform orchestration scenario. The AgentErrorBench [320] dataset brings together the systematic annotation failure trajectories from ALFWorld, GAIA and WebShop, combining module-level failure taxonomy with the interaction trajectories generated by MAS systems during the execution of real tasks. Another noteworthy is AGENTSCOPE [324], which builds a more comprehensive AgentErrata dataset in addition to the public Who&When dataset to support the diagnosis of failure step and failure type at the same time. From Table VIII, it can also be seen that such artificial data sets are still dominated by hundreds of magnitudes, but the labeling quality and granularity are high.

With the advancement of research, the focus of data set construction has begun to shift to automated expansion. AEGIS [319] directly regards the scarcity of large-scale and verifiable attribution data as the core bottleneck. By injecting controllable and traceable errors into the successful trajectories, 9,533 trajectories with rake agent and error mode annotations are constructed, which represents the scale direction of automatic attribution data generation. AgenTracer [326] further uses counterfactual replay + programmed fault injection to automatically label the failure trajectory to form a TracerTraj-2.5K dataset, which closely combines the construction of agent-step attribution data with model training. CORRECT [328] also relies on automatically constructing data, but emphasizes consistency with the true failure distribution : its data set CORRECT-Error contains more than 2,000 annotated trajectories derived from a manually verified error injection process guided by a true distribution. Then, GraphTracer [329] combines automatic data set construction with structured modeling, and uses graph-aware synthetic data generation to generate samples closer to the error propagation mechanism for key information nodes. Combined with the information in Table VIII, it can be seen that this kind of data no longer stays in the flat log, but begins to explicitly serve the graph structure attribution.

In general, the multi-agent anomaly attribution data set is

showing a trend of continuous transformation from small-scale manual annotation to large-scale automatic generation. At the same time, the dimension of data annotation is also constantly refined. It has gradually expanded from the early concentrated agent-step positioning to error mode, module feedback, trace reasoning and information dependence structure. From the perspective of attribution effect, the existing methods have made some progress in the coarse-grained Agent-level attribution. However, in the face of long trajectory, multi-round interaction and cross-agent cascade propagation, the fine-grained attribution of Step-level, Type-level and Trace-level is still significantly more challenging, indicating that there is still much room for improvement in fine-grained anomaly location and interpretation capabilities. At the same time, attribution evaluation is also deepening. The focus of evaluation is gradually expanding from the basic ' root cause localization accuracy ' to the depth of diagnosis, compound error recognition ability and multi-dimensional extensible indicators. This shows that the future data set construction should not only pursue scale expansion, but also need to be further improved in the labeling level, evaluation dimension and real scene adaptability.

E. Existing challenges and future research directions

1) Core challenges: Although multi-agent anomaly attribution has made preliminary progress in task definition, data set construction and method design, it still faces several key challenges.

- The definitions of anomaly, root cause and responsibility have not been unified. Different work is described from the perspectives of system structure, execution stage, function module and causal responsibility, which makes it difficult to align classification criteria and annotation granularity.
- Multi-agent anomalies have significant dynamic coupling and propagation characteristics. Real failures are often triggered by multiple local biases and gradually amplified in multiple rounds of interactions. Therefore, the traditional single-cause single-result and single-point positioning attribution methods are difficult to fully explain complex scenarios.
- Fine-grained attribution is still difficult : existing methods have made some progress in Agent-level coarse-grained attribution, but in the face of long trajectories, multi-round interactions and cross-agent dependencies, the attribution effects of Step-level, Type-level and Trace-level are still limited.
- Although the construction of data sets is shifting from small-scale manual labeling to large-scale automatic generation, the authenticity and generalization are still insufficient, and there are still distribution differences between automatic injection errors and real failures.
- Current evaluation frameworks remain centered on localization accuracy, with limited assessment of diagnostic depth, the ability to cover multi-cause and multi-effect cases, and practical repair value. In addition, attribution, verification, and repair have not yet been integrated into a stable end-to-end closed loop.

2) Future research direction: In the future, failure attribution in MAS should evolve from merely being able to localize faults toward being able to explain, verify, and support repair.

- A unified and extensible framework for anomaly classification and attribution is needed to support alignment across heterogeneous system settings and label granularities, while more effectively capturing complex phenomena such as multi-cause, multi-effect, and anomaly propagation.
- It is necessary to build a larger scale, higher authenticity, and more granular attribution data set. While using automatic generation to reduce costs, manual verification and real log introduction should be strengthened to improve data quality and scene adaptability.
- Future attribution methods should adopt more structured modeling strategies that explicitly represent message passing, information dependencies, behavioral evolution, and anomaly propagation chains, thereby improving fine-grained attribution in long-horizon trajectories.
- Causal inference and multi-cause and multi-result modeling should be further strengthened, from single responsibility point identification to responsibility set, contribution sharing and counterfactual propagation analysis.
- It is also necessary to establish a closed-loop pipeline of attribution, verification, and repair, together with a multidimensional evaluation framework, so that anomaly attribution can provide not only localization results but also trustworthy and verifiable diagnostic evidence that can be translated into practical system repair gains.

IX. Multi-Agent System Self-Evolution

Previous studies shows that the most resilient systems are not rigidly designed from the top down, but rather evolve continuously from the bottom up [349]. Through basic mechanisms like mutation and selection, biological populations adapt to survive in unpredictable environments. Inspired by this process, this section explores how Multi-Agent Systems (MAS) can shift from static, human-engineered frameworks to autonomous, self-evolving systems. Instead of relying on fixed rules, a self-evolving MAS allows agents to continuously update their own behaviors, rewire their communication networks, and build entirely new organizational structures over time. In the following subsections, we detail the motivation for this shift, formalize the evolution process, categorize existing methods, and discuss the open challenges ahead.

A. Motivation

Recent advancements in multi-agent collaboration have introduced highly automated mechanisms [10]—such as dynamic role allocation and trajectory refinement—that already show early forms of evolution. This transition is mainly driven by the growing complexity of target domains. As tasks increasingly move towards open-ended and highly unpredictable environments, systems must adapt beyond static, human-designed boundaries. To succeed in these dynamic settings, multi-agent frameworks must overcome their current reliance on predefined rules. Specifically, existing paradigms face two

primary bottlenecks that limit open-ended capability growth, motivating a clear shift towards true self-evolution.

a) The Rigidity of Static Architectures: Although modern MAS allow for runtime dynamic role allocation, their underlying orchestration logic and interaction protocols I_π are heavily constrained by human design. Current optimization methods primarily focus on maximizing trajectory utility $U(\tau)$ within a fixed interaction framework. However, when the optimal trajectory τ^* under these constraints still fails to solve a complex task, real-world execution frequently encounters unseen edge cases and severe coordination failures where predefined adaptive rules fail. Current systems lack the capability to fundamentally change and evolve their structural connectivity G from scratch. When a sub-task fails or a bottleneck emerges, they can only adjust within a limited scope, rather than autonomously inventing entirely new collaboration structures to bypass failures and adapt to open-world complexities.

b) The Bottleneck in Scaling: As task complexity demands a larger and more diverse set of specialized agents, the search space for orchestration topology G and collaboration policies Π grows exponentially. Manual orchestration or rule-based scaling quickly reaches its limits. Communication overhead and the risk of interference grow non-linearly [350, 351]. Without an evolutionary mechanism to iteratively evaluate, modify, and remove inefficient communication links, expanding a MAS often leads to diminishing returns. Consequently, MAS optimization requires a shift from deterministic coordination to self-improving evolutionary algorithms that can sustainably manage collective intelligence at scale.

To break through these bottlenecks of limited adaptability and scaling complexities, researchers are increasingly turning to **Multi-Agent Evolution**. Inspired by biological evolution, this paradigm moves beyond merely executing collaborative policies to autonomously modifying, recombining, and selecting agent configurations over time.

B. From Attribution to Evolution

As discussed in Section**??**, multi-agent anomaly attribution is crucial for understanding system failures. However, accurate diagnosis alone does not improve the system. A truly autonomous MAS must bridge the gap between identifying errors and mitigating them. To achieve this, we draw inspiration from biological adaptation. In nature, environmental pressures and failures trigger trial-and-error exploration [352]. Similarly, an autonomous MAS should treat recognized anomalies not just as static logs, but as direct triggers to actively search for optimized collaborative states.

The main difference between attribution and evolution is their time focus. Attribution is retrospective: it analyzes past executions to find the root cause, answering "Why did the failure occur?" and "Who is responsible?". Conversely, self-evolution is prospective: it looks to the future, asking "How can the system perform better?" and "How should it adapt to unseen environments?". This difference is clearly seen in biological swarm intelligence. For example, when foraging ants encounter a blocked path, the fading of pheromones serves as a retrospective attribution of failure. Instead of stopping, the colony uses this past feedback to prospectively adapt its route, actively exploring to bypass the obstacle.

Despite their different time focuses, attribution and evolution are deeply connected in the continuous learning loop of an MAS. Self-evolution cannot operate blindly; random structural changes or arbitrary prompt adjustments are computationally inefficient and prone to failure. Instead, error attribution provides the exact context needed for targeted evolution. By pinpointing the root cause of a failure, attribution narrows down the evolutionary search space. It directly guides the system to systematically rewire its organizational topologies and refine the behavioral policies of individual agents.

C. Formalizing Multi-Agent System Self-Evolution

To formally study self-evolving MAS, we extend the static collaboration framework defined in Section**??** into a dynamic, discrete-time process. We conceptualize the evolution by introducing a generation index $k \in \mathbb{N}$. The self-evolving MAS S is thus defined as a time-varying tuple:

$$S^{(k)} = (A^{(k)}, E, C^{(k)}, G^{(k)}, \Pi^{(k)}). \tag{12}$$

where

- $A = \{a_1, \ldots, a_N\}$ is the set of agents;
- E denotes the shared environment that may influence the operation of a MAS S;
- C is the communication protocol that governs message exchange;
- G denotes the orchestration topology describing the architecture of a MAS S;
- Π denotes the set of collaboration policies.

The core of this dynamic framework is the self-evolution mechanism, formalized as a transition mapping Γ. Rather than relying on random changes, this mapping autonomously generates the next-generation system based on historical context:

$$S^{(k+1)} = \Gamma\left(S^{(k)}, H^{(k)}\right), \tag{13}$$

where $H^{(k)}$ encapsulates the historical context accumulated from past executions. It serves as a comprehensive memory pool that stores not only the objective execution trajectories τ but also the corresponding evaluative feedback, such as localized error attributions and peer-agent critiques.

The ultimate objective of the mapping Γ is to monotonically improve the system's global utility U. To ensure the MAS develops generalized problem-solving capabilities rather than overfitting to a single scenario, $U(\cdot, T)$ is evaluated over a diverse distribution of multiple tasks $T \sim p(T)$. This utility quantifies how well the system fulfills its objectives, balancing metrics like task success rate against execution costs. The evolutionary progress is strictly formulated as maximizing the expected utility across this open-ended task space:

$$\mathbb{E}_{T\sim p(T)}\left[U(S^{(k+1)}, T)\right] \geq \mathbb{E}_{T\sim p(T)}\left[U(S^{(k)}, T)\right]. \tag{14}$$

This mathematical formulation confirms that self-evolution is not merely a parametric update, but a comprehensive process of structural and behavioral adaptation. Driven entirely

internally without human intervention,it aims to discover collaborative paradigms where agents, communication protocols, orchestration topologies, and behavioral policies are dynamically tailored to maximize the expected utility within the target task distribution.

D. MAS Self-Evolution Taxonomy

As discussed in SectionIX-A, recent advancements are driving MAS from static, rule-based designs toward autonomous, self-improving frameworks. However, the exact conceptualization of this "evolution" across current literature remains highly heterogeneous. While existing studies provide a strong foundation for MAS adaptation, self-evolution introduces a new layer of complexity: some methods focus on refining individual prompts [], others on updating shared model parameters [], and some go as far as restructuring the entire communication topology []. To clarify this landscape and bridge the discrete-time process formalized in SectionIX-C with practical implementations, we introduce a unified classification framework tailored specifically for MAS self-evolution.

Our proposed taxonomy systematically organizes these diverse mechanisms across two distinct dimensions: *where* the adaptation occurs, and *what* exactly is being adapted. We mathematically ground this by conceptually decomposing the evolutionary transition mapping Γ into three hierarchical scopes, thereby defining the **Evolutionary Locus**:

$$\Gamma = \{\Gamma_{\text{macro}}, \Gamma_{\text{meso}}, \Gamma_{\text{micro}}\} \tag{15}$$

where:

- **Micro-level (Γ_{micro})** focuses on the localized capabilities of individual agents, refining internal components within the agent set $A^{(k)}$.
- **Meso-level (Γ_{meso})** targets the organizational structure of the MAS, evolving the orchestration topology $G^{(k)}$, the communication protocols $C^{(k)}$, and the shared collaboration strategies $\Pi^{(k)}$ among the agent population.
- **Macro-level (Γ_{macro})** captures the holistic evolution of the overall system S within a specified environment E, operating at the highest tier to adapt its overarching systemic paradigms and architectural boundaries to environmental pressures.

Guided by this Locus-Target taxonomy, the following sections systematically dissect the current research landscape of multi-agent self-evolution. We trace this evolutionary trajectory from the bottom up: beginning with the refinement of individual agent capabilities at the micro-level, ascending to the autonomous restructuring of organizational relationships at the meso-level, and finally culminating in the overarching paradigm shifts at the macro-level.

1) Micro-Level: Agent Self-Evolution: The micro-level of our taxonomy focuses on Γ_{micro}, which targets the localized capabilities of individual agents. Formally, an individual agent $a_i \in A^{(k)}$ can be abstracted into four core components: its instructions or persona (p_i), its accumulated memory (m_i), its underlying neural parameters (θ_i), and its accessible toolset (t_i). Crucially, while the target of evolution here is isolated to an individual agent, the driver of this evolution is deeply rooted in the multi-agent context. Rather than relying on external human annotations, these agents leverage peer interactions—such as adversarial self-play, mutual critique, and collaborative knowledge sharing—to generate rich, autonomous feedback signals. Consequently, micro-level evolution seeks to monotonically improve an agent's expected utility by adapting one or more of its internal components based on these multi-agent dynamics. Following this formalization, we categorize micro-level self-evolution into four distinct paradigms.

a) Prompt and Profile Evolution: This paradigm focuses on optimizing the textual prompt p_i, which defines an agent's role, instructions, and operational heuristics. Here, the underlying model parameters remain frozen, and evolution occurs purely through natural language feedback loops. For instance, MorphAgent [353] introduces dynamic agent profiles that autonomously rewrite themselves based on quantitative metrics evaluating role clarity, differentiation, and task-role alignment. Similarly, NegotiationGym [354] optimizes agent negotiation strategies by iteratively refining prompts based on explicitly encoded utility functions derived from past interactions. In frameworks like COPPER [292], the "Actor" agents utilize self-reflection to generate textual critiques, which are then appended to their contextual prompts to refine future trajectories. While prompt evolution is highly interpretable and avoids catastrophic forgetting, its capability ceiling is strictly bounded by the underlying model's context window and parametric limits.

b) Memory Evolution: Instead of relying solely on immediate context, memory evolution focuses on structurally refining and expanding m_i over continuous executions. This paradigm allows agents to build persistent experience banks that transcend single conversational threads. AgentCourt [355], for example, utilizes adversarial self-play in simulated courtrooms to continuously construct a three-level memory comprising legal statutes, practical experiences, and case precedents. Richelieu [356] internalizes diplomatic experiences by persistently storing sub-goal evaluations and opponent credibility scores, retrieving them based on semantic similarity to guide future negotiations. This approach is not limited to static fact-checking; LIET [357] enables embodied agents to construct offline local utility functions and online collaborative knowledge lists, while AnnaAgent [358] uses a tertiary memory mechanism to achieve dynamic, long-term psychological state evolution across multiple counseling sessions. Memory evolution provides a robust pathway for cumulative, "online-persistent" learning, though it shifts the system's operational bottleneck toward retrieval efficiency and the risk of homogenizing historical biases.

c) Parameter Evolution: Recognizing the strict limitations of textual contexts, the most active area of micro-level research targets the neural parameters θ_i. By employing reinforcement learning (RL) directly on the agent's base formulation, evolution is permanently internalized into model weights. This paradigm frequently leverages internal interaction dynamics to generate reward signals, bypassing the need for human-annotated ground truths. CoMAS [359] drives optimization using intrinsic "interaction rewards" evaluated by a judge module during inter-agent discussions. The Multi-Agent Evolve

framework [360] splits a single LLM into Proposer, Solver, and Judge roles, using objective-relative REINFORCE++ for self-improvement. Beyond task efficiency, parameter evolution is critical for robustness. AdvEvo-MARL [361] internalizes systemic safety by co-evolving attackers and defenders in a multi-agent reinforcement learning environment. Likewise, Epistemic Context Learning [362] uses auxiliary RL rewards to help localized parameters evolve the ability to autonomously discern trustworthy peers from misleading ones, allowing smaller models to outperform history-agnostic larger baselines. Although computationally expensive to train, parameter evolution embeds complex capabilities without inflating runtime inference costs.

d) Tool Use Evolution: The final micro-level target involves autonomously expanding the executable toolset t_i. In open-ended environments, fixed APIs are often insufficient. Frameworks like Towards AGI [363] handle systemic failures by deploying a dedicated code-generation LLM to dynamically synthesize entirely new Python functions and tools, escalating to full parameter-level evolution only if tool synthesis continues to fail. While other works like Odyssey [364] focus on equipping agents with vast pre-constructed skill libraries to navigate environments like Minecraft, autonomous tool evolution represents a crucial step toward agents capable of algorithmic self-expansion. This paradigm provides highly modular and auditable capabilities, but strictly depends on the robust verification of generated code to avoid systemic crashes.

In all four micro-level paradigms, although evolution targets individual agents in isolation, its driving force remains fundamentally embedded within the multi-agent context. Agents draw on peer interactions—including adversarial self-play [355, 361], mutual critique [292, 359], and collaborative knowledge sharing [357]—to produce rich, autonomous feedback signals. Thus, by adjusting its internal components in light of multi-agent dynamics, micro-level evolution steadily enhances an agent's expected utility.

2) Meso-level: Organizational Self-Evolution:

3) Macro-level: Systemic Self-Evolution:

X. Conclusion

The conclusion goes here.

Acknowledgments

This should be a simple paragraph before the References to thank those individuals and institutions who have supported your work on this article.

Appendix
Proof of the Zonklar Equations

Use `\appendix` if you have a single appendix: Do not use `\section` anymore after `\appendix`, only `\section*`. If you have multiple appendixes use `\appendices` then use `\section` to start each appendix. You must declare a `\section` before using any `\subsection` or using `\label` (`\appendices` by itself starts a section numbered zero.)

References Section

You can use a bibliography generated by BibTeX as a .bbl file. BibTeX documentation can be easily obtained at: http://mirror.ctan.org/biblio/bibtex/contrib/doc/ The IEEEtran BibTeX style support page is: http://www.michaelshell.org/tex/ieeetran/bibtex/

Simple References

You can manually copy in the resultant .bbl file and set second argument of `\begin` to the number of references (used to reserve space for the reference number labels box).

References

[1] OpenAI, "Introducing GPT-5," Aug. 2025, [Online]. Available: https://openai.com/index/introducing-gpt-5/. [Online]. Available: https://openai.com/index/introducing-gpt-5/

[2] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[3] A. Yang *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[4] Anthropic, "Introducing Claude Opus 4.6," Feb. 2026, [Online]. Available: https://www.anthropic.com/news/claude-opus-4-6. [Online]. Available: https://www.anthropic.com/news/claude-opus-4-6

[5] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022.

[6] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022.

[7] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[8] D. Guo, D. Yang, H. Zhang *et al.*, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *Nature*, vol. 645, pp. 633–638, 2025.

[9] OpenAI, "Introducing GPT-5.4," Mar. 2026, [Online]. Available: https://openai.com/index/introducing-gpt-5-4/. [Online]. Available: https://openai.com/index/introducing-gpt-5-4/

[10] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers Comput. Sci.*, vol. 18, no. 6, p. 186345, 2024, art. no. 186345.

[11] Z. H. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, Y. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, "The rise and potential of large language model based agents: A survey," *Sci. China Inf. Sci.*, vol. 68, no. 2, p. 121101, 2025, art. no. 121101.

[12] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[13] C. Qian, W. Liu, H. Liu, Y. Chen, J. Dang, Y. Li, Z. Yang, C. Xu, H. Li, X. Cong, Y. Shan, and Z. Liu, "ChatDev: Communicative agents for software development," in *Proc. Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2024, pp. 15 174–15 186.

[14] D. A. Boiko, R. MacKnight, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.

[15] A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller, "Augmenting large language models with chemistry tools," *Nature Mach. Intell.*, vol. 6, no. 5, pp. 525–535, May 2024.

[16] B. Zitkovich, T. J. Joshi, A. Irpan, B. Ichter, J. Hsu, A. Herzog, K. Hausman, K. Gopalakrishnan, C. Fu, P. Florence, C. Finn, K. A. Dubey, D. Driess, T. Ding, K. M. Choromanski, X. Chen, Y. Chebotar, J. Carbajal, N. Brown, A. Brohan, M. G. Arenas, and K. Han, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Proc. Conf. Robot Learn. (CoRL)*, 2023, pp. 2165–2183.

[17] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Inner monologue: Embodied reasoning through planning with language models," in *Proc. Conf. Robot Learn. (CoRL)*, 2023, pp. 1769–1782.

[18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[19] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023.

[20] M. Pan, M. Cemri *et al.*, "Why do multiagent systems fail?" in *Proc. Building Trust Workshop at Int. Conf. Learn. Represent. (ICLR)*, 2025.

[21] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[22] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2024, pp. 8048–8057.

[23] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, C. Zhang, S. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversations," in *Proc. Conf. Lang. Model. (COLM)*, 2024.

[24] C. Park, S. Han, X. Guo, A. Ozdaglar, K. Zhang, and J.-K. Kim, "MAPoRL: Multi-agent post-co-training for collaborative large language models with reinforcement learning," in *Proc. Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2025, pp. 30 215–30 248.

[25] Anthropic, "Introducing the Model Context Protocol," Nov. 2024, [Online]. Available: https://www.anthropic.com/news/model-context-protocol. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[26] Google, "Announcing the Agent2Agent protocol (A2A)," Apr. 2025, [Online]. Available: https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/. [Online]. Available: https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

[27] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, "Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025, pp. 76 583–76 599.

[28] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, and R. Qian, "TRAIL: Trace reasoning and agentic issue localization," *arXiv preprint arXiv:2505.08638*, 2025.

[29] X. Ma, X. Xie, Y. Wang, J. Wang, B. Wu, M. Li, and Q. Wang, "Diagnosing failure root causes in platform-orchestrated agentic systems: Dataset, taxonomy, and benchmark," *arXiv preprint arXiv:2509.23735*, 2025.

[30] Y. Wang, W. Wu, J. Wang, and Q. Wang, "From flat logs to causal graphs: Hierarchical failure attribution for LLM-based multi-agent systems," *arXiv preprint arXiv:2602.23701*, 2026.

[31] H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan, H. Liu, S. Liu, J. Qiu, X. Qi, Q. Ren, Y. Wu, H. Wang, H. Xiao, Y. Zhou, S. Zhang, J. Zhang, J. Xiang, Y. Fang, Q. Zhao, D. Liu, C. Qian, Z. Wang, M. Hu, H. Wang, Q. Wu, H. Ji, and M. Wang, "A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence," *arXiv preprint arXiv:2507.21046*, 2025.

[32] Y. Dang, C. Qian *et al.*, "Multi-agent collaboration via evolving orchestration," in *Adv. Neural Inf. Process. Syst.*, 2025.

[33] K. Zhu, Z. Liu, B. Li, M. Tian, Y. Yang, J. Zhang, P. Han, Q. Xie, F. Cui, W. Zhang, X. Ma, X. Yu, G. Ramesh, J. Wu, Z. Liu, P. Lu, J. Zou, and J. You, "Where LLM agents fail and how they can learn from failures," *arXiv preprint arXiv:2509.25370*, 2025.

[34] Z. Zhang, Q. Dai, X. Bo, C. Ma, R. Li, X. Chen *et al.*, "A survey on the memory mechanism of large language model based agents," *ACM Trans. Inf. Syst.*, vol. 43, no. 6, pp. 1–45, 2025, art. no. 155.

[35] H. Wei, Z. Zhang, S. He, T. Xia, S. Pan, and F. Liu, "PlanGenLLMs: A modern survey of LLM planning capabilities," in *Proc. Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2025, pp. 19 497–19 521.

[36] B. Liu, X. Li *et al.*, "Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems," *arXiv preprint arXiv:2504.01990*, 2025.

[37] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, "A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges," *Vicinagearth*, vol. 1, p. 9, 2024, art. no. 9.

[38] K.-T. Tran *et al.*, "Multi-agent collaboration mechanisms: A survey of LLMs," *arXiv preprint arXiv:2501.06322*, 2025.

[39] B. Yan, Z. Zhou *et al.*, "Beyond self-talk: A communication-centric survey of LLM-based multi-agent systems," *arXiv preprint arXiv:2502.14321*, 2025.

[40] A. Ehtesham *et al.*, "A survey of agent interoperability protocols: MCP, ACP, A2A, and ANP," *arXiv preprint arXiv:2505.02279*, 2025.

[41] S. Han *et al.*, "LLM multi-agent systems: Challenges and open problems," *arXiv preprint arXiv:2402.03578*, 2024.

[42] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li, Z. Ren, N. Aletras, X. Wang, H. Zhou, and Z. Meng, "A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems," *arXiv preprint arXiv:2508.07407*, 2025.

[43] OpenAI, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[44] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[45] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "Webarena: A realistic web environment for building autonomous agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[46] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, Y. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.

[47] J. Huang and K. C.-C. Chang, "Towards reasoning in large language models: A survey," in *Findings of the Association for Computational Linguistics (ACL)*, 2023, pp. 1049–1065.

[48] F. Sun, C. Chen, S. Li *et al.*, "A survey of reasoning large language models," *arXiv preprint arXiv:2502.17419*, 2025.

[49] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[50] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[51] M. Li, S. Miao, and P. Li, "Simple is effective: The roles of graphs and large language models in knowledge-graph-based retrieval-augmented generation," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[52] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, and J. Guo, "Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph," in *International Conference on Learning Representations (ICLR)*, 2024.

[53] S. Feng, W. Shi, Y. Bai, V. Balachandran, T. He, and Y. Tsvetkov, "Knowledge card: Filling llms' knowledge gaps with plug-in specialized language models," in *International Conference on Learning Representations (ICLR)*, 2024.

[54] Y. Zhang, P. Xiao, L. Wang, C. Zhang, M. Fang, Y. Du, Y. Puzyrev, R. Yao, S. Qin, Q. Lin, M. Pechenizkiy, D. Zhang, S. Rajmohan, and Q. Zhang, "RuAG: Learned-rule-augmented generation for large language models," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[55] Z. Zhang, A. Zhang, M. Li, H. Zhao, G. Karypis, and A. Smola, "Multimodal chain-of-thought reasoning in language models," *Transactions on Machine Learning Research (TMLR)*, 2024.

[56] Y. Hu, W. Shi, X. Fu, D. Roth, M. Ostendorf, L. Zettlemoyer, N. A. Smith, and R. Krishna, "Visual sketchpad: Sketching as a visual chain of thought for multimodal language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[57] J. Qi, M. Ding, W. Wang, Y. Bai, Q. Lv, W. Hong, B. Xu, L. Hou, J. Li, Y. Dong, and J. Tang, "CogCoM: A visual language model with chain-of-manipulations reasoning," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[58] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 22 199–22 213.

[59] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[60] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk,

and T. Hoefler, "Graph of thoughts: Solving elaborate problems with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 682–17 690.

[61] DeepSeek-AI, "Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning," *Nature*, 2025, published online.

[62] X. Zhang, C. Du, T. Pang, Q. Liu, W. Gao, and M. Lin, "Chain of preference optimization: Improving chain-of-thought reasoning in LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[63] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM test-time compute optimally can be more effective than scaling model parameters," *arXiv preprint arXiv:2408.03314*, 2024.

[64] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[65] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[66] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[67] P. Wang, L. Li, Z. Shao, R. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui, "Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 9426–9439.

[68] H. Luo, Q. Sun, C. Xu, P. Zhao, J.-G. Lou, C. Tao, X. Geng, Q. Lin, S. Chen, Y. Tang, and D. Zhang, "Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[69] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[70] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 12 076–12 100.

[71] I.-C. Chern, S. Chern, S. Chen, W. Yuan, K. Feng, C. Zhou, J. He, G. Neubig, and P. Liu, "Factool: Factuality detection in generative AI – a tool augmented framework for multi-task and multi-domain scenarios," in *arXiv preprint arXiv:2307.13528*, 2023.

[72] Y. Wang, P. Manakul, N. Georgiou, and M. Gales, "Factcheck-GPT: End-to-end fine-grained document-level fact-checking and correction of LLM output," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[73] P. Manakul, A. Liusie, and M. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 9004–9017.

[74] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in *Proceedings of the International Conference on Learning Representations*, 2023.

[75] X. Du, C. Xiao, and Y. Li, "HaloScope: Harnessing unlabeled LLM generations for hallucination detection," in *Advances in Neural Information Processing Systems*, 2024.

[76] Y. Bang, Z. Ji, A. Schelten, A. Hartshorn, T. Fowler, C. Zhang, N. Cancedda, and P. Fung, "HalluLens: LLM hallucination benchmark," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 24 128–24 156.

[77] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[78] L. Gao, Z. Dai, P. Pasupat, A. Chen, A. T. Chaganty, Y. Fan, V. Zhao, N. Lao, H. Lee, D.-C. Juan, and K. Guu, "RARR: Researching and revising what language models say, using language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023, pp. 16 477–16 508.

[79] Y.-S. Chuang, Y. Xie, H. Luo, Y. Kim, J. Glass, and P. He, "DoLa: Decoding by contrasting layers improves factuality in large language models," in *Proceedings of the International Conference on Learning Representations*, 2024.

[80] W. Shi, X. Han, M. Lewis, Y. Tsvetkov, L. Zettlemoyer, and S. W.-t. Yih, "Trusting your evidence: Hallucinate less with context-aware decoding," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, 2024.

[81] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg, "Inference-time intervention: Eliciting truthful answers from a language model," in *Advances in Neural Information Processing Systems*, 2024.

[82] Anonymous, "SADI: Semantic-aligned dynamic intervention for language model behavior steering," in *Proceedings of the International Conference on Learning Representations*, 2025.

[83] ——, "Alleviating hallucinations in large language models through multi-model contrastive decoding and dynamic hallucination detection," in *Advances in Neural Information Processing Systems*, 2025.

[84] K. Tian, E. Mitchell, H. Yao, C. D. Manning, and C. Finn, "Fine-tuning language models for factuality," in *Proceedings of the International Conference on Learning Representations*, 2024.

[85] H. Zhang, S. Diao, Y. Lin, Y. R. Fung, Q. Lian, X. Wang, Y. Chen, and H. Ji, "R-tuning: Instructing large language models to say "I don't know"," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, 2024.

[86] J. Wu *et al.*, "Mitigating LLM hallucination via behaviorally calibrated reinforcement learning," *arXiv preprint arXiv:2512.19920*, 2025.

[87] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[88] A. Zweiger, J. Pari, H. Guo, E. Akyürek, Y. Kim, and P. Agrawal, "Self-adapting language models," *arXiv preprint arXiv:2506.10943*, 2025.

[89] D. Surís, S. Menon, and C. Vondrick, "ViperGPT: Visual inference via Python execution for reasoning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023.

[90] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive abstractive processing for tree-organized retrieval," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[91] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[92] C. Li, Z. Liang, Y. Wan, Y. Lu, L. Yun, Z. Wen, J. Cheng, Y. Li, and J. Li, "TopViewRS: Vision–language models as top-view spatial reasoners," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2024.

[93] H. Shao, S. Qian, H. Xiao, G. Song, Z. Zong, L. Wang, Y. Liu, and H. Li, "Visual CoT: Advancing multi-modal language models with a comprehensive dataset and benchmark for chain-of-thought reasoning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[94] B. J. Gutiérrez, Y. Yang, Y. Zhong, H. Zhu, N. Peng, and J. Yu, "HippoRAG 2: From RAG to memory," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025.

[95] S. Ong, J. Huang, H. Sidahmed, T. Peng, M. Chen, Y. J. Kwon, B. Kim, G. Thattai, and D. Didericksen, "Do LLM agents have regret? A case study in online learning and games," *arXiv preprint arXiv:2406.10996*, 2025.

[96] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language agent tree search unifies reasoning, acting, and planning in language models," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[97] C. Li, J. Liang, A. Zeng, X. Chen, K. Hausman, D. Sadigh, S. Levine, L. Fei-Fei, F. Xia, and B. Ichter, "Chain of code: Reasoning with a language model-augmented code emulator," *arXiv preprint arXiv:2312.04474*, 2024.

[98] Y. Du, F. Wei, and H. Zhang, "AnyTool: Self-reflective, hierarchical agents for large-scale API calls," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[99] Y. Li, Y. Lin, Z. Lin, J. Wang, L. Ding, D. Tao, and M.-H. Lee, "RAIN: Your language models can align themselves without finetuning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[100] Z. Qi, M. Ma, J. Xu, L. Li, and L. Wang, "Mutual reasoning makes smaller LLMs stronger problem-solvers," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[101] X. Guan, L. Zhang, Y. Liu, N. Shang, Y. Sun, Y. Zhu, F. Yang, and M. Yang, "rStar-Math: Small LLMs can master math reasoning with self-evolved deep thinking," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025.

[102] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM test-time compute optimally can be more effective than scaling model parameters," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[103] W. Yang, Z. Lin, and G. Wang, "Towards thinking-optimal scaling of test-time compute for LLM reasoning," *arXiv preprint arXiv:2502.18080*, 2025.

[104] A. Setlur, C. Nagpal, A. Fisch, X. Geng, J. Eisenstein, R. Agarwal, A. Agarwal, J. Berant, and A. Kumar, "Rewarding progress: Scaling automated process verifiers for LLM reasoning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[105] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy," *Nature*, vol. 630, no. 8017, pp. 625–630, Jun 2024.

[106] L. Cao, "Learn to refuse: Making large language models more controllable and reliable through knowledge scope limitation and refusal mechanism," *arXiv preprint arXiv:2311.01041*, 2023.

[107] G. Grand, A. K. Lew, V. K. Mansinghka, and J. Andreas, "Syntactic and semantic control of large language models via sequential Monte Carlo," *arXiv preprint arXiv:2504.13139*, 2025.

[108] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model based agents," *ACM Transactions on Information Systems*, 2024.

[109] Y. Wu *et al.*, "From human memory to AI memory: A survey on memory mechanisms in the era of LLMs," *arXiv preprint arXiv:2504.15965*, 2025.

[110] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th annual acm symposium on user interface software and technology*, 2023, pp. 1–22.

[111] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2024.

[112] J. Lu, S. An, M. Lin, G. Pergola, Y. He, D. Yin, X. Sun, and Y. Wu, "MemoChat: Tuning LLMs to use memos for consistent long-range open-domain conversation," *arXiv preprint arXiv:2308.08239*, 2023.

[113] Q. Wang *et al.*, "Recursively summarizing enables long-term dialogue memory in large language models," *Neurocomputing*, 2025.

[114] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," *arXiv preprint arXiv:2504.19413*, 2025.

[115] Z. Pan, Q. Wu, H. Jiang, X. Luo, H. Cheng, D. Li, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and J. Gao, "SeCom: On memory construction and retrieval for personalized conversational agents," in *International Conference on Learning Representations (ICLR)*, 2025.

[116] D. Patel *et al.*, "ENGRAM: Effective, lightweight memory orchestration for conversational agents," *arXiv preprint arXiv:2511.12960*, 2025.

[117] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents," *Transactions on Machine Learning Research*, 2024.

[118] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," *arXiv preprint arXiv:2501.13956*, 2025.

[119] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[120] Z. Kang *et al.*, "Crafting personalized agents through retrieval-augmented generation on editable memory graphs," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[121] A. Aadhithya *et al.*, "From isolated conversations to hierarchical schemas: Dynamic tree memory representation for LLMs," in *International Conference on Learning Representations (ICLR)*, 2025.

[122] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-MEM: Agentic memory for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[123] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing large language models with long-term memory," *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[124] B. Pneg *et al.*, "Towards lifelong dialogue agents via timeline-based memory management," in *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

[125] N. Chen *et al.*, "Compress to impress: Unleashing the potential of compressive memory in real-world long-term conversations," in *Proceedings of the International Conference on Computational Linguistics (COLING)*, 2025.

[126] J. Gao *et al.*, "MemOS: An operating system for memory-augmented generation in large language models," *arXiv preprint*, 2025.

[127] S. Yan *et al.*, "MemRL: Self-evolving agents via runtime reinforcement learning on episodic memory," *arXiv preprint arXiv:2601.03192*, 2026.

[128] S. Agashe, J. Han, S. Gan, J. Yang, A. Li, and X. E. Wang, "Agent s: An open agentic framework that uses computers like a human," *arXiv preprint arXiv:2410.08164*, 2025.

[129] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research (TMLR)*, 2024.

[130] J. Xie, K. Zhang, J. Chen, T. Zhu, R. Lou, Y. Tian, Y. Xiao, and Y. Su, "Travelplanner: A benchmark for real-world planning with language agents," in *International Conference on Machine Learning (ICML)*, 2024.

[131] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le, and E. Chi, "Least-to-most prompting enables complex reasoning in large language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[132] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 2609–2634.

[133] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[134] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami, "An llm compiler for parallel function calling," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 24 370–24 391.

[135] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, "Llm-planner: Few-shot grounded planning for embodied agents with large language models," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 2986–2997.

[136] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, "Llm+p: Empowering large language models with optimal planning proficiency," *arXiv preprint arXiv:2304.11477*, 2023.

[137] Y. Hao, Y. Zhang, and C. Fan, "Planning anything with rigor: General-purpose zero-shot planning with llm-based formalized programming," in *International Conference on Learning Representations (ICLR)*, 2025.

[138] X. Ning, Z. Lin, Z. Zhou, Z. Wang, H. Yang, and Y. Wang, "Skeleton-of-thought: Prompting llms for efficient parallel generation," in *International Conference on Learning Representations (ICLR)*, 2024.

[139] L. E. Erdogan *et al.*, "Plan-and-act: Improving planning of agents for long-horizon tasks," *arXiv preprint arXiv:2503.09572*, 2025.

[140] A. Prasad, A. Koller, M. Hartmann, P. Clark, A. Sabharwal, M. Bansal, and C. Baral, "Adapt: As-needed decomposition and planning with language models," in *Findings of the North American Chapter of the Association for Computational Linguistics (NAACL Findings)*, 2024.

[141] R. Yang, J. Chen, Y. Zhang, S. Yuan, A. Chen, K. Richardson, Y. Xiao, and D. Yang, "Selfgoal: Your language agents already know how to achieve high-level goals," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

[142] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better llm agents," in *International Conference on Machine Learning (ICML)*, 2024.

[143] Z. Wang, S. Cai, A. Liu, X. Ma, and Y. Liang, "Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[144] Y. Zhu *et al.*, "Knowagent: Knowledge-augmented planning for llm-based agents," *arXiv preprint arXiv:2403.03101*, 2024.

[145] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[146] Z. Zhao, W. S. Lee, and D. Hsu, "Large language models as commonsense knowledge for large-scale task planning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[147] X. Wang, C. Li, Z. Wang, F. Bai, H. Luo, J. Zhang, N. Jojic, E. P. Xing, and Z. Hu, "Promptagent: Strategic planning with language models enables expert-level prompt optimization," in *International Conference on Learning Representations (ICLR)*, 2024.

[148] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu, "Reasoning with language model is planning with world model," *arXiv preprint arXiv:2305.14992*, 2023.

[149] D. Zhang, S. Zhoubian, Y. Yue, Y. Dong, and J. Tang, "Rest-mcts*: Llm self-training via process reward guided tree search," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[150] J. Zhang *et al.*, "Aflow: Automating agentic workflow generation," in *International Conference on Learning Representations (ICLR)*, 2025.

[151] Y. Zhuang, X. Yu, K. Wang, H. Sun, and C. Zhang, "Toolchain*: Efficient action space navigation in large language models with a* search," in *International Conference on Learning Representations (ICLR)*, 2024.

[152] J. Ye, Z. Wu, J. Gao, Z. Wu, X. Jiang, Z. Li, and L. Kong, "Implicit search via discrete diffusion: A study on chess," in *International Conference on Learning Representations (ICLR)*, 2025.

[153] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, "Understanding the planning of llm agents: A survey," *arXiv preprint arXiv:2402.02716*, 2024.

[154] M. Lee *et al.*, "Planrag: A plan-then-retrieval augmented generation for generative large language models as decision makers," *arXiv preprint arXiv:2406.12430*, 2024.

[155] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[156] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian *et al.*, "Toolllm: Facilitating large language models to master 16000+ real-world apis," in *International Conference on Learning Representations (ICLR)*, 2024.

[157] Z. Liu, T. Hoang, J. Zhang, M. Zhu, T. Lan, S. Kokane, J. Tan, W. Yao, Z. Liu, Y. Feng *et al.*, "Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[158] R. Yang, L. Song, Y. Li, S. Zhao, Y. Ge, X. Li, and Y. Shan, "Gpt4tools: Teaching large language model to use tools via self-instruction," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[159] W. Liu *et al.*, "Toolace: Winning the points of llm function calling," in *International Conference on Learning Representations (ICLR)*, 2025.

[160] Q. Wu, W. Liu, J. Luan, and B. Wang, "Toolplanner: A tool augmented llm for multi granularity instructions with path planning and feedback," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 18 315–18 339.

[161] Z. Qi *et al.*, "Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning," in *International Conference on Learning Representations (ICLR)*, 2025.

[162] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive apis," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[163] S. Hao, T. Liu, Z. Wang, and Z. Hu, "Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[164] S. Yuan, K. Song, J. Chen, X. Tan, Y. Shen, K. Ren, D. Li, and D. Yang, "Easytool: Enhancing llm-based agents with concise tool instruction," in *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025, pp. 951–972.

[165] OpenAI, "Function calling and other api updates," 2023. [Online]. Available: https://openai.com/index/function-calling-and-other-api-updates/

[166] S. G. Patil, H. Mao, C. Cheng-Jie Ji, F. Yan, V. Suresh, I. Stoica, and J. E. Gonzalez, "The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 48 371–48 392.

[167] S. Fan, X. Cong, Y. Fu, Z. Zhang, S. Zhang, Y. Liu, Y. Wu, Y. Lin, Z. Liu, and M. Sun, "Workflowllm: Enhancing workflow orchestration capability of large language models," in *International Conference on Learning Representations (ICLR)*, 2025.

[168] M. Zhuge *et al.*, "Gptswarm: Language agents as optimizable graphs," *arXiv preprint arXiv:2402.16823*, 2024.

[169] X. Liu, X. Xu, X. Song, J. Z. Pan, and P. Zhou, "Gentool: Enhancing tool generalization in language models," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.

[170] D. Mekala, J. Weston, J. Lanchantin, R. Raileanu, A. Shrivastava, and J. P. S. Reddy, "Toolverifier: Generalization to new tools via self-verification," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.

[171] Y. Chen, J. Yoon, D. S. Sachan, Q. Wang, V. Cohen-Addad, M. Bateni, C.-Y. Lee, and T. Pfister, "Re-invoke: Tool invocation rewriting for zero-shot tool retrieval," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 4705–4726.

[172] Z. Shi, Y. Wang, L. Yan, P. Ren, S. Wang, D. Yin, and Z. Ren, "Retrieval models aren't tool-savvy: Benchmarking tool retrieval for large language models," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 24 497–24 524.

[173] Y. Huang *et al.*, "Metatool benchmark for large language models: Deciding whether to use tools and which to use," *arXiv preprint arXiv:2310.03128*, 2024.

[174] Y. Shi *et al.*, "SkillWeaver: Web agents can self-improve by discovering and honing skills," *arXiv preprint arXiv:2504.07079*, 2025.

[175] J. Wang *et al.*, "Reinforcement learning for self-improving agent with skill library," *arXiv preprint arXiv:2512.17102*, 2025.

[176] B. Liu, X. Li, J. Zhang, J. Wang, T. He, S. Hong, H. Liu, S. Zhang, K. Song, K. Zhu *et al.*, "Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems," *arXiv preprint arXiv:2504.01990*, 2025.

[177] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, "A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges," *Vicinagearth*, vol. 1, no. 1, p. 9, 2024.

[178] W. Long, W. Wen, P. Zhai, and L. Zhang, "Role play: Learning adaptive role-specific strategies in multi-agent interactions," *Knowledge-Based Systems*, vol. 324, p. 113811, 2025.

[179] Y. Dong, X. Zhu, Z. Pan, L. Zhu, and Y. Yang, "Villageragent: A graph-based multi-agent framework for coordinating complex task dependencies in minecraft," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 16 290–16 314.

[180] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, "Chateval: Towards better llm-based evaluators through multi-agent debate," in *The Twelfth International Conference on Learning Representations*, 2024.

[181] K. He, D. Fu, L. Yang, W. Xu, and C. Wang, "Selfcorrect-agent: Toward robust and generalizable llm-based agents," *Neurocomputing*, p. 132546, 2025.

[182] Y. Zhong, J. G. Kuba, X. Feng, S. Hu, J. Ji, and Y. Yang, "Heterogeneous-agent reinforcement learning," *Journal of Machine Learning Research*, vol. 25, no. 32, pp. 1–67, 2024.

[183] R. Ye, X. Liu, Q. Wu, X. Pang, Z. Yin, L. Bai, and S. Chen, "X-mas: Towards building multi-agent systems with heterogeneous llms," *arXiv preprint arXiv:2505.16997*, 2025.

[184] X. Guo, D. Shi, J. Yu, and W. Fan, "Heterogeneous multi-agent reinforcement learning for zero-shot scalable collaboration," *Neurocomputing*, vol. 648, p. 130716, 2025.

[185] G. Li, H. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "Camel: Communicative agents for" mind" exploration of large language model society," *Advances in neural information processing systems*, vol. 36, pp. 51 991–52 008, 2023.

[186] T. Yao, Z. Li, and Z. Shen, "Hieramas: Optimizing intra-node llm mixtures and inter-node topology for multi-agent systems," *arXiv preprint arXiv:2602.20229*, 2026.

[187] Y. Zhou and Y. Chen, "Adaptive heterogeneous multi-agent debate for enhanced educational and factual reasoning in large language models," *Journal of King Saud University Computer and Information Sciences*, vol. 37, no. 10, p. 330, 2025.

[188] M. Vaccaro Jr, M. Friday, and A. Zaghi, "Multi-agentic llms for personalizing stem texts," *Applied Sciences*, vol. 15, no. 13, p. 7579, 2025.

[189] Y. Kim, C. Park, H. Jeong, Y. S. Chan, X. Xu, D. McDuff, H. Lee, M. Ghassemi, C. Breazeal, and H. W. Park, "Mdagents: An adaptive collaboration of llms for medical decision-making," *Advances in Neural Information Processing Systems*, vol. 37, pp. 79 410–79 452, 2024.

[190] D. S. Nguyen, M. T. Nguyen, P. T. Nguyen, J. Di Rocco, and D. Di Ruscio, "Automated summarization of software documents: an llm-based multi-agent approach," *Automated Software Engineering*, vol. 33, no. 2, p. 43, 2026.

[191] Z. Li, H. Yang, Y. Liu, Y. Xiang, H. Gao, J. Liu, and J. Liu, "Optdispro: Llm-based multi-agent framework for flexibly adapting heuristic optimal disflow," *IEEE Transactions on Smart Grid*, 2025.

[192] R. Barbosa, R. Santos, and P. Novais, "Collaborative problem-solving with llm: a multi-agent system approach to solve complex tasks using autogen," in *International Conference on Practical Applications of Agents and Multi-Agent Systems*. Springer, 2024, pp. 203–214.

[193] A. Campbell and A. S. Wu, "Multi-agent role allocation: issues, approaches, and multiple perspectives," *Autonomous agents and multi-agent systems*, vol. 22, no. 2, pp. 317–355, 2011.

[194] A. Amayuelas, J. Yang, S. Agashe, A. Nagarajan, A. Antoniades, X. E. Wang, and W. Wang, "Self-resource allocation in multi-agent llm systems," *arXiv preprint arXiv:2504.02051*, 2025.

[195] U. M. Borghoff, P. Bottoni, and R. Pareschi, "An organizational theory for multi-agent interactions integrating human agents, llms, and specialized ai," *Discover Computing*, vol. 28, no. 1, p. 138, 2025.

[196] S. Youwai, D. Phim, V. G. Murcia, and R. C. Onas, "Large language model-based multi-agent systems for automated foundation design: router-driven task classification and expert selection framework," *AI in Civil Engineering*, vol. 5, no. 1, p. 5, 2026.

[197] Y. Yue, G. Zhang, B. Liu, G. Wan, K. Wang, D. Cheng, and Y. Qi, "Masrouter: Learning to route llms for multi-agent systems," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 15 549–15 572.

[198] E. Tarasova, V. Erofeeva, O. Granichin, and K. Chernikov, "Decentralized adaptive task allocation for dynamic multi-agent systems," *Scientific Reports*, vol. 15, no. 1, p. 39226, 2025.

[199] H. Goel, M. Omama, B. Chalaki, V. Tadiparthi, E. M. Pari, and S. P. Chinchali, "R3dm: Enabling role discovery and diversity through dynamics models in multi-agent reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2025, pp. 19 600–19 620.

[200] T. Fei, C. Chen, Y. Pan, M. Zheng, and M. Song, "Codedelegator: Mitigating context pollution via role separation in code-as-action agents," *arXiv preprint arXiv:2601.14914*, 2026.

[201] K. Zhu, Z. Tian, S. Wang, M. Leng, and X. Mao, "Atomizer: An llm-based collaborative multi-agent framework for intent-driven commit untangling," *arXiv preprint arXiv:2601.01233*, 2026.

[202] Y. Chen, Y. Yuan, Z. Zhang, Y. Zheng, J. Liu, F. Ni, and J. Hao, "Sheetagent: A generalist agent for spreadsheet reasoning and manipulation via large language models," in *ICML 2024 Workshop on LLMs and Cognition*, 2024.

[203] D. Jin, Z. Jin, X. Chen, and C. Wang, "Mare: Multi-agents collaboration framework for requirements engineering," *arXiv preprint arXiv:2405.03256*, 2024.

[204] H. Yao, L. Da, V. Nandam, J. Turnau, Z. Liu, L. Pang, and H. Wei, "Comal: Collaborative multi-agent large language models for mixed-autonomy traffic," in *Proceedings of the 2025 SIAM International Conference on Data Mining (SDM)*. SIAM, 2025, pp. 409–418.

[205] Z. Wang, Y. Zhu, H. Zhao, X. Zheng, D. Sui, T. Wang, W. Tang, Y. Wang, E. Harrison, C. Pan *et al.*, "Colacare: Enhancing electronic health record modeling through large language model-driven multi-agent collaboration," in *Proceedings of the ACM on Web Conference 2025*, 2025, pp. 2250–2261.

[206] H. Wei, J. Qiu, H. Yu, and W. Yuan, "Medco: Medical education copilots based on a multi-agent framework," in *European Conference on Computer Vision*. Springer, 2024, pp. 119–135.

[207] S. Jinxin, Z. Jiabao, W. Yilei, W. Xingjiao, L. Jiawen, and H. Liang, "Cgmi: Configurable general multi-agent interaction framework," *arXiv preprint arXiv:2308.12503*, 2023.

[208] Y. Xu, J. Hu, Z. Zhao, Z. Duan, X. Sun, and X. Yang, "Multiagentesc: A llm-based multi-agent collaboration framework for emotional support conversation," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 4665–4681.

[209] G. Chen, S. Dong, Y. Shu, G. Zhang, J. Sesay, B. Karlsson, J. Fu, and Y. Shi, "Autoagents: a framework for automatic agent generation," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024, pp. 22–30.

[210] D. Yang, J. Wei, M. Li, J. Liu, L. Liu, M. Hu, J. He, Y. Ju, W. Zhou, Y. Liu *et al.*, "Medaide: information fusion and anatomy of medical intents via llm-based agent collaboration," *Information Fusion*, p. 103743, 2025.

[211] J. Li, Y. Lai, W. Li, J. Ren, M. Zhang, X. Kang, S. Wang, P. Li, Y.-Q. Zhang, W. Ma *et al.*, "Agent hospital: A simulacrum of hospital with evolvable medical agents," *arXiv preprint arXiv:2405.02957*, 2024.

[212] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian *et al.*, "Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors," in *The Twelfth International Conference on Learning Representations*, 2024.

[213] R. Perera, A. Basnayake, and M. Wickramasinghe, "Auto-scaling llm-based multi-agent systems through dynamic integration of agents," *Frontiers in Artificial Intelligence*, vol. 8, p. 1638227, 2025.

[214] T. Wang, H. Dong, V. Lesser, and C. Zhang, "Roma: multi-agent reinforcement learning with emergent roles," in *Proceedings of the 37th International Conference on Machine Learning*, 2020, pp. 9876–9886.

[215] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," *Advances in neural information processing systems*, vol. 30, 2017.

[216] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls *et al.*, "Value-decomposition networks for cooperative multi-agent learning," *arXiv preprint arXiv:1706.05296*, 2017.

[217] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *Journal of Machine Learning Research*, vol. 21, no. 178, pp. 1–51, 2020.

[218] Z. Zhou, Z. Liu, J. Liu, Q. Shao, Y. Wang, K. Shao, D. Jin, and F. Xu, "Resmas: Resilience optimization in llm-based multi-agent systems," *arXiv preprint arXiv:2601.04694*, 2026.

[219] C. Tu, Z. Yu, J. Huang, F. Huang, Y. Wu, L. Han, L. Wang, and R. Huang, "Adaptive role learning with evolutionary multi-agent reinforcement learning for uav-vehicle collaboration in sparse mobile crowdsensing," *IEEE Internet of Things Journal*, 2025.

[220] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: a survey of progress and challenges," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024, pp. 8048–8057.

[221] B. Yan, Z. Zhou, L. Zhang, L. Zhang, Z. Zhou, D. Miao, Z. Li, C. Li, and X. Zhang, "Beyond self-talk: A communication-centric survey of llm-based multi-agent systems," *arXiv preprint arXiv:2502.14321*, 2025.

[222] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin *et al.*, "Metagpt: Meta programming for a multi-agent collaborative framework," in *The twelfth international conference on learning representations*, 2023.

[223] Z. Wang, S. Moriyama, W.-Y. Wang, B. Gangopadhyay, and S. Takamatsu, "Talk structurally, act hierarchically: A collaborative framework for llm multi-agent systems," *arXiv preprint arXiv:2502.11098*, 2025.

[224] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, and Y. Cheng, "Magis: Llm-based multi-agent framework for github issue resolution," *Advances in Neural Information Processing Systems*, vol. 37, pp. 51 963–51 993, 2024.

[225] T. Ma, Y. Qian, Z. Zhang, Z. Wang, X. Qian, F. Bai, Y. Ding, X. Luo, S. Zhang, K. Murugesan *et al.*, "Autodata: A multi-agent system for open web data collection," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[226] X. Mou, C. Qian, W. Liu, L. Yan, Y. Hu, X. Huang, and Z. Wei, "EcoLANG: Efficient and effective agent communication language induction for social simulation," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 5287–5304.

[227] Q. Wang, T. Wang, Z. Tang, Q. Li, N. Chen, J. Liang, and B. He, "Megaagent: A large-scale autonomous llm-based multi-agent system without predefined sops," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 4998–5036.

[228] G. Zhang, Y. Yue, Z. Li, S. Yun, G. Wan, K. Wang, D. Cheng, J. X. Yu, and T. Chen, "Cut the crap: An economical communication pipeline for llm-based multi-agent systems," in *The Thirteenth International Conference on Learning Representations*, 2025.

[229] W. Fan, T. Tognoli, H. P. Zou, C. Miao, Y. Wang, and X. Zhang, "Todycomm: Task-oriented dynamic communication for multi-round llm-based multi-agent system," *arXiv preprint arXiv:2602.03688*, 2026.

[230] N. Nascimento, P. Alencar, and D. Cowan, "Self-adaptive large language model (llm)-based multiagent systems," in *2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*. IEEE, 2023, pp. 104–109.

[231] J. Chen, S. Saha, and M. Bansal, "Reconcile: Round-table conference improves reasoning via consensus among diverse llms," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 7066–7085.

[232] L. Yang, S. Li, and A. Deng, "Dynamic consensus communication mechanism for large language model-based multi-agent systems," *Journal of Signal Processing Systems*, vol. 98, no. 1, p. 10, 2026.

[233] B. Pan, J. Lu, K. Wang, L. Zheng, Z. Wen, Y. Feng, M. Zhu, and W. Chen, "Agentcoord: Visually exploring coordination strategy for llm-based multi-agent collaboration," *Computers & graphics*, p. 104338, 2025.

[234] C. Zhu, M. Dastani, and S. Wang, "A survey of multi-agent deep reinforcement learning with communication," *Autonomous Agents and Multi-Agent Systems*, vol. 38, no. 1, p. 4, 2024.

[235] S. Shaw, E. Wenzel, A. Walker, and G. Sartoretti, "Formic: Foraging via multiagent rl with implicit communication," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4877–4884, 2022.

[236] A. Buscemi, D. Proverbio, A. Di Stefano, T. A. Han, G. Castignani, and P. Liò, "When numbers start talking: Implicit numerical coordination among llm-based agents," *arXiv preprint arXiv:2601.03846*, 2026.

[237] Y. Wang, fangwei zhong, J. Xu, and Y. Wang, "Tom2c: Target-oriented multi-agent communication and cooperation with theory of mind," in *International Conference on Learning Representations*, 2022.

[238] D. Ge, H. Ji, and B. Huang, "Efficient training in multi-agent reinforcement learning: A communication-free framework for the box-pushing problem," in *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, vol. 89213. American Society of Mechanical Engineers, 2025, p. V02BT02A039.

[239] J. Zou, X. Yang, R. Qiu, G. Li, K. Tieu, P. Lu, K. Shen, H. Tong, Y. Choi, J. He *et al.*, "Latent collaboration in multi-agent systems," *arXiv preprint arXiv:2511.20639*, 2025.

[240] C. Hunt, *TCP/IP network administration*. " O'Reilly Media, Inc.", 2002, vol. 2.

[241] L. DeNardis, *The global war for internet governance*. Yale University Press, 2014.

[242] D. Gourley and B. Totty, *HTTP: the definitive guide*. " O'Reilly Media, Inc.", 2002.

[243] J. Chen, J. Sun, and G. Wang, "From unmanned systems to autonomous intelligent systems," *Engineering*, vol. 12, pp. 16–19, 2022.

[244] I.-L. Stefan and I. Dumitrache, "Interconnected autonomous systems," in *2025 25th International Conference on Control Systems and Computer Science (CSCS)*. IEEE, 2025, pp. 149–154.

[245] Y. Tang and T. Runkler, "Llm-based agentic systems for software engineering: Challenges and opportunities," *arXiv preprint arXiv:2601.09822*, 2026.

[246] Y. Yang, Q. Peng, J. Wang, Y. Wen, and W. Zhang, "Llm-based multi-agent systems: Techniques and business perspectives," *arXiv preprint arXiv:2411.14033*, 2024.

[247] W. Chen, Z. You, R. Li, C. Qian, C. Zhao, C. Yang, R. Xie, Z. Liu, M. Sun *et al.*, "Internet of agents: Weaving a web of heterogeneous agents for collaborative intelligence," in *The Thirteenth International Conference on Learning Representations*, 2025.

[248] D. C. Kwon and X. Zhang, "Cp-agentnet: Autonomous and explainable communication protocol design using generative agents," in *2025 IEEE 33rd International Conference on Network Protocols (ICNP)*. IEEE, 2025, pp. 1–12.

[249] A. Ehtesham, A. Singh, G. K. Gupta, and S. Kumar, "A survey of agent interoperability protocols: Model context protocol (mcp), agent communication protocol (acp), agent-to-agent protocol (a2a), and agent network protocol (anp)," *arXiv preprint arXiv:2505.02279*, 2025.

[250] S. Marro, E. La Malfa, J. Wright, G. Li, N. Shadbolt, M. Wooldridge, and P. Torr, "A scalable communication protocol for networks of large language models," *arXiv preprint arXiv:2410.11905*, 2024.

[251] Anthropic, "Model Context Protocol," 2024. [Online]. Available: https://www.anthropic.com/news/model-contextprotocol

[252] G. Chang, "ANP: Agent Network Protocol," 2024. [Online]. Available: https://www.agent-networkprotocol.com/

[253] Eclipse, "Language Model Operating System (LMOS)," 2025. [Online]. Available: https://eclipse.dev/lmos/

[254] L. F. AI and I. Data, "ACP: Agent Communication Protocol," 2025. [Online]. Available: https://agentcommunicationprotocol.dev/introduction/welcome

[255] AIEngineerFoundation, "Agent Protocol," 2025. [Online]. Available: https://agentprotocol.ai/

[256] J. Liu, Z. Kong, C. Yang, F. Yang, T. Li, P. Dong, J. Nanjekye, H. Tang, G. Yuan, W. Niu *et al.*, "Rcr-router: Efficient role-aware context routing for multi-agent llm systems with structured memory," *arXiv preprint arXiv:2508.04903*, 2025.

[257] J. Zhao, C. Chen, C. Qiao, L. Zheng, M. Han, and Y. L. Y. X. X. X. M. Zhang, "Tcandon-router: Adaptive reasoning router for multi-agent collaboration," *arXiv preprint arXiv:2601.04544*, 2026.

[258] Y. Yang, H. Chai, S. Shao, Y. Song, S. Qi, R. Rui, and W. Zhang, "Agentnet: Decentralized evolutionary coordination for LLM-based multi-agent systems," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[259] R. Zhang, X. Jiang, Z. Yang, Z. Zhang, J. Gao, Y. Xiao, H. Lai, X. Chu, J. Zhao, and Y. Wang, "Stackplanner: A centralized hierarchical multi-agent system with task-experience memory management," *arXiv preprint arXiv:2601.05890*, 2026.

[260] A. Li, Y. Xie, S. Li, F. Tsung, B. Ding, and Y. Li, "Agent-oriented planning in multi-agent systems," in *The Thirteenth International Conference on Learning Representations*, 2025.

[261] Y. Dang, C. Qian, X. Luo, J. Fan, Z. Xie, R. Shi, W. Chen, C. Yang, X. Che, Y. Tian *et al.*, "Multi-agent collaboration via evolving orchestration," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[262] Á. Aso-Mollar and E. Onaindia, "An efficient approach for cooperative multi-agent learning problems," in *2024 IEEE 36th International Conference on Tools with Artificial Intelligence (ICTAI)*. IEEE, 2024, pp. 694–701.

[263] B. Jin, T. Collins, D. Yu, M. Cemri, S. Zhang, M. Li, J. Tang, T. Qin, Z. Xu, J. Lu *et al.*, "Controlling performance and budget of a centralized multi-agent llm system with reinforcement learning," *arXiv preprint arXiv:2511.02755*, 2025.

[264] B. de Mol, D. Barbieri, J. Viebahn, and D. Grossi, "Centrally coordinated multi-agent reinforcement learning for power grid topology control," in *Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems*, 2025, pp. 460–475.

[265] J. Wang, K. Chen, X. Song, K. Zhang, L. Ai, E. Yang, and B. Shi, "Symphony: A decentralized multi-agent framework for scalable collective intelligence," *arXiv preprint arXiv:2508.20019*, 2025.

[266] S. Liu, T. Chen, R. Amiri, and C. Amato, "Learning decentralized llm collaboration with multi-agent actor critic," *arXiv preprint arXiv:2601.21972*, 2026.

[267] H. Yang, J. Chen, M. Siew, T. L. Botran, and C. Joe-Wong, "Llm-powered decentralized generative agents with adaptive hierarchical knowledge graph for cooperative planning," in *The First MARW: Multi-Agent AI in the Real World Workshop at AAAI 2025*.

[268] L. Ratnabala, R. Peter, A. Fedoseev, and D. Tsetserukou, "Hippo-mat: Decentralized task allocation using graphsage and multi-agent deep reinforcement learning," *arXiv preprint arXiv:2503.07662*, 2025.

[269] C. H. Chen and M. F. Shiu, "Agentflow: Resilient adaptive cloud-edge framework for multi-agent coordination," *arXiv preprint arXiv:2505.07603*, 2025.

[270] W. Zhang, C. Cui, Y. Zhao, R. Hu, Y. Liu, Y. Zhou, and B. An, "Agentorchestra: A hierarchical multi-agent framework for general-purpose task solving," *arXiv e-prints*, pp. arXiv–2506, 2025.

[271] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, 2023, pp. 2609–2634.

[272] S. Yu, X. Xu, K. Deng, L. Li, and L. Tian, "Tree of agents: Improving long-context capabilities of large language models through multi-perspective reasoning," *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 4574–4592.

[273] S. Yun, J. Peng, P. Li, W. Fan, J. Chen, J. Zou, G. Li, and T. Chen, "Graph-of-agents: A graph-based framework for multi-agent llm collaboration," in *The Fourteenth International Conference on Learning Representations*, 2026.

[274] G. Yu, "Adaptorch: Task-adaptive multi-agent orchestration in the era of llm performance convergence," *arXiv preprint arXiv:2602.16873*, 2026.

[275] W. Gu, J. Han, H. Wang, X. Li, and B. Cheng, "Explain-analyze-generate: A sequential multi-agent collaboration method for complex reasoning," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 7127–7140.

[276] Y. Zhang, R. Sun, Y. Chen, T. Pfister, R. Zhang, and S. Ö. Arık, "Chain of agents: Large language models collaborating on long-context tasks," *Advances in Neural Information Processing Systems*, vol. 37, pp. 132 208–132 237, 2024.

[277] S. Wang, Z. Tan, Z. Chen, S. Zhou, T. Chen, and J. Li, "Anymac: Cascading flexible multi-agent collaboration via next-agent prediction," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 11 566–11 578.

[278] Z. Wang, Y. Yu, W. Zheng, W. Ma, and M. Zhang, "Macrec: A multi-agent collaboration framework for recommendation," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 2760–2764.

[279] L. Xu, Z. Hu, D. Zhou, H. Ren, Z. Dong, K. Keutzer, S. K. Ng, and J. Feng, "Magic: Investigation of large language model powered multi-agent in cognition, adaptability, rationality and collaboration," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 7315–7332.

[280] A. Barrak, "Traceability and accountability in role-specialized multi-agent llm pipelines," in *2025 40th IEEE/ACM International Conference*

on Automated Software Engineering Workshops (ASEW). IEEE, 2025, pp. 315–322.

[281] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *The Eleventh International Conference on Learning Representations*, 2023.

[282] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Forty-first international conference on machine learning*, 2024.

[283] J. Wang, W. Jue, B. Athiwaratkun, C. Zhang, and J. Zou, "Mixture-of-agents enhances large language model capabilities," in *The Thirteenth International Conference on Learning Representations*, 2025.

[284] E. Zhang, E. Zhu, G. Bansal, A. Fourney, H. Mozannar, and J. Gerrits, "Optimizing sequential multi-step tasks with parallel llm agents," *arXiv preprint arXiv:2507.08944*, 2025.

[285] Y. Li, S. Liu, T. Zheng, and M. Song, "Parallelized planning-acting for efficient llm-based multi-agent systems," *arXiv preprint arXiv:2503.03505*, 2025.

[286] Z. Gu, X. Zhu, Y. Cai, H. Shen, X. Chen, Q. Wang, J. Li, X. Shi, H. Guo, W. Huang *et al.*, "Agentgroupchat-v2: Divide-and-conquer is what llm-based multi-agent system need," *arXiv preprint arXiv:2506.15451*, 2025.

[287] J. Yu, Y. Ding, and H. Sato, "Dyntaskmas: A dynamic task graph-driven framework for asynchronous and parallel llm-based multi-agent systems," in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 35, no. 1, 2025, pp. 288–296.

[288] S. Liu, Y. Lu, S. Chen, X. Hu, J. Zhao, T. Fu, and Y. Zhao, "Drugagent: Automating AI-aided drug discovery programming through LLM multi-agent collaboration," in *2nd AI4Research Workshop: Towards a Knowledge-grounded Scientific Research Lifecycle*, 2024.

[289] C. Zhang, K. Yang, S. Hu, Z. Wang, G. Li, Y. Sun, C. Zhang, Z. Zhang, A. Liu, S.-C. Zhu, X. Chang, J. Zhang, F. Yin, Y. Liang, and Y. Yang, "Proagent: Building proactive cooperative agents with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 38, 2024, pp. 17 591–17 599.

[290] J. Zhang, X. Xu, N. Zhang, R. Liu, B. Hooi, and S. Deng, "Exploring collaboration mechanisms for LLM agents: A social psychology view," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 14 544–14 607.

[291] Z. Li, W. Zhao, and J. Pajarinen, "Cooperative multi-agent planning with adaptive skill synthesis," *arXiv preprint arXiv:2502.10148*, 2025.

[292] X. Bo, Z. Zhang, Q. Dai, X. Feng, L. Wang, R. Li, X. Chen, and J.-R. Wen, "Reflective multi-agent collaboration based on large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 138 595–138 631.

[293] X. Zhao, K. Wang, and W. Peng, "An electoral approach to diversify LLM-based multi-agent collective decision-making," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 2712–2727.

[294] Y. Zhang, X. Yang, S. Feng, D. Wang, Y. Zhang, and K. Song, "Can llms beat humans in debating? a dynamic multi-agent framework for competitive debate," *arXiv preprint arXiv:2408.04472*, 2024.

[295] Y. Fu, H. Peng, T. Khot, and M. Lapata, "Improving language model negotiation with self-play and in-context learning from ai feedback," *arXiv preprint arXiv:2305.10142*, 2023.

[296] S. Mao, Y. Cai, Y. Xia, W. Wu, X. Wang, F. Wang, Q. Guan, T. Ge, and F. Wei, "ALYMPICS: LLM agents meet game theory," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 2845–2866.

[297] L. Cai, J. He, Y. Li, J. Liang, Y. Lin, Z. Quan, Y. Zeng, and J. Xu, "Rtbagent: A llm-based agent system for real-time bidding," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, p. 104–113.

[298] T. Liang, Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, Y. Yang, S. Shi, and Z. Tu, "Encouraging divergent thinking in large language models through multi-agent debate," in *Proceedings of the 2024 conference on empirical methods in natural language processing*, 2024, pp. 17 889–17 904.

[299] R. Ye, Y. Zhang, Y. Zhang, H. Kuang, Z. Wei, and P. Sun, "Multi-agent kto: Reinforcing strategic interactions of large language model in language game," *arXiv preprint arXiv:2501.14225*, 2025.

[300] T. Zhu, B. Chen, J. Zhou, H. Zhou, S. Sankararaman, and X. Dai, "Align: Aligned delegation with performance guarantees for multi-agent llm reasoning," *arXiv preprint arXiv:2602.00127*, 2026.

[301] J. Lee, R. Chang, D. Kwon, H. Singh, and N. Verma, "Gemmas: Graph-based evaluation metrics for multi agent systems," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2025, pp. 1522–1532.

[302] L. Gioacchini, G. Siracusano, D. Sanvito, K. Gashteovski, D. Friede, R. Bifulco, and C. Lawrence, "Agentquest: A modular benchmark framework to measure progress and improve llm agents," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: System Demonstrations)*, 2024, pp. 185–193.

[303] S. Agashe, Y. Fan, A. Reyna, and X. E. Wang, "LLM-coordination: Evaluating and analyzing multi-agent coordination abilities in large language models," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 8053–8072.

[304] J. Chen, X. Hu, S. Liu, S. Huang, W.-W. Tu, Z. He, and L. Wen, "LLMArena: Assessing capabilities of large language models in dynamic multi-agent environments," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 13 055–13 077.

[305] W. Wang, D. Zhang, T. Feng, B. Wang, and J. Tang, "Battleagentbench: A benchmark for evaluating cooperation and competition capabilities of language models in multi-agent systems," *arXiv preprint arXiv:2408.15971*, 2024.

[306] K. Zhu, H. Du, Z. Hong, X. Yang, S. Guo, Z. Wang, Z. Wang, C. Qian, X. Tang, H. Ji, and J. You, "MultiAgentBench : Evaluating the collaboration and competition of LLM agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 8580–8622.

[307] S. Schmidgall, R. Ziaei, C. Harris, E. Reis, J. Jopling, and M. Moor, "Agentclinic: a multimodal agent benchmark to evaluate ai in simulated clinical environments," *arXiv preprint arXiv:2405.07960*, 2025.

[308] Z. Fan, L. Wei, J. Tang, W. Chen, W. Siyuan, Z. Wei, and F. Huang, "AI hospital: Benchmarking large language models in a multi-agent medical interaction simulator," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 10 183–10 213.

[309] R. Shu, N. Das, M. Yuan, M. Sunkara, and Y. Zhang, "Towards effective genai multi-agent collaboration: design and evaluation for enterprise applications," *arXiv preprint arXiv:2412.05449*, 2024.

[310] L. Geng and E. Y. Chang, "Realm-bench: A real-world planning benchmark for llms and multi-agent systems," *arXiv preprint arXiv:2502.18836*, 2025.

[311] J. Zhang, Y. Fan, K. Cai, X. Sun, and K. Wang, "OSC: Cognitive orchestration through dynamic knowledge alignment in multi-agent LLM collaboration," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 6320–6337.

[312] A. A. Jafari, C. Ozcinar, and G. Anbarjafari, "A lightweight modular framework for constructing autonomous agents driven by large language models: Design, implementation, and applications in agentforge," *arXiv preprint arXiv:2601.13383*, 2026.

[313] Y. Liu, J. Cai, Y. Li, Q. Meng, Z. Liu, X. Li, C. Qian, C. Shi, and C. Yang, "Masfactory: A graph-centric framework for orchestrating llm-based multi-agent systems with vibe graphing," *arXiv preprint arXiv:2603.06007*, 2026.

[314] C. P. Bechlioulis and K. J. Kyriakopoulos, "Collaborative multi-robot transportation in obstacle-cluttered environments via implicit communication," *Frontiers in Robotics and AI*, vol. 5, p. 90, 2018.

[315] H. Zhao, X. Zhou, B. Ivanovic, and N. Mehr, "Udon: Uncertainty-weighted distributed optimization for multi-robot neural implicit mapping under extreme communication constraints," *arXiv preprint arXiv:2509.12702*, 2025.

[316] E. Yang and C. Mavrogiannis, "Implicit communication in human-robot collaborative transport," in *2025 20th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. IEEE, 2025, pp. 23–33.

[317] M. Z. Pan, M. Cemri, L. A. Agrawal, S. Yang, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, K. Ramchandran, D. Klein, J. E. Gonzalez, M. Zaharia, and I. Stoica, "Why do multiagent systems fail?" in *Proceedings of the Workshop on Building Trust in Language Models and Applications at the International Conference on Learning Representations (ICLR)*, 2025.

[318] B. Lin, K. Yang, Y. Lai, Y. Zhang, C. Zhang, G. Zhang, X. Yu, M. Yu, X. Wang, and Y. Wang, "Agentask: Multi-agent systems need to ask," *arXiv preprint arXiv:2510.07593*, 2025.

[319] F. Kong, R. Zhang, H. Yin, G. Zhang, X. Zhang, Z. Chen, Z. Zhang, X. Zhang, S.-C. Zhu, and X. Feng, "Aegis: Automated error generation and attribution for multi-agent systems," *arXiv preprint arXiv:2509.14295*, 2025.

[320] K. Zhu, Z. Liu, B. Li, M. Tian, Y. Yang, J. Zhang, P. Han, Q. Xie, F. Cui, W. Zhang *et al.*, "Where llm agents fail and how they can learn from failures," *arXiv preprint arXiv:2509.25370*, 2025.

[321] R. Lu, Y. Li, and Y. Huo, "Exploring autonomous agents: A closer look at why they fail when completing tasks," *arXiv preprint arXiv:2508.13143*, 2025.

[322] X. Ma, X. Xie, Y. Wang, J. Wang, B. Wu, M. Li, and Q. Wang, "Diagnosing failure root causes in platform-orchestrated agentic systems: Dataset, taxonomy, and benchmark," *arXiv preprint arXiv:2509.23735*, 2025.

[323] ——, "Demystifying the lifecycle of failures in platform-orchestrated agentic workflows," *arXiv preprint arXiv:2509.23735*, 2026.

[324] J. Bi, Y. Gao, T. Xu, F. Yang, and M. Yang, "Diagnosing with insights: Structured analysis of agent failures via behavioral abstractions," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026, under review.

[325] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, "Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2025, pp. 76 583—-76 599.

[326] G. Zhang, J. Wang, J. Chen, W. Zhou, K. Wang, and S. Yan, "Agentracer: Who is inducing failure in the llm agentic systems?" *arXiv preprint arXiv:2509.03312*, 2025.

[327] Y. Ge, L. Xie, Z. Li, Y. Pei, and T. Zhang, "Who is introducing the failure? automatically attributing failures of multi-agent systems via spectrum analysis," *arXiv preprint arXiv:2509.13782*, 2025.

[328] Y. Yu, M. Li, S. Xu, J. Fu, X. Hou, F. Lai, and B. Wang, "Correct: Condensed error recognition via knowledge transfer in multi-agent systems," *arXiv preprint arXiv:2509.24088*, 2025.

[329] H. Zhang, Y. Shi, X. Gu, H. You, Z. Zhang, L. Gan, Y. Yuan, and J. Huang, "Graphtracer: Graph-guided failure tracing in llm agents for robust multi-turn deep search," *arXiv preprint arXiv:2510.10581*, 2025.

[330] X. Shen, Q. Zhang, S. Wang, Z. Tan, X. Zhao, L. Yao, V. Tadiparthi, H. N. Mahjoub, E. M. Pari, K. Lee *et al.*, "Metacognitive self-correction for multi-agent system via prototype-guided next-execution reconstruction," *arXiv preprint arXiv:2510.14319*, 2025.

[331] X. Zheng, Z. Chen, C. Lin, H. Wei, H. Chen, W. Cheng, and D. Luo, "Trajectory graph copilot: Pre-action error diagnosis in llm agents," *arXiv preprint arXiv:2505.08638*, 2025.

[332] L. Advani, "Trajectory guard–a lightweight, sequence-aware model for real-time anomaly detection in agentic ai," *arXiv preprint arXiv:2601.00516*, 2026.

[333] R. Sheng, Y. Yang, C. Shi, Y. Lin, Z. Chen, H. Qu, and F. Cheng, "Dills: Interactive diagnosis of llm-based multi-agent systems via layered summary of agent behaviors," *arXiv preprint arXiv:2602.05446*, 2026.

[334] Anonymous, "Seeing the whole elephant: A benchmark for failure attribution in LLM-based multi-agent systems," in *Submitted to ACL Rolling Review - January 2026*, 2026, under review.

[335] A. West, Y. Weng, M. Zhu, Z. Lin, Z. Ning, and Y. Zhang, "Abduct, act, predict: Scaffolding causal inference for automated failure attribution in multi-agent systems," *arXiv preprint arXiv:2509.10401*, 2025.

[336] A. Banerjee, A. Nair, and T. Borogovac, "Where did it all go wrong? a hierarchical look into multi-agent error attribution," *arXiv preprint arXiv:2510.04886*, 2025.

[337] M. Ma, J. Zhang, F. Yang, Y. Kang, Q. Lin, S. Rajmohan, and D. Zhang, "Dover: Intervention-driven auto debugging for llm multi-agent systems," *arXiv preprint arXiv:2512.06749*, 2025.

[338] S. Barke, A. Goyal, A. Khare, A. Singh, S. Nath, and C. Bansal, "Agentrx: Diagnosing ai agent failures from execution trajectories," *arXiv preprint arXiv:2602.02475*, 2026.

[339] X. Wang, M. Yin, E. Koh, and M. D. Dogan, "Xagen: An explainability tool for identifying and correcting failures in multi-agent workflows," *arXiv preprint arXiv:2512.17896*, 2025.

[340] K. Sun, W. Li, B. Dong, Y. Lin, J. Zhang, and B. Shi, "Scope delineation before localization: A two-stage framework for enhancing failure attribution in multi-agent systems," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 39, 2026, pp. 33 108–33 116.

[341] J. Chen, Y. Wang, J. Wang, X. Xie, J. Hu, Q. Wang, and F. Xu, "Understanding individual agent importance in multi-agent system via counterfactual reasoning," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 39, no. 15, 2025, pp. 15 785–15 794.

[342] W. Epperson, G. Bansal, V. C. Dibia, A. Fourney, J. Gerrits, E. Zhu, and S. Amershi, "Interactive debugging and steering of multi-agent ai systems," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–15.

[343] G. Ma, J. Zhu, H. Guo, W. Shi, J. Shen, J. Liu, and Y. Liang, "Automatic failure attribution and critical step prediction method for multi-agent systems based on causal inference," *arXiv preprint arXiv:2509.08682*, 2025.

[344] S. Triantafyllou, A. Sukovic, D. Mandal, and G. Radanovic, "Agent-specific effects: A causal effect propagation analysis in multi-agent mdps," *arXiv preprint arXiv:2310.11334*, 2023.

[345] S. Triantafyllou, A. Sukovic, Y. Zolfimoselo, and G. Radanovic, "Counterfactual effect decomposition in multi-agent sequential decision making," *arXiv preprint arXiv:2410.12539*, 2024.

[346] C. Zhu, S. Hong, J. Wu, K. Chawla, Y. Tang, Y. Yin, N. Wolfe, E. Babinsky, and D. Liu, "Raffles: Reasoning-based attribution of faults for llm systems," in *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 7659–7688.

[347] Y. Wang, W. Wu, J. Wang, and Q. Wang, "From flat logs to causal graphs: Hierarchical failure attribution for llm-based multi-agent systems," *arXiv preprint arXiv:2602.23701*, 2026.

[348] Y. In, M. Tanjim, J. Subramanian, S. Kim, U. Bhattacharya, W. Kim, S. Park, S. Sarkhel, and C. Park, "Rethinking failure attribution in multi-agent systems: A multi-perspective benchmark and evaluation," *arXiv preprint arXiv:2603.25001*, 2026.

[349] E. Bonabeau, M. Dorigo, and G. Theraulaz, *Swarm intelligence: from natural to artificial systems*. Oxford university press, 1999, no. 1.

[350] Y. Kim, K. Gu, C. Park, C. Park, S. Schmidgall, A. A. Heydari, Y. Yan, Z. Zhang, Y. Zhuang, M. Malhotra *et al.*, "Towards a science of scaling agent systems," *arXiv preprint arXiv:2512.08296*, 2025.

[351] S. K. A. Koduri, "Efficiency-first design for llm-based multi-agent systems: A framework and empirical analysis."

[352] R. S. Galhardo, P. J. Hastings, and S. M. Rosenberg, "Mutation as a stress response and the regulation of evolvability," *Critical reviews in biochemistry and molecular biology*, vol. 42, no. 5, pp. 399–435, 2007.

[353] S. Lu, J. Shao, B. Luo, and T. Lin, "Morphagent: Empowering agents through self-evolving profiles and decentralized collaboration," *arXiv preprint arXiv:2410.15048*, 2024.

[354] S. Mangla, C. Hokamp, J. Boylan, D. G. Ghalandari, Y. Jauhari, L. Cassidy, and O. Duffy, "Negotiationgym: Self-optimizing agents in a multi-agent social simulation environment," *arXiv preprint arXiv:2510.04368*, 2025.

[355] G. Chen, L. Fan, Z. Gong, N. Xie, Z. Li, Z. Liu, C. Li, Q. Qu, H. Alinejad-Rokny, S. Ni *et al.*, "Agentcourt: Simulating court with adversarial evolvable lawyer agents," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 5850–5865.

[356] Z. Guan, X. Kong, F. Zhong, and Y. Wang, "Richelieu: Self-evolving llm-based agents for ai diplomacy," *Advances in Neural Information Processing Systems*, vol. 37, pp. 123 471–123 497, 2024.

[357] X. Li, C. Bai, Z. Li, J. Zheng, T. Xiao, and J. Zhang, "Learn as individuals, evolve as a team: Multi-agent llms adaptation in embodied environments," *arXiv preprint arXiv:2506.07232*, 2025.

[358] M. Wang, P. Wang, L. Wu, X. Yang, D. Wang, S. Feng, Y. Chen, B. Wang, and Y. Zhang, "Annaagent: Dynamic evolution agent system with multi-session memory for realistic seeker simulation," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 23 221–23 235.

[359] X. Xue, Y. Zhou, G. Zhang, Z. Zhang, Y. Li, C. Zhang, Z. Yin, P. Torr, W. Ouyang, and L. Bai, "Comas: Co-evolving multi-agent systems via interaction rewards," *arXiv preprint arXiv:2510.08529*, 2025.

[360] Y. Chen, Y. Wang, S. Zhu, H. Yu, T. Feng, M. Zhang, M. Patwary, and J. You, "Multi-agent evolve: Llm self-improve through co-evolution," *arXiv preprint arXiv:2510.23595*, 2025.

[361] Z. Pan, Y. Zhang, Z. Liu, Y. Y. Tang, Z. Zhang, H. Luo, Y. Han, J. Zhang, D. Wu, H.-Y. Chen *et al.*, "Advevo-marl: Shaping internalized safety through adversarial co-evolution in multi-agent reinforcement learning," *arXiv preprint arXiv:2510.01586*, 2025.

[362] R. Zhou, M. Song, X. Wu, S. Cheng, X. Yin, Y. Xie, Z. Hao, W. Hua, L. Pan, S. Poria *et al.*, "Epistemic context learning: Building trust the right way in llm-based multi-agent systems," *arXiv preprint arXiv:2601.21742*, 2026.

[363] I. Kar, S. Zonunpuia, and Z. Ralte, "Towards agi a pragmatic approach towards self evolving agent," *arXiv preprint arXiv:2601.11658*, 2026.

[364] S. Liu, Y. Li, K. Zhang, Z. Cui, W. Fang, Y. Zheng, T. Zheng, and M. Song, "Odyssey: Empowering minecraft agents with open-world skills," *arXiv preprint arXiv:2407.15325*, 2024.